

BK Restaurant Management System

System Design Project Final Report

Project Team Members

Sachin Lakshitha (Student ID: SE/2022/020)

Duvini Nimethra (Student ID: SE/2022/023)

Dinuja Ranaweera (Student ID: SE/2022/026)

Nethmi Navodya (Student ID: SE/2022/030)

Supervisor: Dr. Tiroshan Madushanka

FACULTY OF SCIENCE

Software Engineering Teaching Unit

University of Kelaniya, Sri Lanka

SENG 31242 – System Design Project

FINAL DESIGN REPORT

BK Restaurant Management System

Project: BK Restaurant Management System

GitHub Organisation: bk-restaurant-system

Repository: bk-restaurant-management-system

Programme: B.Sc. (Hons.) in Software Engineering

Year / Semester: Year 3, Semester 1

Course Code: SENG 31242

Date: 2026

Version: 2.0

Declaration

We, the undersigned, hereby declare that this report is our own original work and has not been submitted, in whole or in part, for any other degree or qualification. All sources consulted have been duly acknowledged.

Sachin Lakshitha (SE/2022/020)

Signature: _____ Date: _____

Duvini Nimethra (SE/2022/023)

Signature: _____ Date: _____

Dinuja Ranaweera (SE/2022/026)

Signature: _____ Date: _____

Nethmi Navodya (SE/2022/030)

Signature: _____ Date: _____

Supervisor Declaration

I confirm that I have supervised this project and that the work submitted is satisfactory for the purposes of examination.

Supervisor Name: Dr. Tiroshan Madushanka Date: _____

Signature: _____

Acknowledgements

We would like to express our deepest gratitude to our academic supervisors and the Software Engineering Teaching Unit at the University of Kelaniya for their invaluable guidance. We also extend our sincere thanks to Mr. N. C. Sanjeewa De Silva, the owner of BK Restaurant & BK Bar, for providing us with the opportunity to analyze and design a system for his business, and for his continuous feedback throughout the project lifecycle.

Abstract

The operational workflow across multiple restaurant branches is digitized and centralized by the proposed BK Restaurant Management System. Inefficient manual processes are replaced with an integrated digital platform. Currently, ad-hoc WhatsApp orders, verbal kitchen communication, and physical ledger books for inventory are relied upon by the restaurant, leading to operational errors and a lack of centralized oversight. A comprehensive n-tier architecture is introduced by the proposed system, featuring a web-based Point of Sale (POS), an automated Kitchen Display System (KDS), a Flutter-based mobile application for GPS delivery tracking, and a centralized management dashboard. Over an 8-week period, rigorous fact-finding, requirements engineering, and architectural design phases were executed. Sourced from the Software Requirements Specification (SRS) and Software Design Specification (SDS), a scalable, reliable, and cost-effective

solution utilizing modern web technologies including React, Node.js, and PostgreSQL is provided by the resulting design.

Keywords: System Design, POS, KDS, N-Tier Architecture, React, Node.js, Flutter, Restaurant Management

Table of Contents

1. Introduction
2. System Analysis
3. System Design
4. Conclusion
5. References

List of Figures

1. Figure 1.1: Project Timeline (Gantt Chart)
2. Figure 2.1: Context Diagram
3. Figure 2.2: Use Case Diagram
4. Figure 2.3: Activity Diagram - UC-01
5. Figure 2.4: Activity Diagram - UC-04
6. Figure 2.5: Activity Diagram - UC-05
7. Figure 2.6: Activity Diagram - UC-07
8. Figure 2.7: Activity Diagram - UC-09
9. Figure 2.8: Activity Diagram - UC-10
10. Figure 2.9: Sequence Diagram - UC-01
11. Figure 2.10: Sequence Diagram - UC-02

12. Figure 2.11: Sequence Diagram - UC-03
13. Figure 2.12: Sequence Diagram - UC-04
14. Figure 2.13: Sequence Diagram - UC-06
15. Figure 2.14: Sequence Diagram - UC-08
16. Figure 2.15: Sequence Diagram - UC-09
17. Figure 2.16: Sequence Diagram - UC-12
18. Figure 2.17: Sequence Diagram - UC-14
19. Figure 2.18: Sequence Diagram - UC-15
20. Figure 3.1: Component Diagram
21. Figure 3.2: Deployment Diagram
22. Figure 3.3: Class Diagram / ERD State Behaviors
23. Figure 3.4: Sequence Diagram - UC-02 (POS Order Entry)
24. Figure 3.5: Sequence Diagram - UC-03 (Modify/Cancel Order)
25. Figure 3.6: Sequence Diagram - UC-04 (Kitchen Order Processing)
26. Figure 3.7: Sequence Diagram - UC-07 (Rider Assignment)
27. Figure 3.8: Sequence Diagram - UC-09 (Live Delivery Tracking)
28. Figure 3.9: Sequence Diagram - UC-10 (Inventory Adjustment)
29. Figure 3.10: Use Case Diagram (Role-Based Access)
30. Figure 3.11: Customer Order Creation Flow
31. Figure 3.12: Kitchen Order Processing Flow
32. Figure 3.13: Delivery Rider Flow

- 33. Figure 3.14: Inventory Management Flow
- 34. Figure 3.15: Owner Monitoring Flow
- 35. Figure 3.16: UI - Staff Registration
- 36. Figure 3.17: UI - Employee Dashboard
- 37. Figure 3.18: UI - Manager Dashboard (Global)
- 38. Figure 3.19: UI - Overall Analytics
- 39. Figure 3.20: UI - Inventory Dashboard
- 40. Figure 3.21: UI - Login Screen

List of Tables

- 1. Table 2.1: User Classes and Characteristics
- 2. Table 2.2: Operating Environment
- 3. Table 2.3: Fact-Gathering Techniques
- 4. Table 2.4: Identified Pain Points and Solutions
- 5. Table 2.5: Functional Requirements Summary
- 6. Table 2.6: Non-Functional Requirements Summary

List of Abbreviations

Abbreviation	Definition
API	Application Programming Interface
ADR	Architectural Decision Record
AWS	Amazon Web Services
DB	Database

ERD	Entity-Relationship Diagram
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token – used for secure session authentication.
KDS	Kitchen Display System – a screen in the kitchen showing incoming orders.
NoSQL	Not Only SQL
ODM	Object Document Mapper
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
POS	Point of Sale
RBAC	Role-Based Access Control
REST API	Representational State Transfer Application Programming Interface.
SaaS	Software as a Service
SDS	System Design Specification
SLA	Service Level Agreement
SME	Small and Medium Enterprise.
SPA	Single Page Application
SQL	Structured Query Language

SRS	Software Requirements Specification
UC	Use Case – a description of system interactions with actors.
UML	Unified Modeling Language
UI	User Interface
UX	User Experience

CHAPTER 1: INTRODUCTION

1.1 Background

Efficient order processing, accurate kitchen communication, and reliable delivery logistics are heavily relied upon within the hospitality and restaurant industry. A predominantly manual and fragmented workflow is currently utilized by BK Restaurant and BK Bar, a growing enterprise with branches in Ambalangoda and Kahawa. The agility required to scale and meet modern customer expectations is found to be lacking in this traditional approach to restaurant management. To bridge this gap, modern enterprise systems require centralized digital control to coordinate real-time transaction updates, resource utilization, and client-facing communication.

The primary operations of BK Restaurant consist of handling walk-in dining customers, executing takeaway orders, and managing self-operated delivery fleets. Due to the lack of an integrated technical infrastructure, these operations are conducted independently at each branch. Information silos are created, and significant operational latencies are introduced. By establishing a unified system design, these disparate workflows can be consolidated into a centralized platform.

1.2 Problem

Several critical pain points are presented by the current manual system. Customer orders are primarily received via ad-hoc WhatsApp messages and phone calls, which must be manually processed by cashiers. Once taken, orders are verbally relayed to the kitchen staff, which frequently results in preparation errors and missed modifications. Furthermore, delivery logistics are left unmonitored; once the premises are left by a rider, no visibility

into their location or delivery status is maintained by the branch manager. Finally, inventory is managed using physical ledger books, whereby real-time, centralized oversight of stock levels, sales, and employee attendance across the three branches cannot be maintained by the business owner.

In addition to day-to-day transaction errors, the manual inventory tracking system leads to frequent stock-outs of critical raw materials. Since stock counts are conducted manually at the end of each week, shortages are often only identified when items are completely depleted. Financial performance is also negatively impacted because the business owner is unable to perform real-time audit controls on cash collections and sales summaries, exposing the enterprise to potential losses.

1.3 Aims

The construction of a unified, digital ecosystem that centralizes branch operations is primarily aimed for by the BK Restaurant Management System. Sourced from the requirements and design specifications, the following objectives are established:

Walk-in and external order workflows are to be digitized through a robust Point of Sale (POS) interface.

Verbal miscommunications are to be eliminated by the implementation of a synchronized Kitchen Display System (KDS) refreshing within 2 seconds.

Delivery logistics and tracking are to be automated via a proprietary Rider App with live GPS integration.

A cloud-synced, multi-branch dashboard is to be provided for the owner, enabling real-time attendance auditing, leave management, and sales analytics.

Stock-outs are to be mitigated by transitioning to a real-time digital tracking system with automated low-stock alerts and direct digital supplier requests.

1.4 Scope

The design and requirements specification of the core management platform are encompassed within the scope of this project, as detailed in the Software Requirements Specification (SRS) in Appendix A. The POS, KDS, Rider App, and the centralized

backend APIs required to support role-based access control (RBAC) are included. The scope is extended to inventory stock-out alerts and digital supplier requests, as well as digital leave management. Hardware installation and third-party food delivery app integrations (e.g., UberEats, PickMe) are considered out of scope for the initial phase.

1.5 Organisation of Report

This report is organized into six main chapters. The Introduction is provided in Chapter 1. The Project Methodology is defined in Chapter 2. The System Analysis is detailed in Chapter 3, where fact-finding methods, use case modeling, and requirements engineering are covered. The System Design is presented in Chapter 4, detailing architectural patterns, UML modeling, and database schemas. The Conclusion is provided in Chapter 4. The full SRS and SDS documents are attached as appendices to serve as exhaustive technical sources.

1.6 Project Timeline

The project followed an 8-week agile timeline, tracking the progression from idea discovery to the final system design. The schedule was managed via GitHub Projects, with a breakdown of major milestones illustrated in the Gantt chart below.

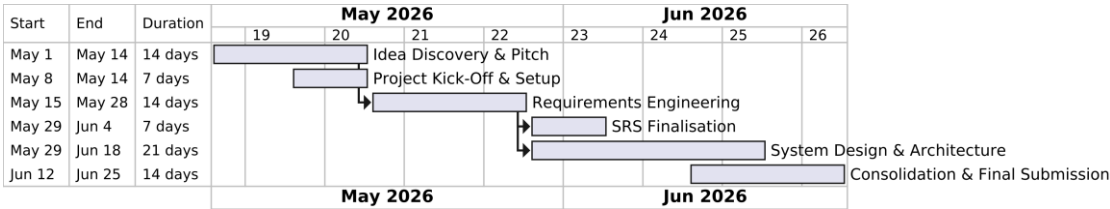


Figure 1.1: Project Timeline Gantt Chart

CHAPTER 2: SYSTEM ANALYSIS

2.1 Fact-Finding Documentation

Rigorous fact-finding was conducted to accurately capture the operational requirements of BK Restaurant. A comprehensive mixed-methods approach was employed, combining structured stakeholder interviews with direct on-site observations. This dual approach ensured that both managerial expectations and ground-level operational bottlenecks were accurately identified. For instance, detailed interviews with the proprietor, Mr. Sanjeewa,

revealed critical issues with WhatsApp-based order ingestion and a severe lack of centralized oversight. Direct observation confirmed these claims, noting significant manual transcription delays. Two formal techniques were employed and documented:

Technique	Description	Artefacts Produced
Client Interviews	Structured interviews were conducted with Mr. N. C. Sanjeewa De Silva to understand current workflows and desired outcomes.	Interview transcripts, pain-point list.
On-Site Observation	Direct observation of the kitchen and cashier workflows was executed during peak hours.	Process notes, timing data, paper forms.

2.2 Existing System Analysis & Pain Points

The critical pain points of the manual system were identified and mapped to proposed system resolutions.

ID	Pain Point	Impact	Proposed Resolution
P-01	Ad-hoc WhatsApp orders are manually transcribed.	Preparation errors and delayed orders.	A unified digital POS interface is implemented.
P-02	Kitchen communication relies on verbal shouting.	Missed modifiers and high stress.	An automated Kitchen Display System (KDS) is deployed.
P-03	Delivery riders cannot be tracked once dispatched.	Lack of visibility for the manager.	GPS tracking via a Flutter-based Rider App is integrated.
P-04	Inventory is managed via physical ledger books.	Frequent stock-outs and cash leakage.	Real-time digital inventory tracking is established.

2.3 User Classes and Characteristics

User profiles were synthesized from the fact-finding phase to guide system design.

Persona	Demographics & Tech Level	Responsibilities & Pain Points
Sanjeewa (Owner)	45 yrs, Moderate Tech	Needs multi-branch oversight. Pain point: Cash leakage and inventory blind spots.
Kasun (Manager)	32 yrs, High Tech	Handles local stock and shifts. Pain point: Surprised by stock-outs mid-shift.
Nimali (Cashier)	22 yrs, High Tech	Inputs orders under high pressure. Pain point: WhatsApp order transcription errors.
Ruwan (Rider)	26 yrs, Moderate Tech	Outdoor delivery. Pain point: No structured routing, reliant on phone calls.

2.4 Functional Requirements

The core functional requirements (FRs) were rigorously mapped to their source and verification methods.

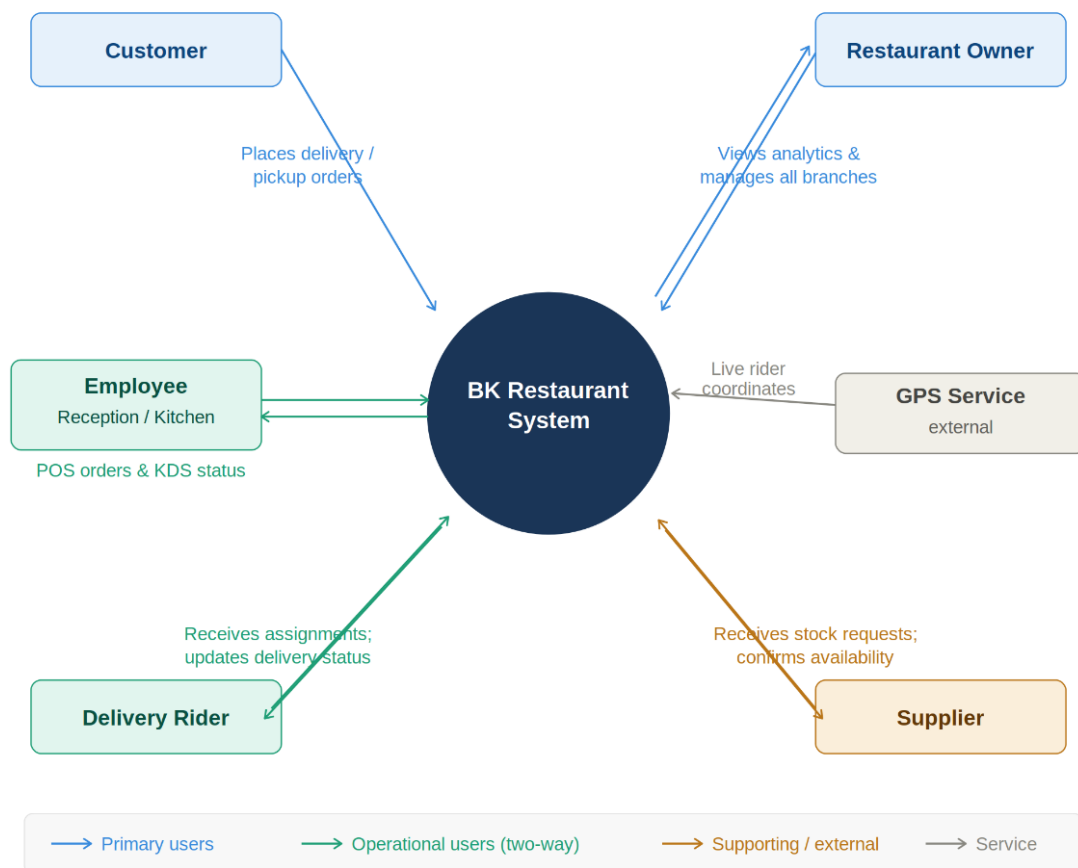
ID	Requirement (Title & Description)	Priority	Source	Test
FR-01	Central Dashboard: A unified dashboard for multi-branch tracking must be provided.	High	Client Interview	TC-FR-01
FR-02	POS Order Entry: Orders must be permitted to be input swiftly by cashiers.	High	Observation	TC-FR-02
FR-03	Kitchen Display Sync: The KDS is	High	Survey	TC-FR-

	required to update within 2s of order placement.			03
FR-04	Rider GPS Tracking: Real-time tracking of assigned delivery riders must be enabled.	Medium	Client Interview	TC-FR-04
FR-05	Inventory Alerts: Automated low-stock notifications must be transmitted to managers.	Medium	Competitor Analysis	TC-FR-05

2.5 System Boundary Diagrams

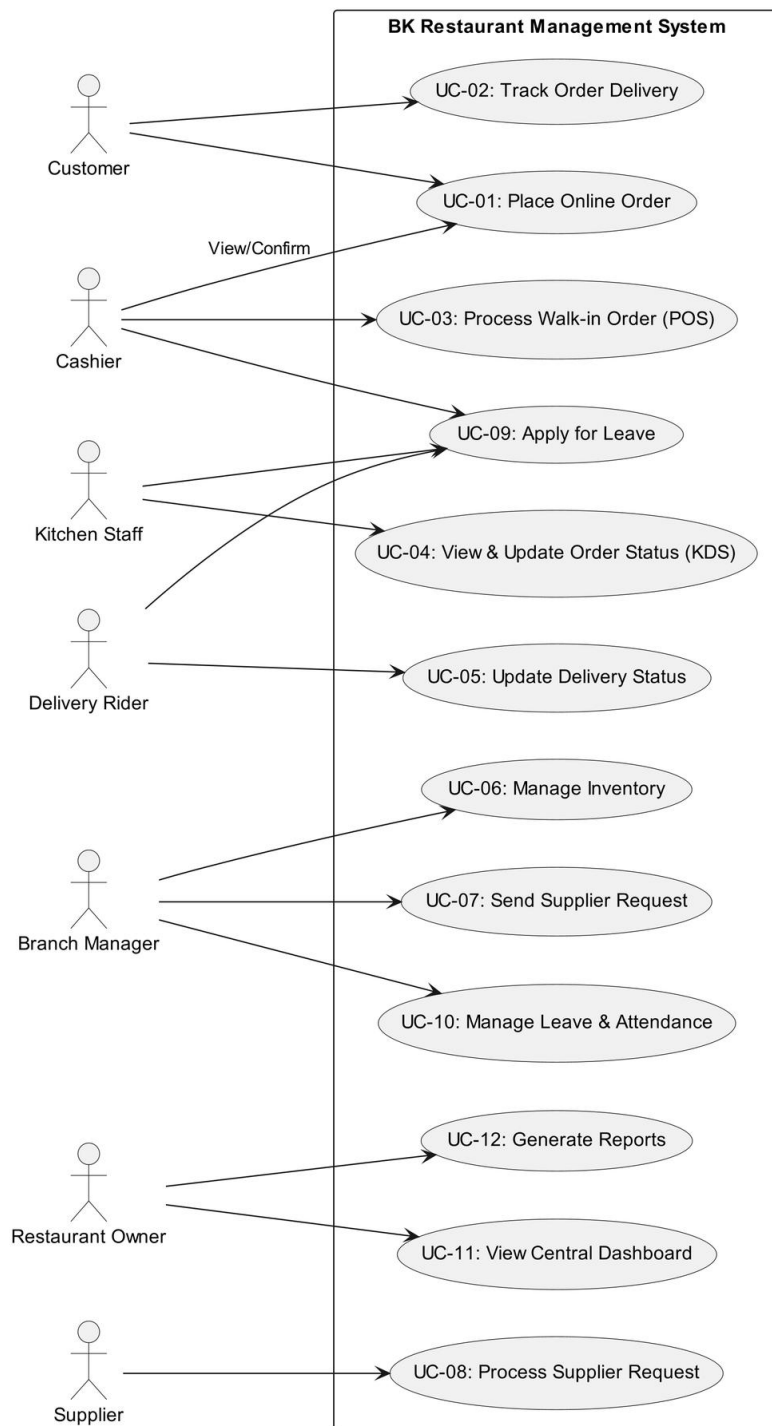
The overall system boundaries and external entity interactions were modeled using Context and Use Case diagrams.

Figure 2.1: Context Diagram



Context Diagram

Figure 2.2: Use Case Diagram



Use Case Diagram

2.6 Use Cases

The complete scope of system interactions was modeled through detailed use cases. Explicit business rules were embedded within each use case.

UC-01: Log In and Access Role-Based Features

Field	Content
Use Case ID	UC-01
Use Case Name	Log In and Access Role-Based Features
Actor(s)	Restaurant Owner, Branch Managers, Employees/Cashiers, Delivery Riders, Suppliers
Preconditions	User is registered in the system and is not currently logged in.
Postconditions Success	– User is authenticated and redirected to their role-specific dashboard.
Postconditions Failure	– Authentication fails; user remains on the login page.
Main Flow	1. User navigates to the login screen. 2. User enters username and password. 3. System validates credentials. 4. System retrieves user role. 5. System grants access to role-based features.
Alternative Flow 1	At step 3: If credentials are invalid, system displays an 'Invalid login' error message.
Alternative Flow 2	At step 2: User clicks 'Forgot Password', redirecting to password reset flow.
Business Rules	BR-01: Users can only access modules permitted by their assigned role.

UC-02: Create Customer Orders

Field	Content
Use Case ID	UC-02
Use Case Name	Create Customer Orders
Actor(s)	Employees/Cashiers
Preconditions	Employee is logged in to the POS module.
Postconditions – Success	Order is recorded, inventory is marked for update, and kitchen is notified.
Postconditions – Failure	Order is discarded; no inventory changes occur.
Main Flow	1. Employee initiates new order. 2. Employee selects order type (Dine-in, Takeaway, Delivery). 3. Employee adds menu items. 4. System calculates total. 5. Employee confirms order and accepts payment. 6. System saves order.
Alternative Flow 1	At step 3: If an item is out of stock, system prevents adding it and displays an alert.
Alternative Flow 2	None
Business Rules	BR-02: Order total must be greater than zero.

UC-03: Modify and Cancel Orders

Field	Content
Use Case ID	UC-03
Use Case Name	Modify and Cancel Orders
Actor(s)	Employees/Cashiers
Preconditions	Employee is logged in; an active order exists.

Postconditions Success	–	Order is successfully updated or canceled; system logs the change.
Postconditions Failure	–	Order remains in its previous state.
Main Flow		1. Employee searches for an active order. 2. System displays order details. 3. Employee selects 'Modify' or 'Cancel'. 4. Employee makes changes or confirms cancellation. 5. System updates order status and recalculates totals if modified.
Alternative Flow 1		At step 3: If order is already in 'Dispatched' or 'Completed' status, system denies modification.
Alternative Flow 2		None
Business Rules		BR-03: Orders cannot be canceled once they are dispatched.

UC-04: Update Order Status

Field	Content
Use Case ID	UC-04
Use Case Name	Update Order Status
Actor(s)	Employees/Cashiers
Preconditions	Employee is logged in; active orders exist in the queue.
Postconditions Success	– Order status is updated (e.g., Preparing, Ready, Completed).
Postconditions Failure	– Status update fails; error logged.
Main Flow	1. Employee views active order queue. 2. Employee selects an order. 3. Employee changes status from 'Pending' to 'Preparing'. 4. System saves the new status. 5. Status updates on real-time

	displays.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-04: Status progression must follow a logical sequence (Pending -> Preparing -> Ready).

UC-05: Generate Bills and Invoices

Field	Content
Use Case ID	UC-05
Use Case Name	Generate Bills and Invoices
Actor(s)	Employees/Cashiers
Preconditions	Order has been created and confirmed.
Postconditions Success	– Digital bill/invoice is generated and stored in order history.
Postconditions Failure	– Invoice generation fails; user is prompted to retry.
Main Flow	1. Employee finalizes an order. 2. System calculates final amount including taxes/fees. 3. System generates a digital invoice. 4. System offers option to print or email the invoice. 5. System archives invoice in order history.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-05: Invoices must include a unique transaction ID and timestamp.

UC-06: Place External Orders

Field	Content
Use Case ID	UC-06
Use Case Name	Place External Orders
Actor(s)	Customers
Preconditions	Customer has access to phone or WhatsApp.
Postconditions – Success	Order is received by the restaurant and entered into the system.
Postconditions – Failure	Order is not placed (e.g., outside delivery hours).
Main Flow	1. Customer contacts restaurant via Phone/WhatsApp. 2. Customer provides order details and delivery address. 3. Employee enters details into the system (invoking UC-02). 4. Customer receives confirmation.
Alternative Flow 1	At step 2: If address is outside delivery zone, customer is informed.
Alternative Flow 2	None
Business Rules	BR-06: Delivery orders require a valid contact number and address.

UC-07: Receive Delivery Assignments

Field	Content
Use Case ID	UC-07
Use Case Name	Receive Delivery Assignments
Actor(s)	Delivery Riders

Preconditions	Rider is logged into the mobile app and marked as 'Available'.
Postconditions Success	– Order is assigned to the rider; order status becomes 'Dispatched'.
Postconditions Failure	– Assignment fails; order remains 'Ready for Delivery'.
Main Flow	1. System identifies a 'Ready' delivery order. 2. System assigns order to an available rider. 3. Rider receives notification on mobile app. 4. Rider accepts assignment. 5. System updates order status to 'Dispatched'.
Alternative Flow 1	At step 4: Rider rejects assignment; system reassigns to the next available rider.
Alternative Flow 2	None
Business Rules	BR-07: Riders can only be assigned orders from their designated branch.

UC-08: Update Delivery Status via Mobile

Field	Content
Use Case ID	UC-08
Use Case Name	Update Delivery Status via Mobile
Actor(s)	Delivery Riders
Preconditions	Rider has an active assigned delivery.
Postconditions Success	– Delivery is marked 'Completed'; customer notified.
Postconditions Failure	– Status remains unchanged; rider prompted to retry.

Main Flow	1. Rider arrives at delivery location. 2. Rider hands over the order. 3. Rider opens mobile app. 4. Rider marks order as 'Delivered'. 5. System logs completion time and updates central database.
Alternative Flow 1	At step 2: Customer is unavailable; rider marks order as 'Delivery Failed'.
Alternative Flow 2	None
Business Rules	BR-08: Riders must be within reasonable GPS proximity to the delivery address to mark as delivered.

UC-09: Monitor Delivery Progress

Field	Content
Use Case ID	UC-09
Use Case Name	Monitor Delivery Progress
Actor(s)	Restaurant Owner
Preconditions	Owner is logged in; deliveries are currently active.
Postconditions Success	– Owner successfully views real-time delivery locations.
Postconditions Failure	– Map or tracking data fails to load.
Main Flow	1. Owner navigates to 'Delivery Tracking' dashboard. 2. System retrieves GPS coordinates of active riders. 3. System overlays rider locations and estimated delivery times on a map view. 4. Owner selects a specific rider for detailed order info.
Alternative Flow 1	At step 2: If a rider's GPS is off, system displays their last known location and a warning.

Alternative Flow 2	None
Business Rules	BR-09: GPS data must be updated at least every 60 seconds.

UC-10: Manage Local Inventory

Field	Content
Use Case ID	UC-10
Use Case Name	Manage Local Inventory
Actor(s)	Branch Managers
Preconditions	Branch Manager is logged into the inventory module.
Postconditions Success	– Inventory counts are accurately adjusted and saved.
Postconditions Failure	– Inventory update fails; previous counts remain.
Main Flow	1. Manager navigates to 'Inventory Management'. 2. System displays current stock levels. 3. Manager performs manual stock count. 4. Manager enters adjustments into the system. 5. System saves updated inventory records.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-10: Inventory updates automatically after sales; manual adjustments require a logged reason.

UC-11: Review Low-Stock Alerts

Field	Content
Use Case ID	UC-11

Use Case Name	Review Low-Stock Alerts
Actor(s)	Branch Managers
Preconditions	Inventory item falls below the defined threshold.
Postconditions Success	– Manager acknowledges alert and initiates a stock request.
Postconditions Failure	– Manager ignores alert; item risks stocking out.
Main Flow	1. System detects an item count below minimum threshold. 2. System generates a low-stock alert. 3. Manager reviews alerts on dashboard. 4. Manager acknowledges the alert. 5. Manager proceeds to create a stock request.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-11: Alerts remain active until inventory count is increased above the threshold.

UC-12: Process Stock Requests

Field	Content
Use Case ID	UC-12
Use Case Name	Process Stock Requests
Actor(s)	Branch Managers, Suppliers
Preconditions	Manager has identified a need for supplies.
Postconditions Success	– Stock request is submitted and fulfilled by supplier.

Postconditions Failure	– Stock request is rejected or fails to send.
Main Flow	1. Manager creates a new stock request. 2. Manager adds items and quantities. 3. System sends request to corresponding Supplier. 4. Supplier logs in and reviews request. 5. Supplier approves and dispatches items. 6. Manager confirms receipt.
Alternative Flow 1	At step 5: Supplier lacks stock and rejects/modifies the request.
Alternative Flow 2	None
Business Rules	BR-12: Received stock automatically updates branch inventory counts.

UC-13: Manage Employee Shifts and Schedules

Field	Content
Use Case ID	UC-13
Use Case Name	Manage Employee Shifts and Schedules
Actor(s)	Branch Managers
Preconditions	Manager is logged into the employee module.
Postconditions Success	– Weekly schedule is published and accessible to employees.
Postconditions Failure	– Schedule is not saved.
Main Flow	1. Manager opens the scheduling tool. 2. Manager assigns employees to specific shifts. 3. System checks for scheduling conflicts. 4. Manager finalizes the schedule. 5. System notifies employees of their upcoming shifts.

Alternative Flow 1	At step 3: System detects an overlapping shift and prompts manager to resolve it.
Alternative Flow 2	None
Business Rules	BR-13: Employees cannot be scheduled for overlapping shifts.

UC-14: Record Employee Attendance

Field	Content
Use Case ID	UC-14
Use Case Name	Record Employee Attendance
Actor(s)	Employees/Cashiers
Preconditions	Employee is physically at the branch and has system access.
Postconditions Success	– Clock-in/clock-out time is recorded in the system.
Postconditions Failure	– Attendance record fails to save.
Main Flow	1. Employee accesses the attendance module. 2. Employee enters unique pin or credentials. 3. Employee selects 'Clock In' (or 'Clock Out'). 4. System logs the timestamp and employee ID. 5. System displays confirmation.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-14: Clock-in must occur within the designated branch network or location.

UC-15: Monitor Centralized Branch Operations & Analytics

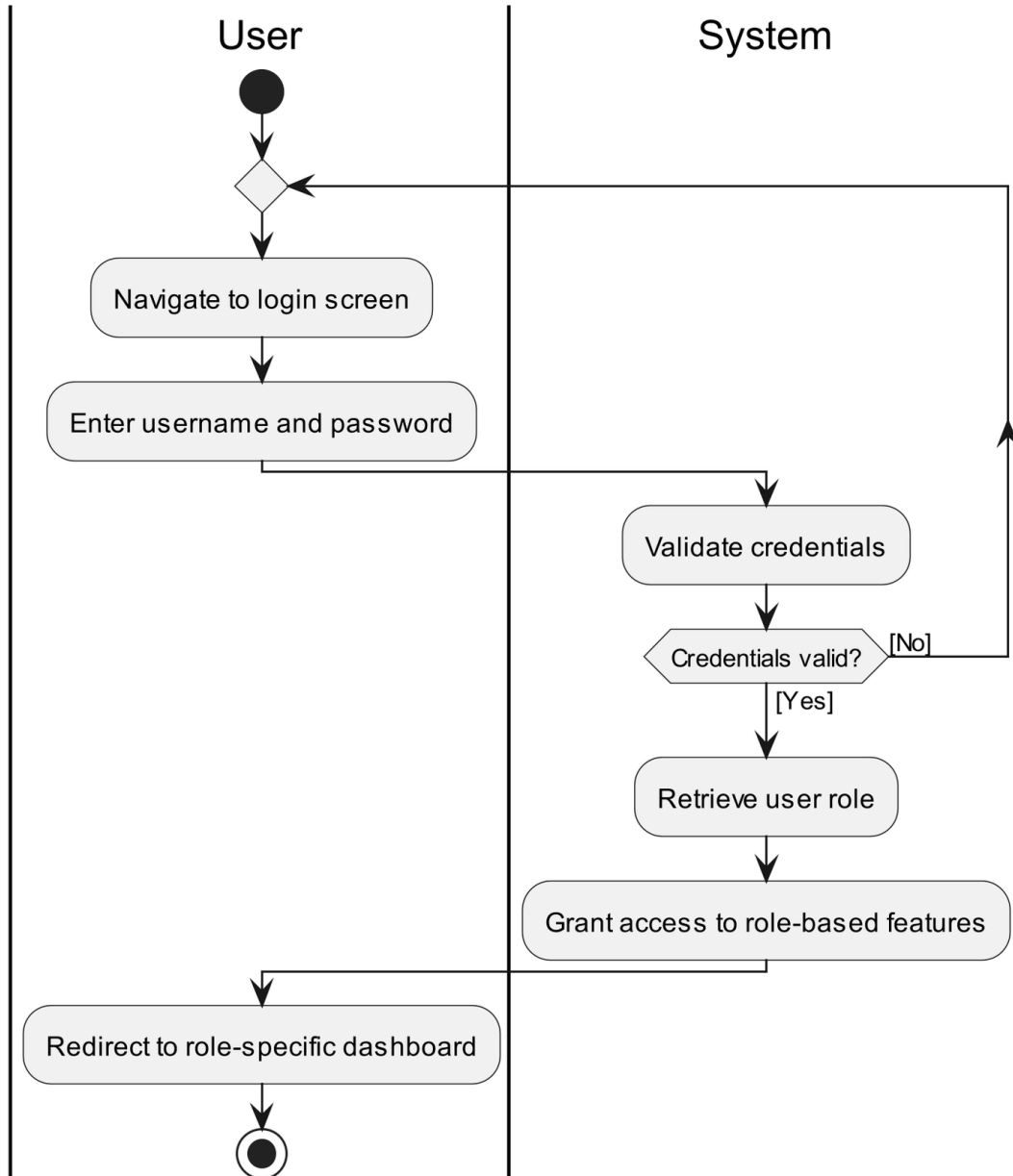
Field	Content
Use Case ID	UC-15
Use Case Name	Monitor Centralized Branch Operations & Analytics
Actor(s)	Restaurant Owner
Preconditions	Owner is logged into the central admin dashboard.
Postconditions – Success	Owner successfully generates and views performance reports.
Postconditions – Failure	Data fails to aggregate; dashboard displays an error.
Main Flow	1. Owner opens the analytics dashboard. 2. System aggregates data from all branches. 3. System displays sales, inventory, and delivery metrics. 4. Owner applies filters (e.g., date range, specific branch). 5. System updates visual charts and data tables.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	BR-15: Analytics dashboard is strictly restricted to the Restaurant Owner role.

2.7 Activity Diagrams

The dynamic aspects of the system were modeled via activity diagrams.

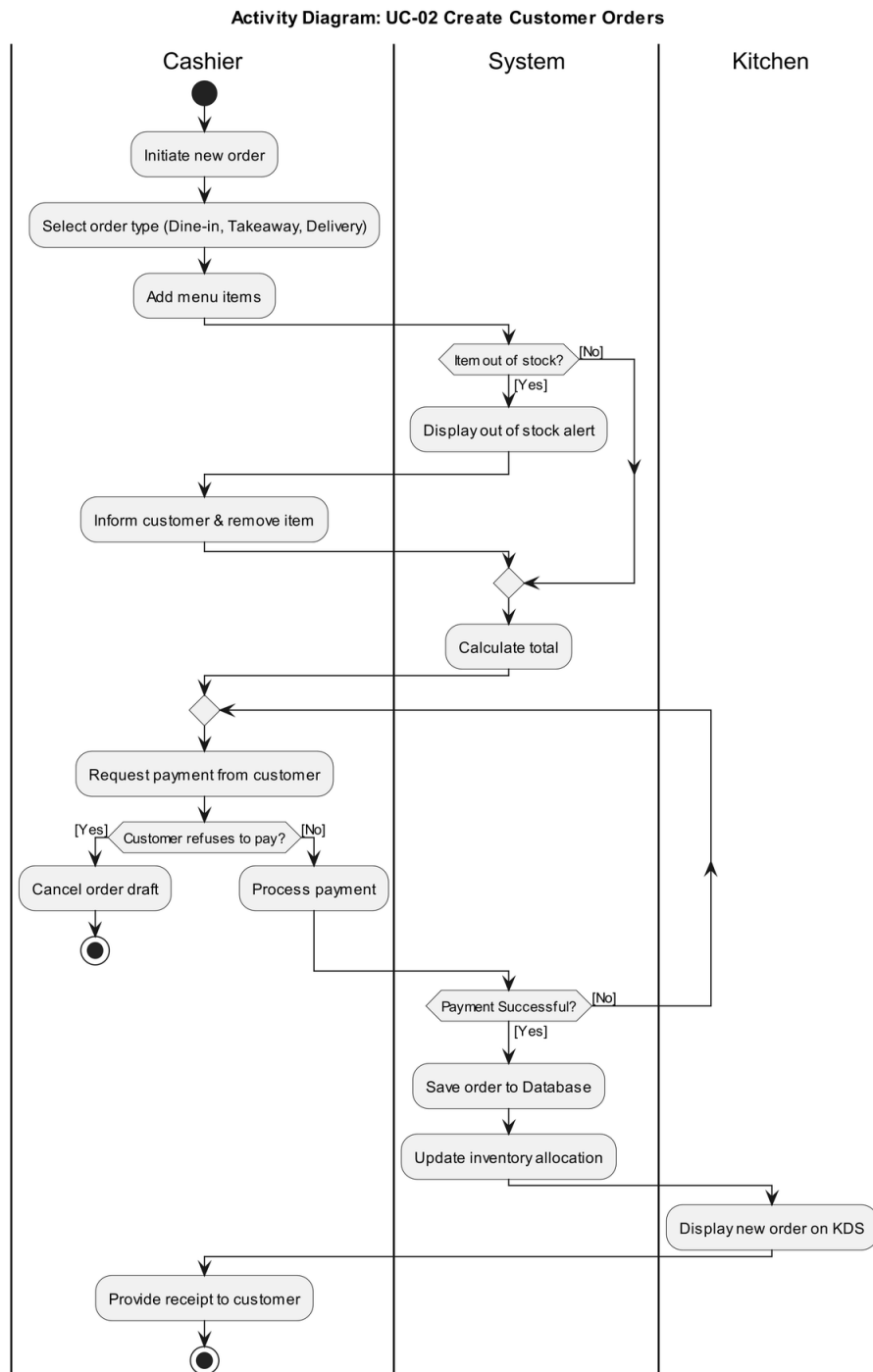
Figure 2.3: Activity Diagram - UC-01

Activity Diagram: UC-01 Log In and Access Role-Based Features



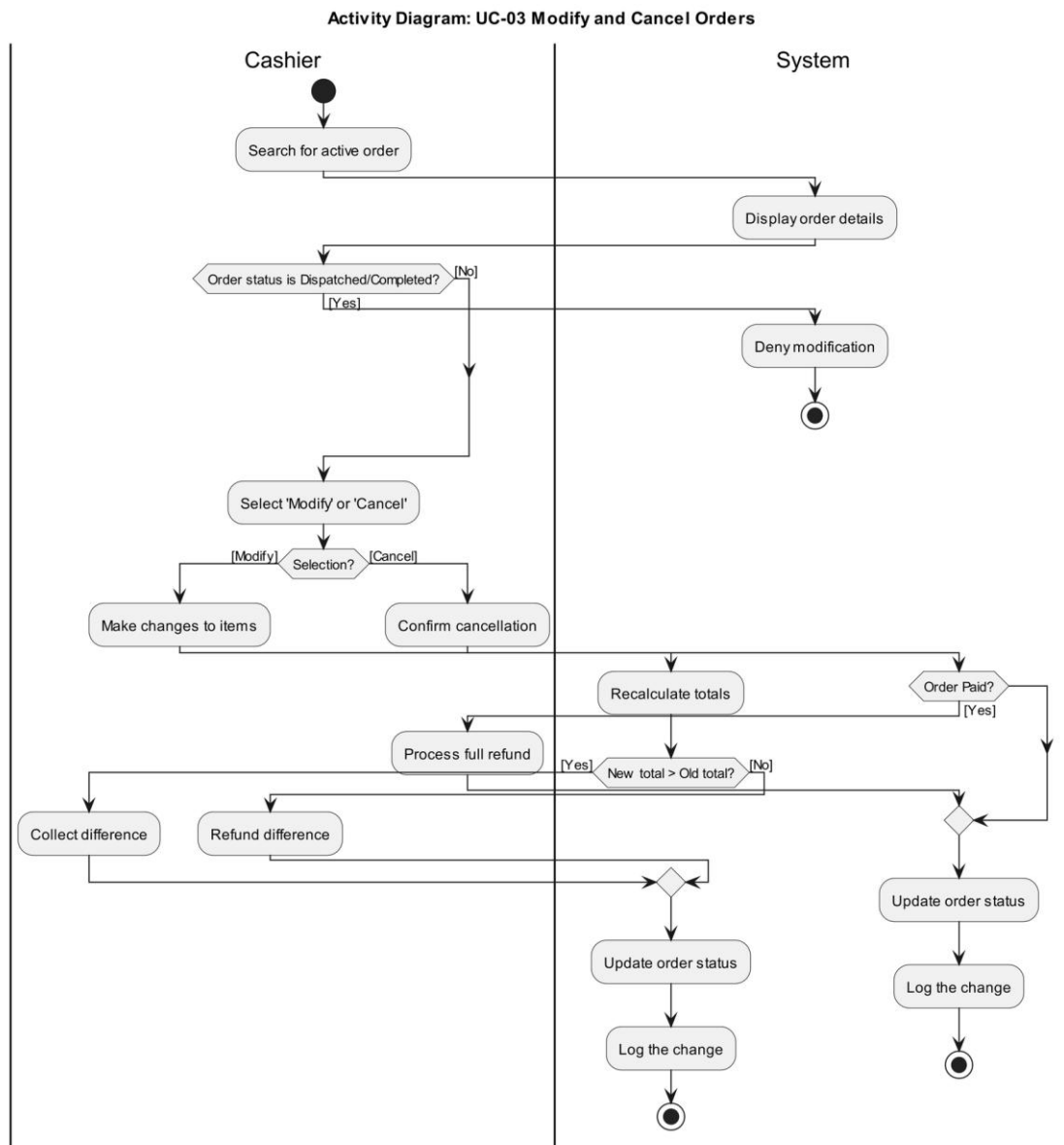
AD-01

Figure 2.4: Activity Diagram - UC-02



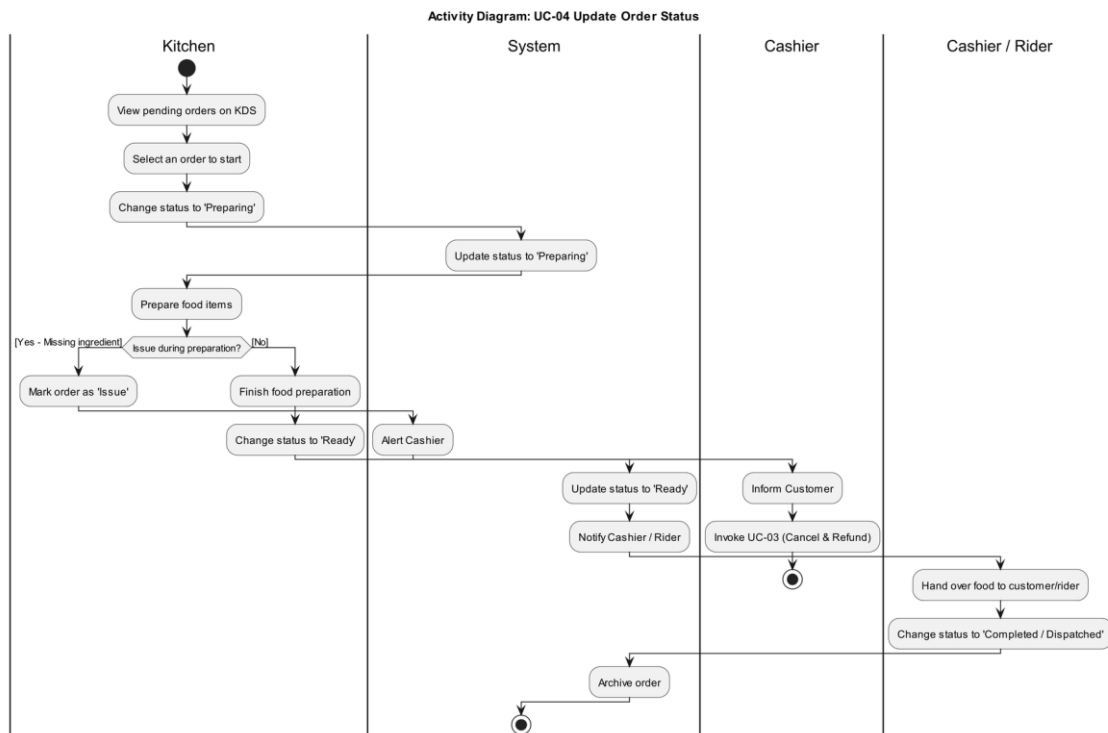
AD-02

Figure 2.5: Activity Diagram - UC-03



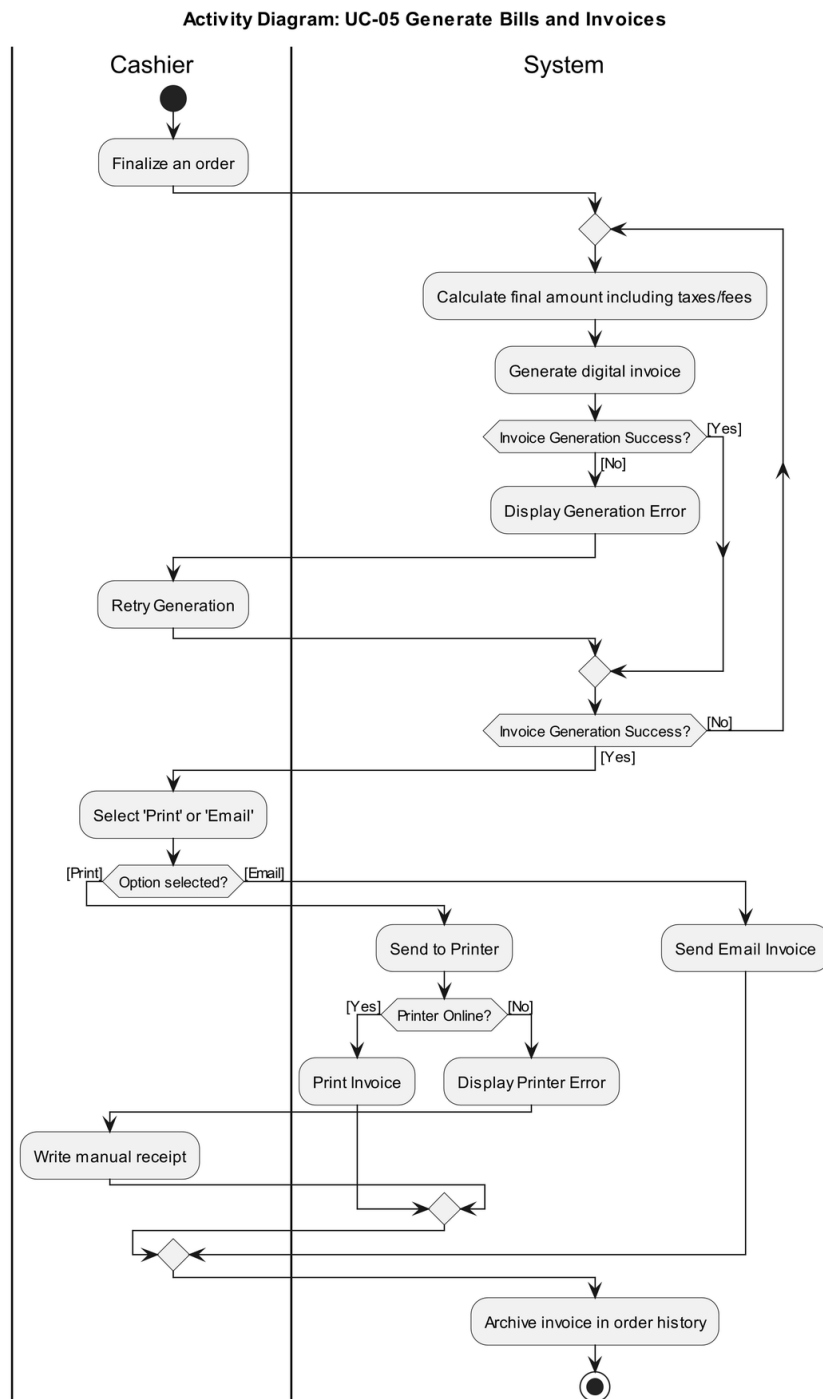
AD-03

Figure 2.6: Activity Diagram - UC-04



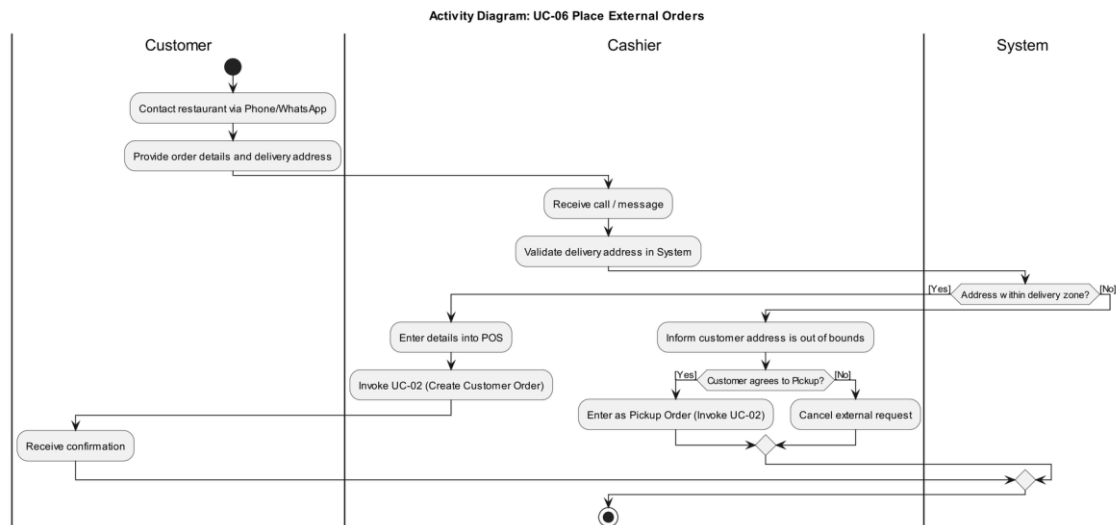
AD-04

Figure 2.7: Activity Diagram - UC-05



AD-05

Figure 2.8: Activity Diagram - UC-06



AD-06

Figure 2.9: Activity Diagram - UC-07

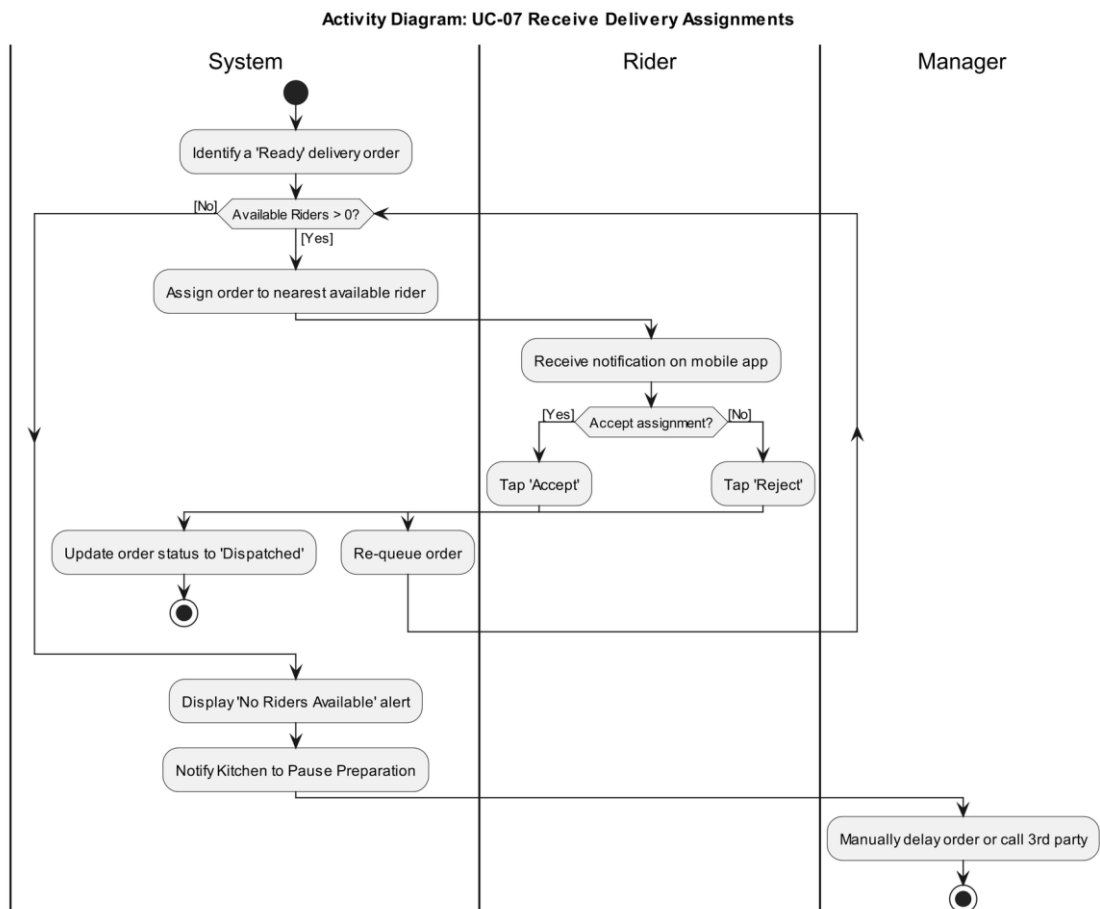


Figure 2.10: Activity Diagram - UC-08

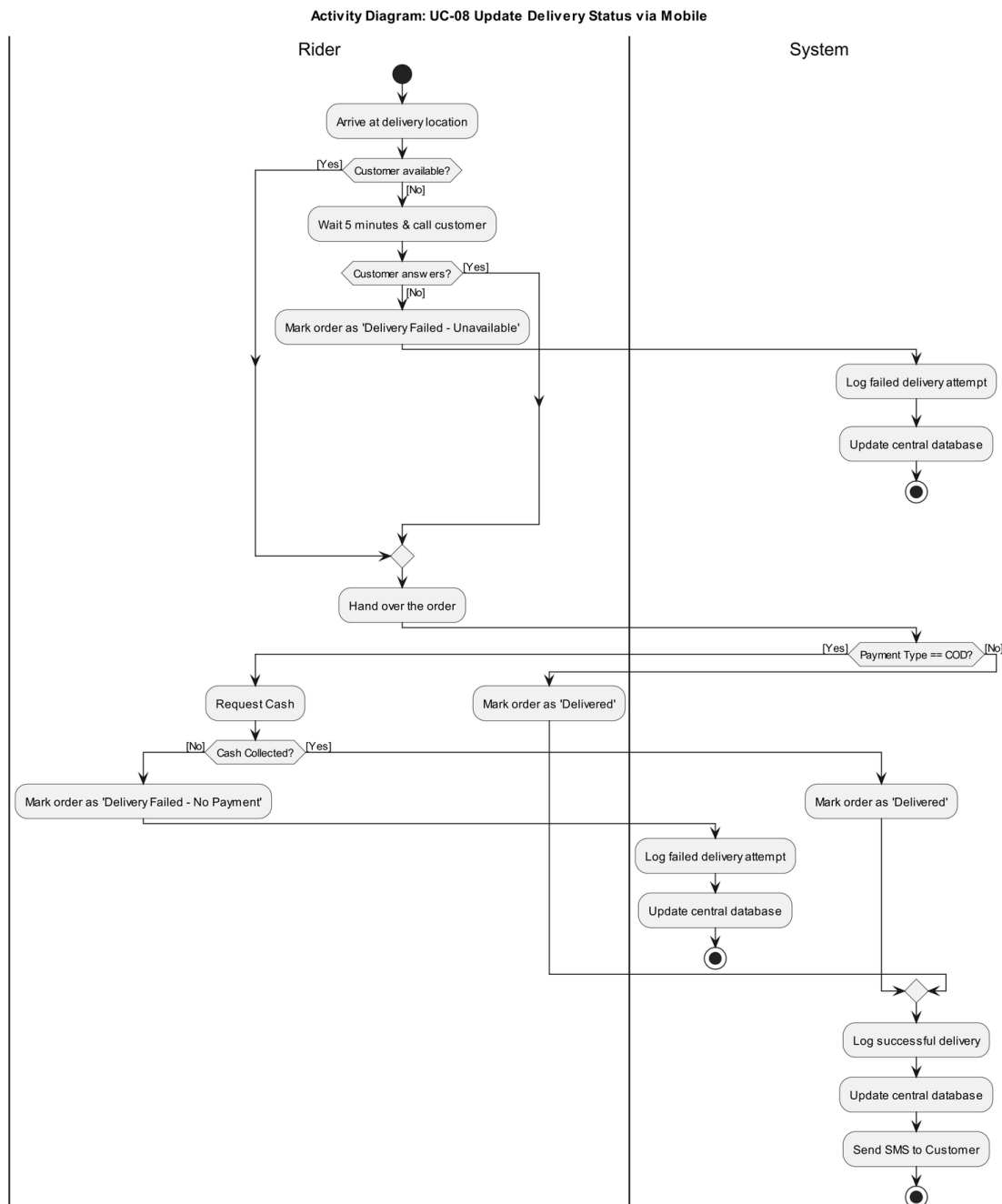
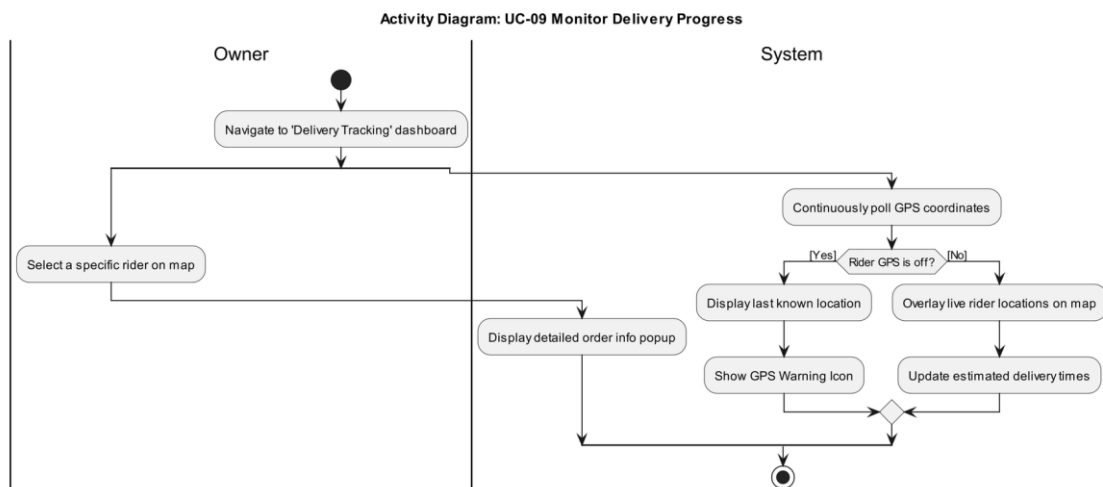
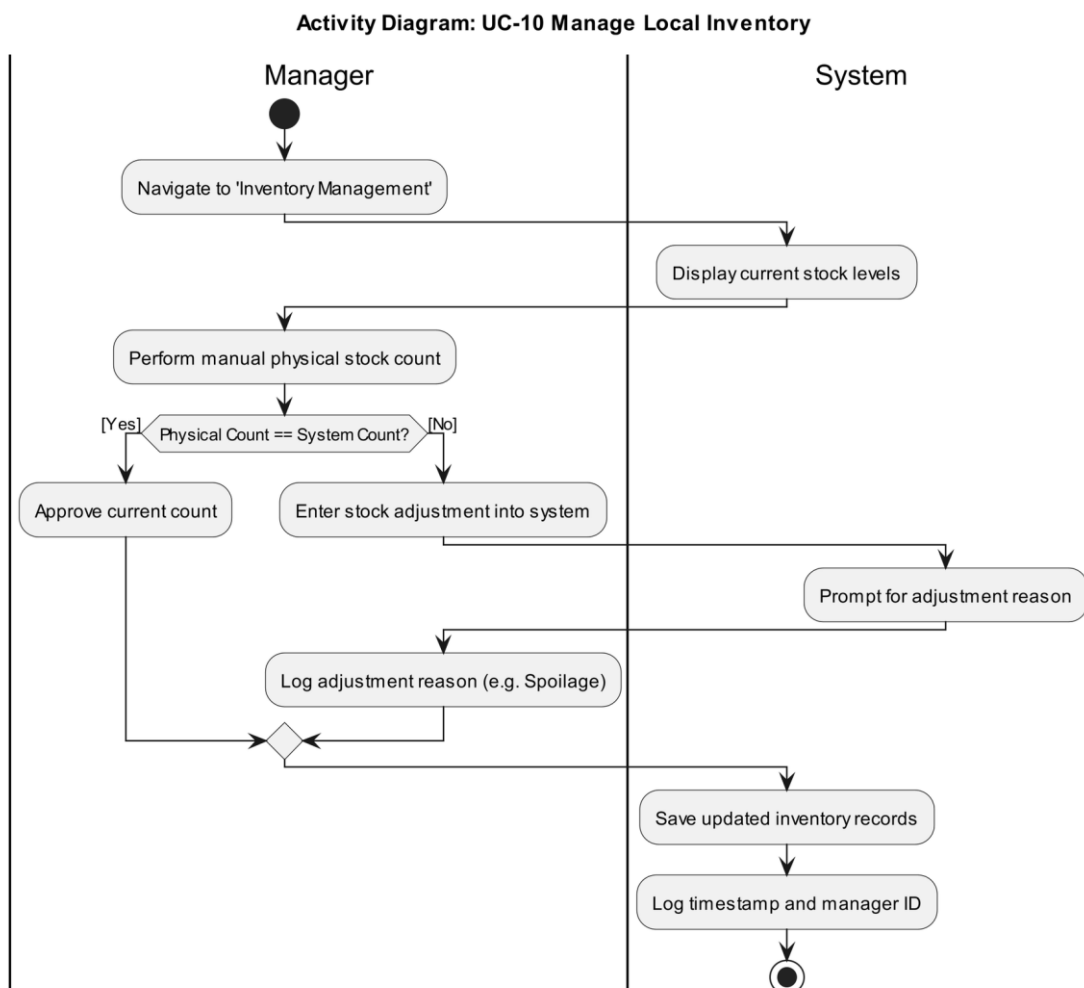


Figure 2.11: Activity Diagram - UC-09



AD-09

Figure 2.12: Activity Diagram - UC-10



2.8 Feasibility Study & Alternatives Considered

Before selecting the proposed custom n-tier architecture, an extensive feasibility study was conducted. Two alternative solutions were evaluated and rejected:

1. Alternative 1: Off-the-Shelf POS (e.g., Square, Toast). This approach was rejected on economic grounds due to prohibitive monthly SaaS fees per branch and transaction commissions which erode margins.
2. Alternative 2: Monolithic Desktop Application. This option was rejected on technical grounds because it could not support the cloud-synced multi-branch dashboard or the mobile Rider App.

CHAPTER 3: SYSTEM DESIGN

3.1 Architectural Decision Records (ADRs)

The system architecture was meticulously designed. The following ADRs document the core technical stack decisions:

ADR-01: Adoption of React.js for Frontend

Context: A highly interactive, fast-loading UI was required for the POS and KDS to minimize order processing latency.

Decision: React.js was selected for all web-based client applications.

Alternatives Considered: Angular (rejected due to steeper learning curve), Vanilla JS (rejected due to lack of state management).

Rationale: Fast rendering is ensured by React's virtual DOM, meeting the <2s KDS sync requirement.

ADR-02: Adoption of Flutter for Rider App

Context: A mix of Android and iOS personal devices is used by delivery riders.

Decision: Flutter was chosen for the Rider Mobile Application.

Alternatives Considered: Native Swift/Kotlin (rejected due to requiring two separate codebases).

Rationale: A unified codebase with near-native performance is provided by Flutter.

ADR-03: Selection of PostgreSQL

Context: Highly relational data (Orders, Invoices, Inventory) is processed by the system.

Decision: PostgreSQL was selected as the primary relational database.

Alternatives Considered: MongoDB (rejected due to lack of strict ACID compliance for financial transactions).

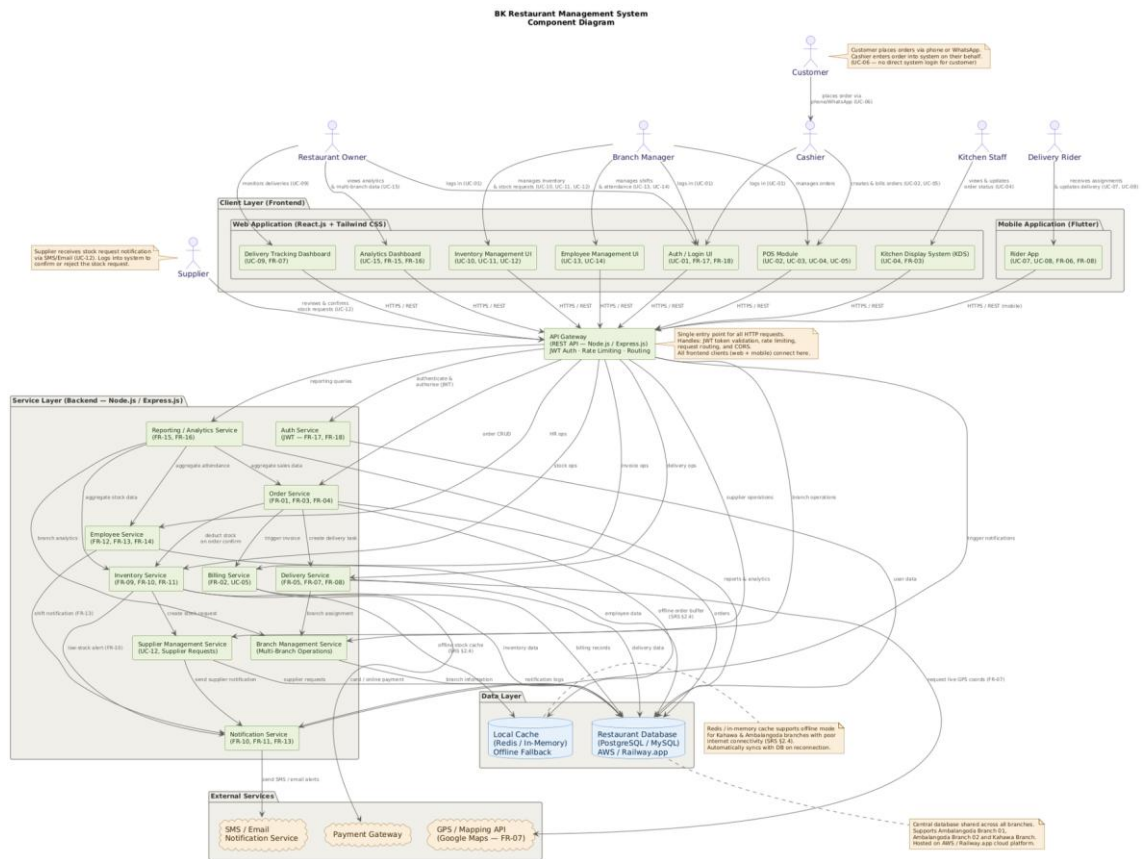
Rationale: ACID compliance is guaranteed by PostgreSQL, which is mandatory for billing integrity.

3.2 System Architecture Diagrams

Figure 3.1: Component Diagram

The comprehensive Component Diagram below illustrates the detailed internal architecture of the system. It maps the exhaustive relationships between all distinct software components, defining the interfaces between the React.js client modules, the individual Node.js backend microservices, and the structured PostgreSQL database tables.

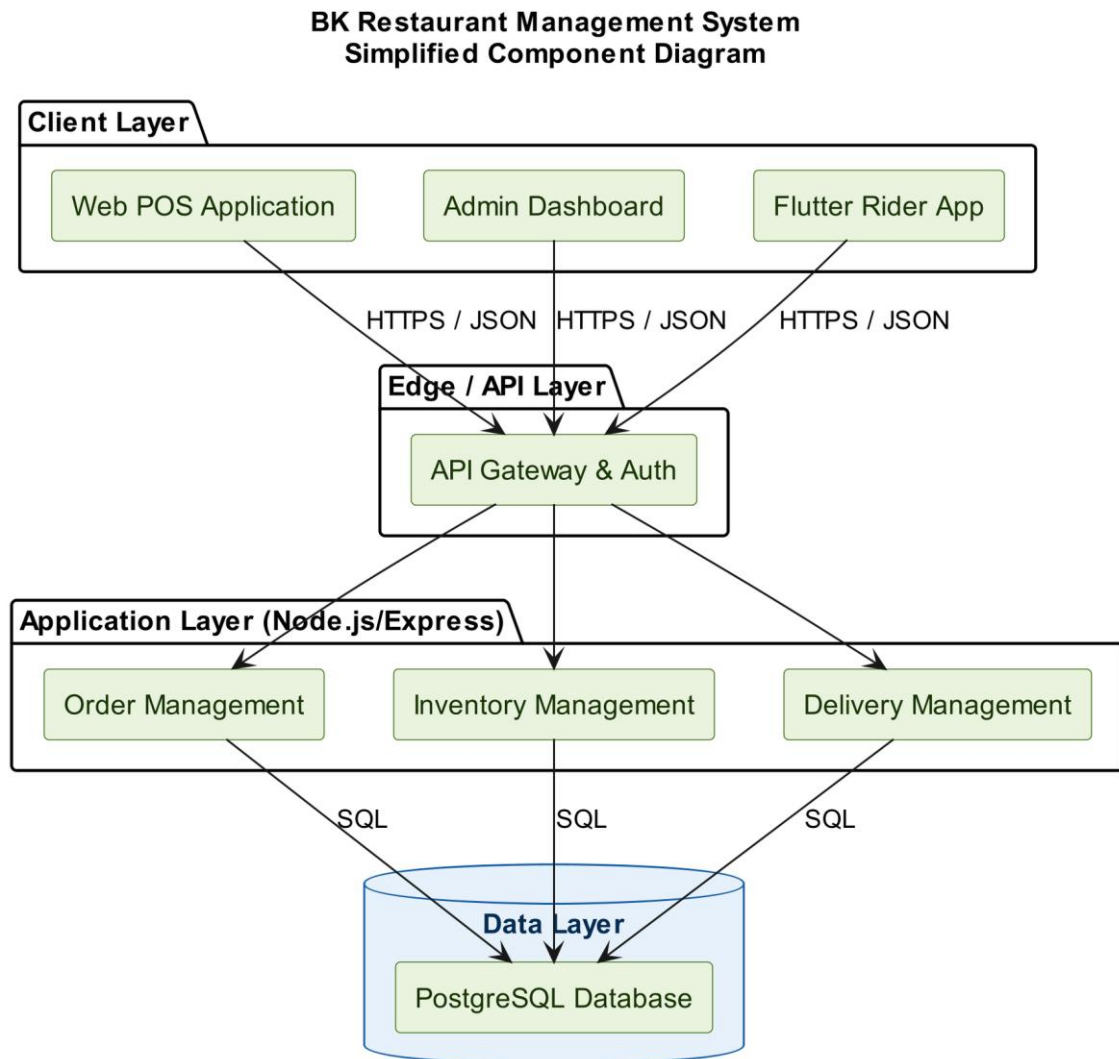
SENG 31242 - BK Restaurant Management System



Component Diagram

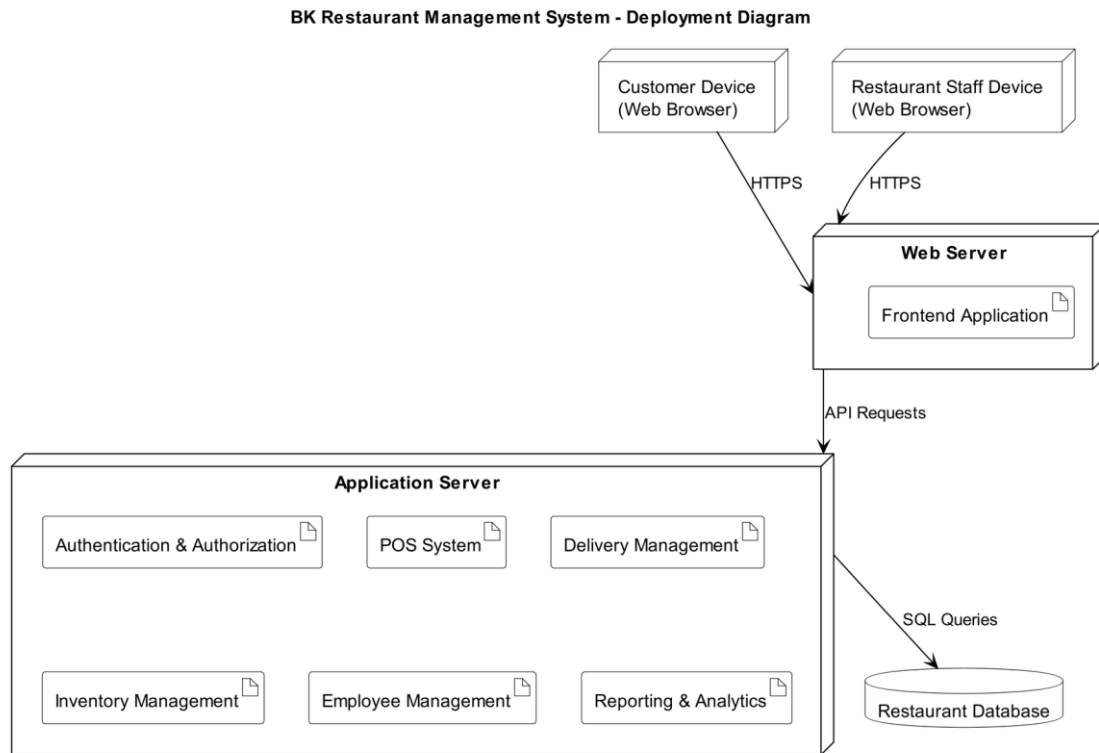
Figure 3.1.1: Simplified Component Diagram

A high-level view highlighting the interactions between the Client Layer, Application Layer, and Data Layer.



Simplified Component Diagram

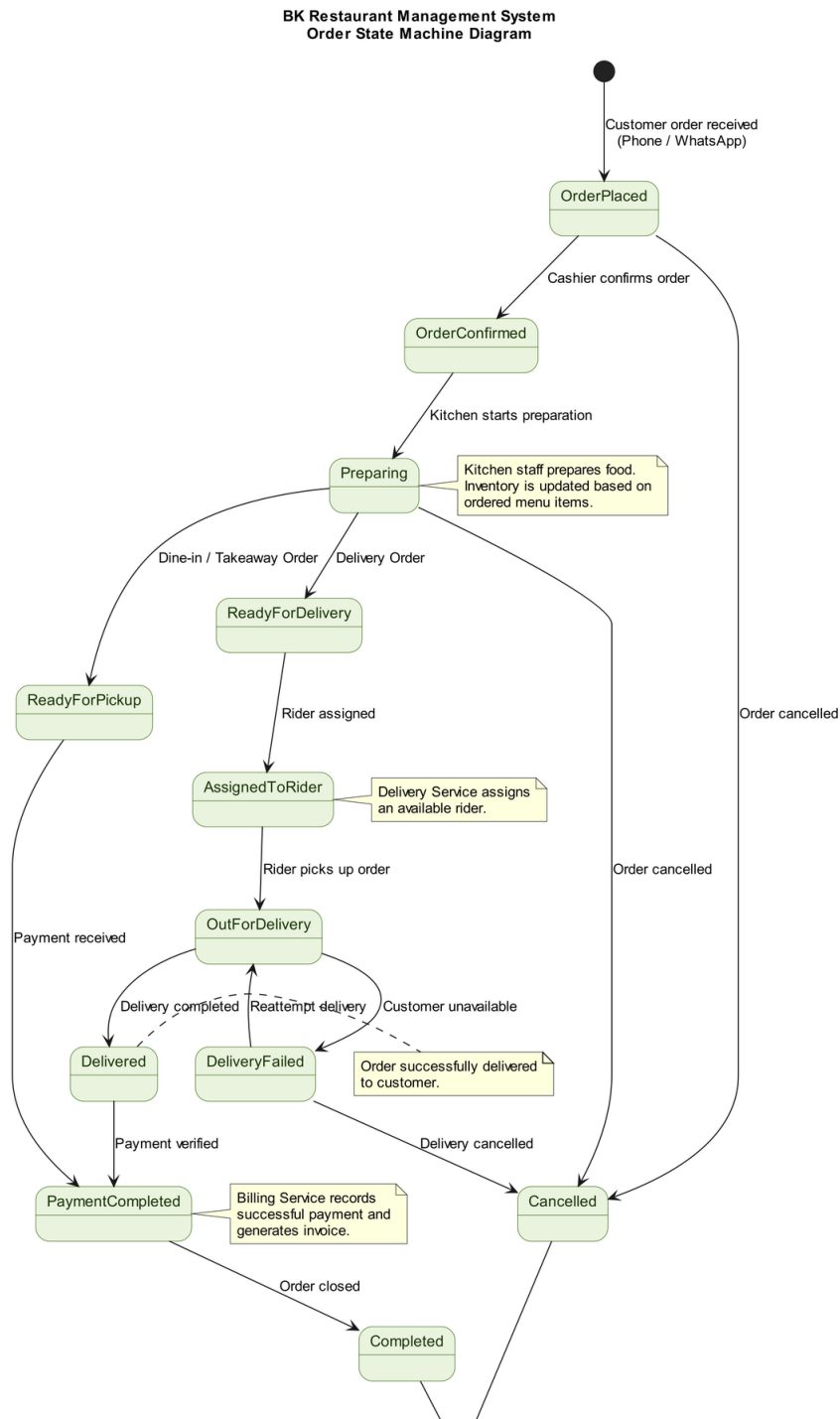
Figure 3.2: Deployment Diagram



Deployment Diagram

The Deployment Diagram above illustrates the physical hardware and software nodes of the BK Restaurant Management System. Client devices access the platform securely via HTTPS through a centralized Web Server, which serves the Frontend Application. The Web Server then routes internal API requests to the Application Server, which hosts the core backend micro-services (POS, Delivery, Inventory). Finally, all persistent transactional data is managed securely by the backend PostgreSQL Database.

Figure 3.3: Class Diagram / ERD State Behaviors



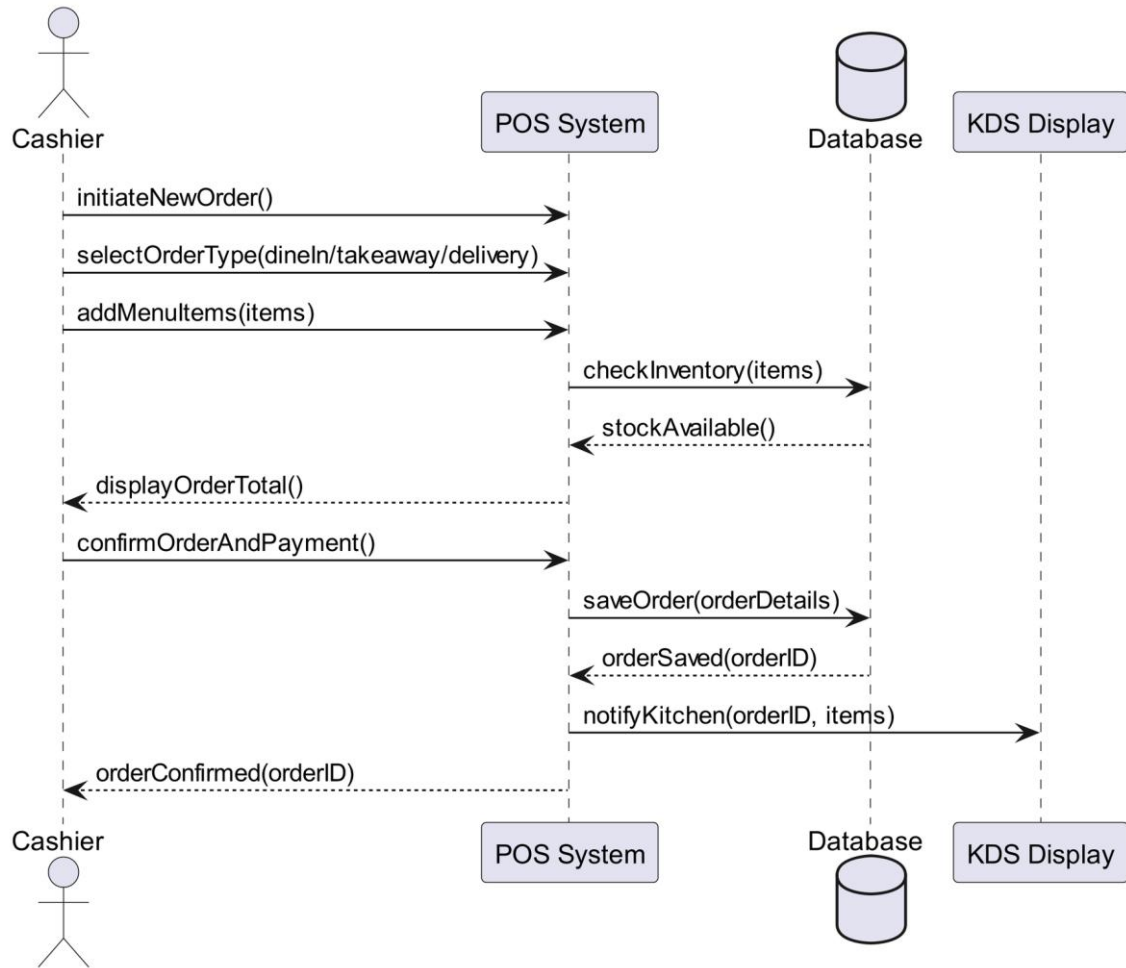
Class Diagram

3.3 Sequence Diagrams

The messaging between system components was documented via Sequence Diagrams.

Figure 3.4: Sequence Diagram - UC-02 (POS Order Entry)

Real-World Scenario: A walk-in customer arrives at the front desk to place a dine-in order. The Cashier uses the Point of Sale (POS) system to log the order type and add menu items. In real-time, the POS verifies ingredient availability against the database. Once payment is confirmed, the order is permanently saved and instantly pushed to the Kitchen Display System (KDS), eliminating the need for physical paper tickets.



SD-02

Figure 3.5: Sequence Diagram - UC-03 (Modify/Cancel Order)

Real-World Scenario: A customer changes their mind immediately after ordering and wants to add a beverage. The Cashier searches for the active order ID. The system fetches the current state, and the Cashier modifies the items. The backend dynamically recalculates the total, updates the database, and synchronizes the modification so the kitchen is immediately aware of the change.

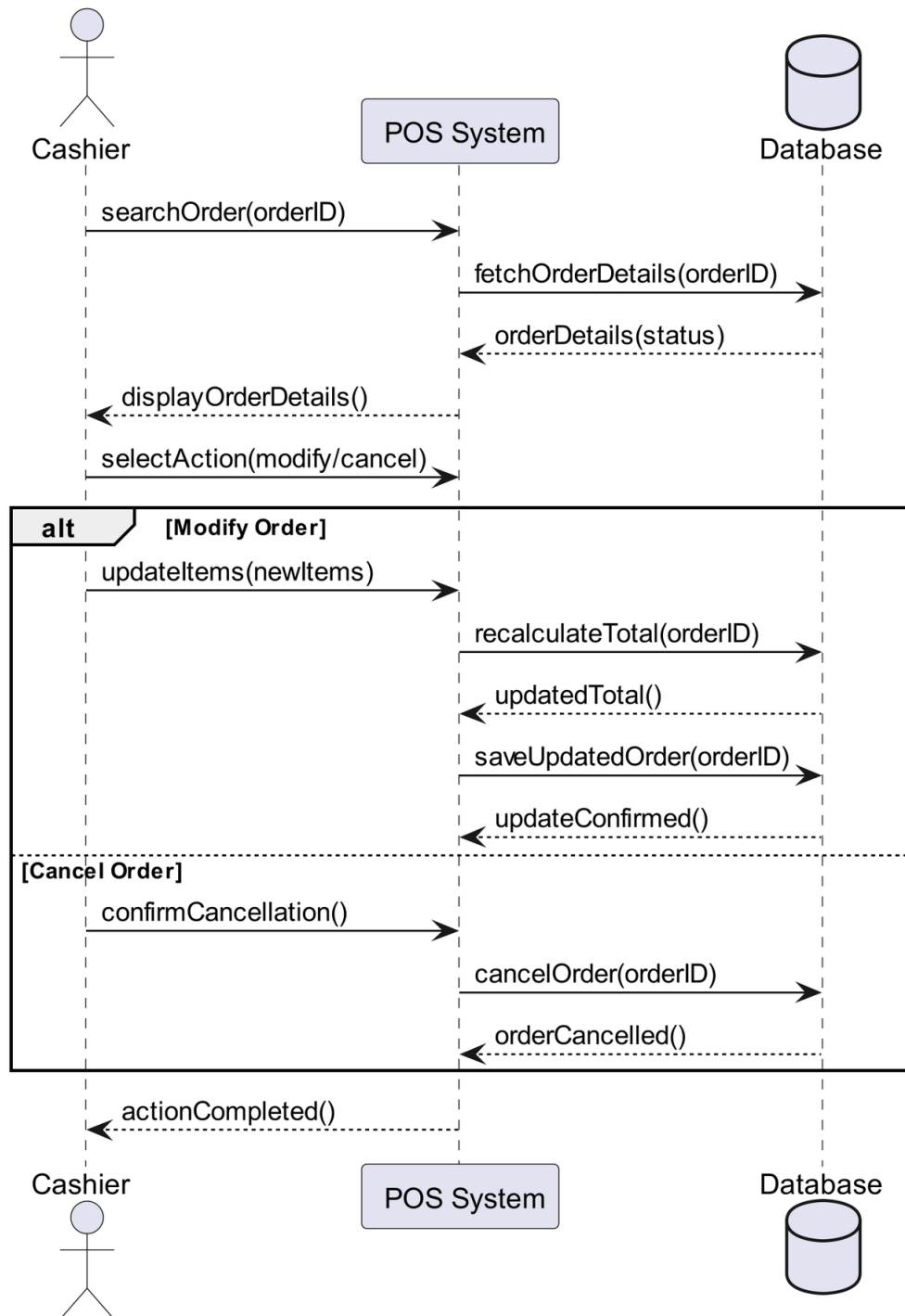


Figure 3.6: Sequence Diagram - UC-04 (Kitchen Order Processing)

Real-World Scenario: In the bustling kitchen environment, the Head Chef monitors the KDS display which continuously polls for active orders. When an order is completed, the chef taps the screen to change its status to "Ready". This triggers a backend update that marks the order as ready for pickup or dispatch, ensuring seamless synchronization between the kitchen and the front-of-house staff.

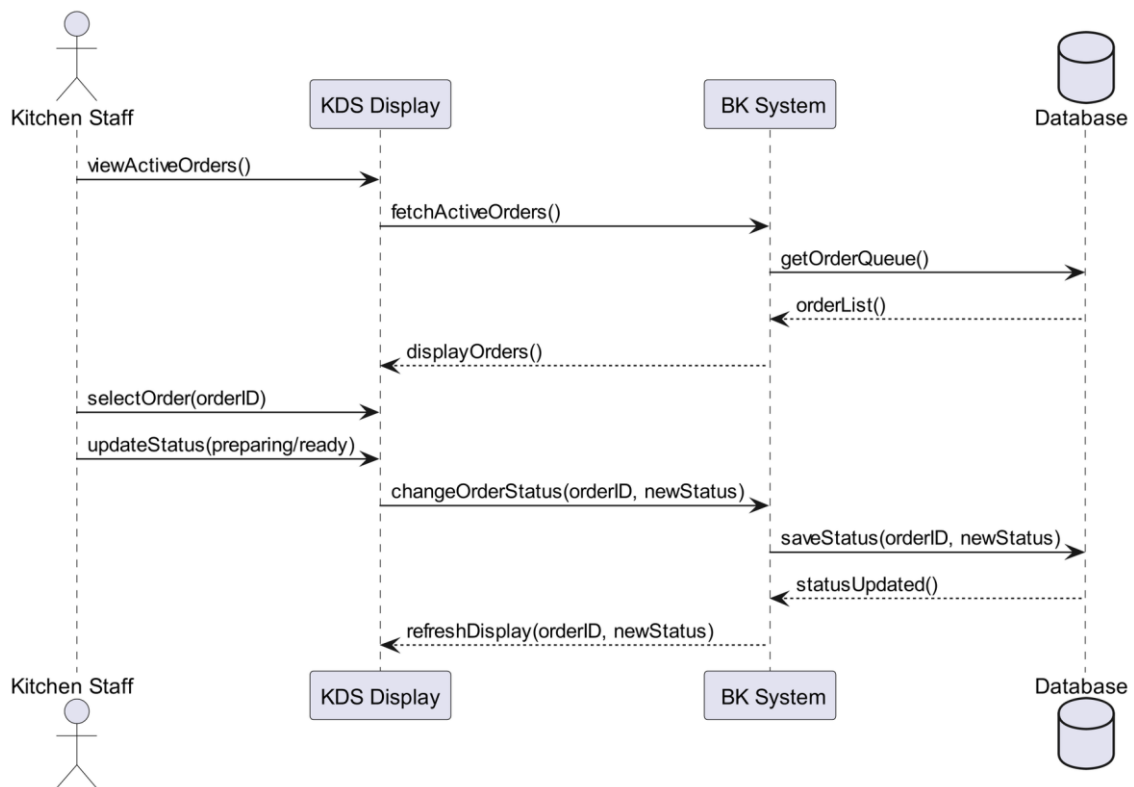
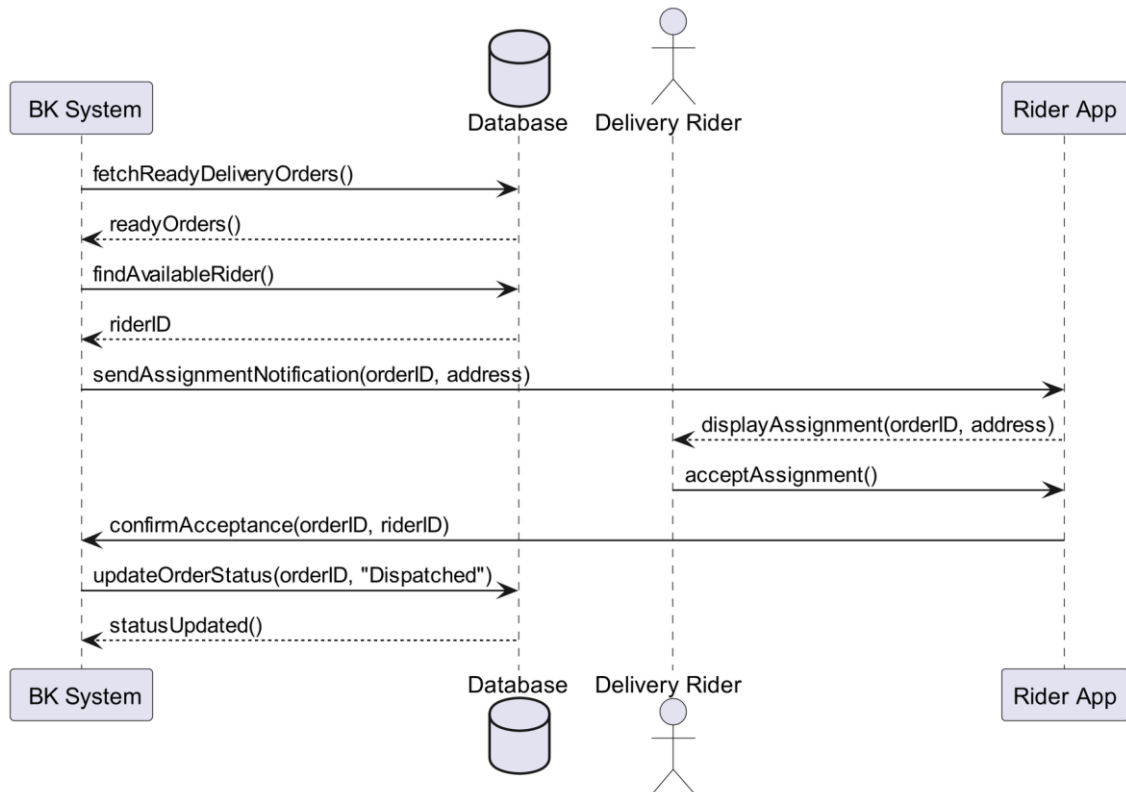


Figure 3.7: Sequence Diagram - UC-07 (Rider Assignment)

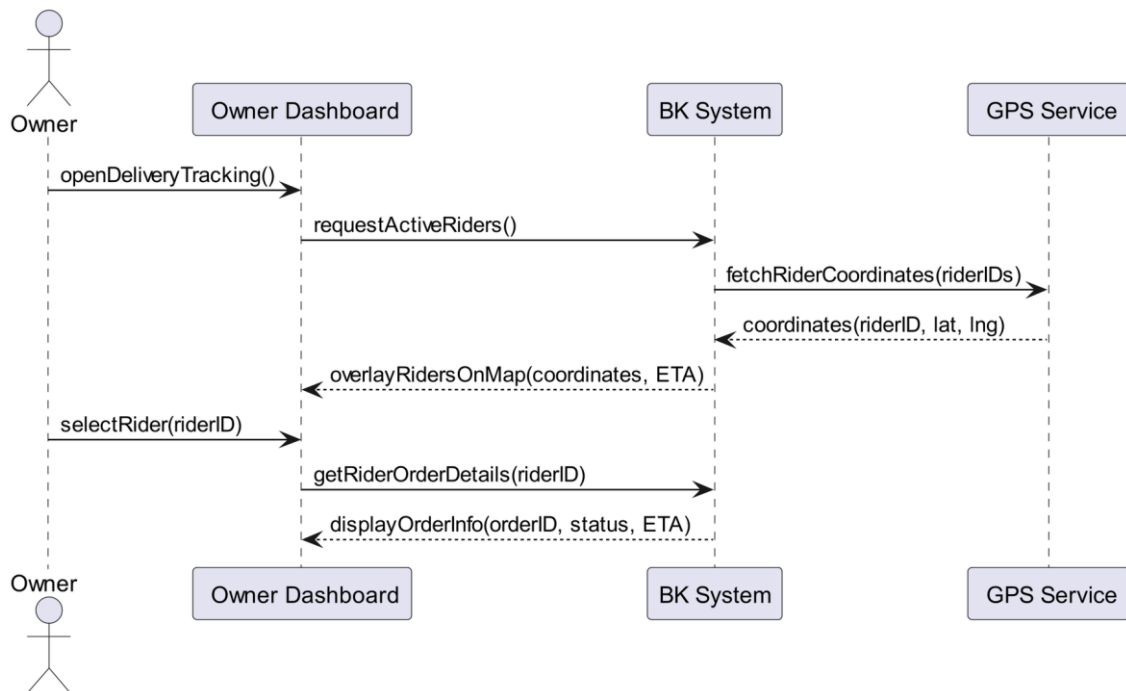
Real-World Scenario: An external takeaway order is marked as "Ready" by the kitchen. The central system automatically scans the database for available delivery riders. A push notification is sent to the designated Rider's mobile app. Once the Rider taps "Accept" on their device, the system updates the backend status to "Dispatched", handing off the chain of custody to the delivery fleet.



SD-07

Figure 3.8: Sequence Diagram - UC-09 (Live Delivery Tracking)

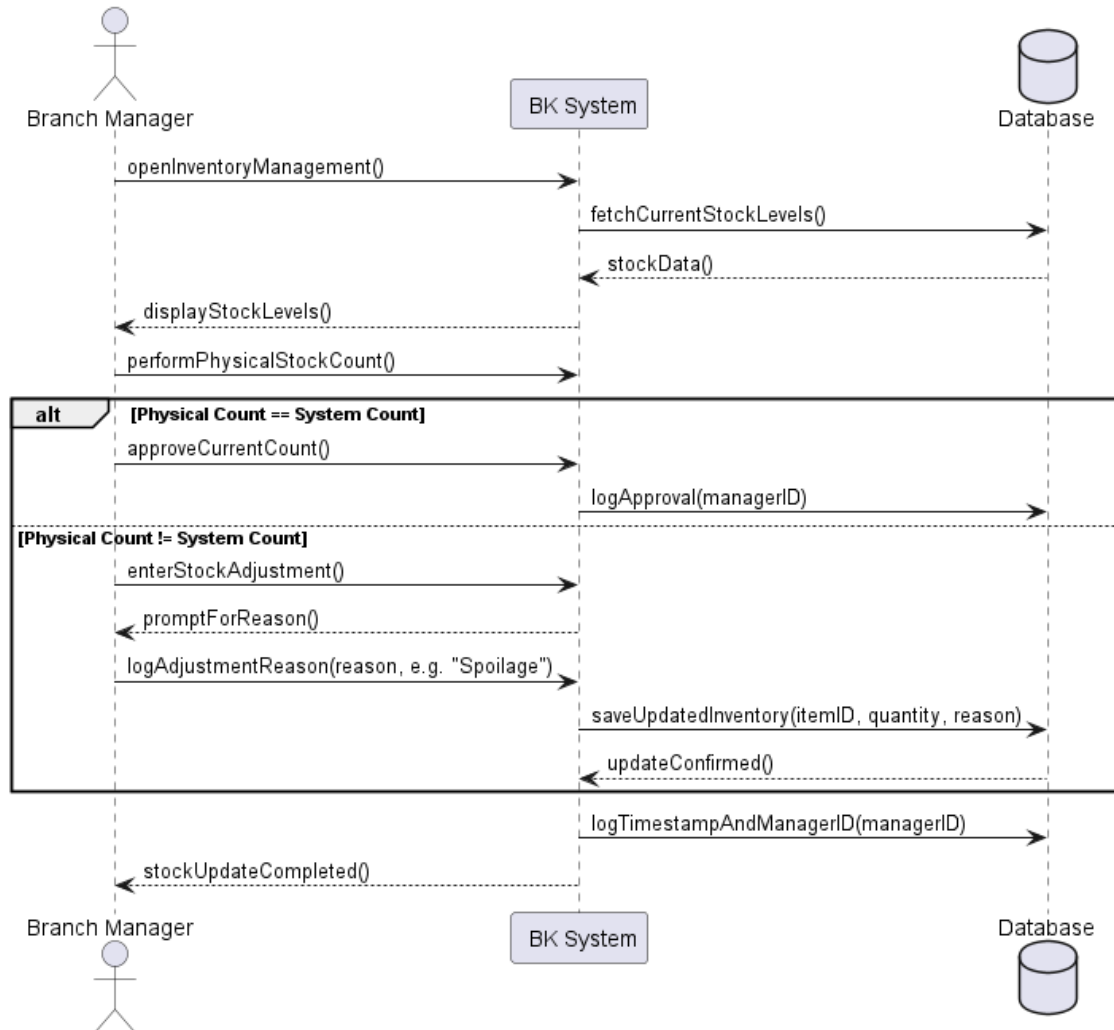
Real-World Scenario: The Restaurant Owner wants to check the efficiency of the delivery fleet during peak hours. By opening the Owner Dashboard, the system requests live telemetry data. The backend fetches continuous GPS coordinates from the external GPS service and streams them back to the frontend, plotting the active riders on a live map interface with calculated ETAs.



SD-09

Figure 3.9: Sequence Diagram - UC-10 (Inventory Adjustment)

Real-World Scenario: During an end-of-day audit, a Branch Manager discovers that a bag of flour was damaged. The Manager navigates to the Inventory module, fetches current stock levels, and enters a manual stock adjustment along with the reason ("Spoilage"). The system immutably logs the transaction against the Manager's ID for strict auditing purposes and updates the master inventory.



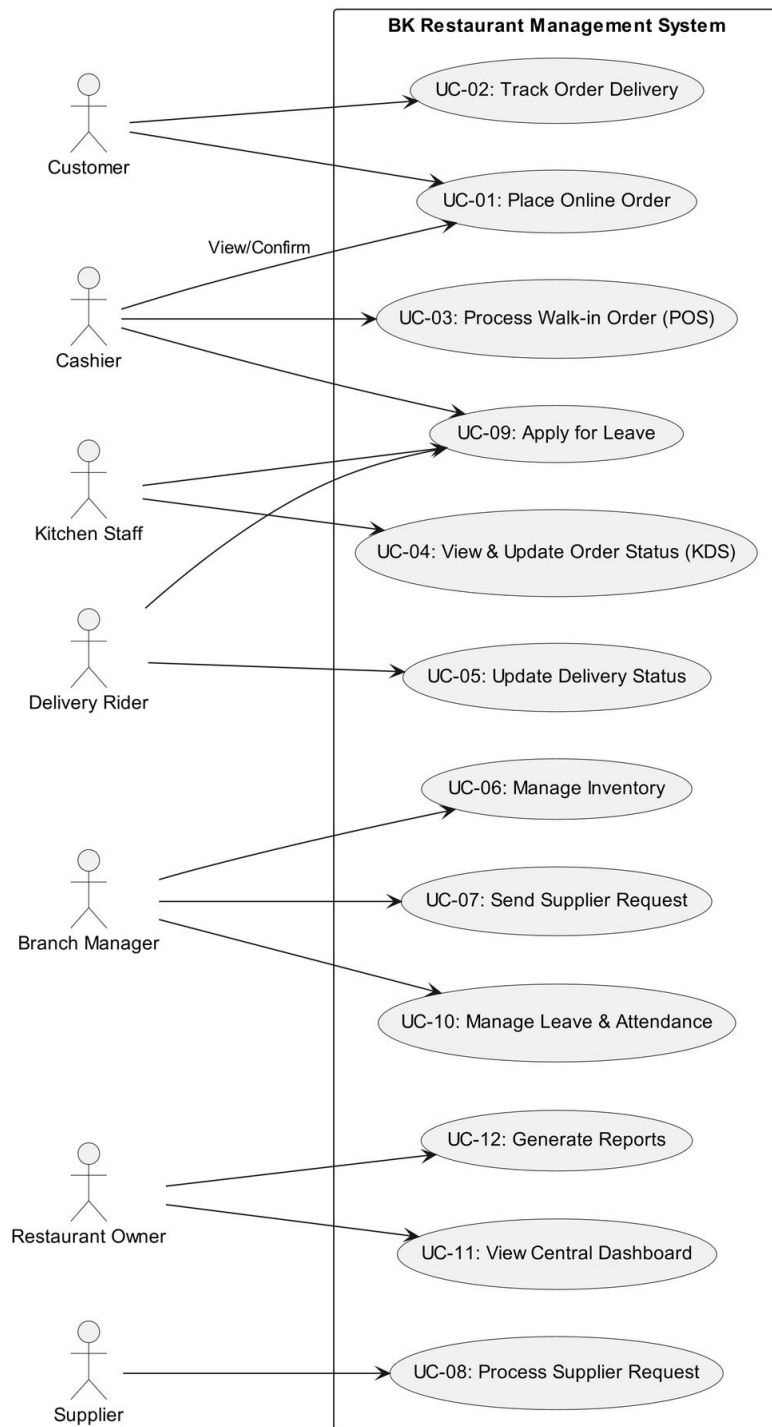
SD-10

3.4 Use Case Diagrams

Figure 3.10: Use Case Diagram (Role-Based Access)

Description: This Use Case diagram maps the comprehensive role-based access controls across the system. It illustrates exactly which features (Use Cases) are available to each

specific user role (Owner, Manager, Cashier, Kitchen, Rider), serving as the foundational navigational map for the UI flows.



3.5 User Interface Design & Flow Mockups

The user experience was mapped through comprehensive wireframes and flows.

3.5.1 Navigational User Flows

Figure 3.11: Customer Order Creation Flow

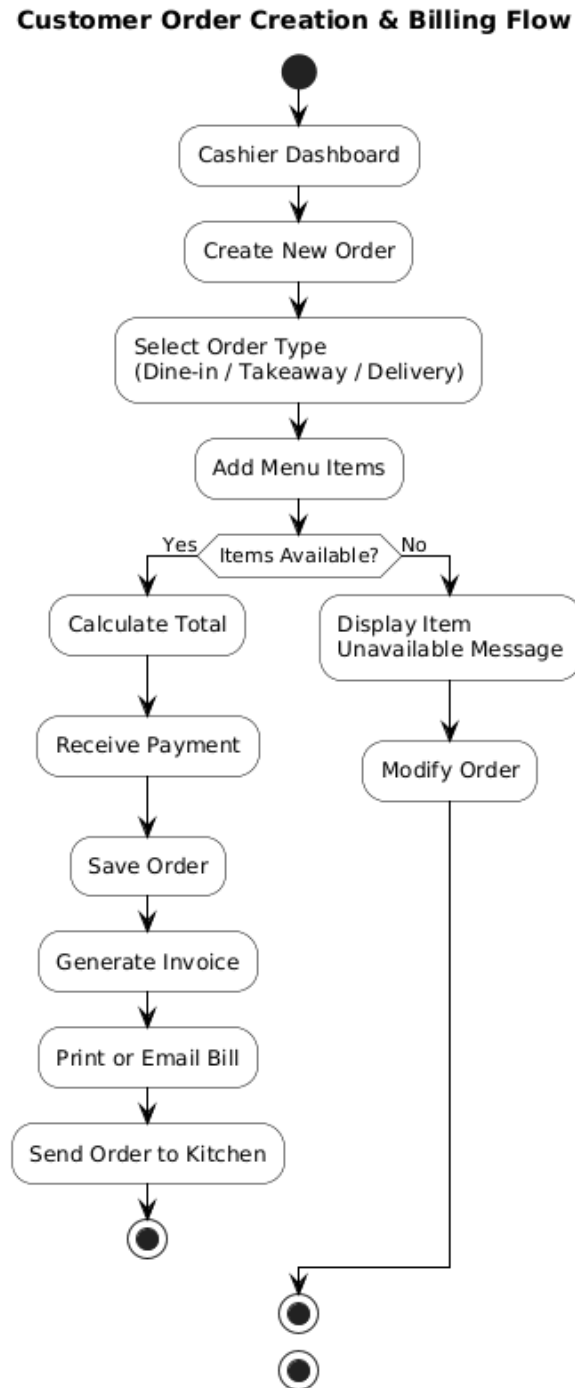


Figure 3.12: Kitchen Order Processing Flow

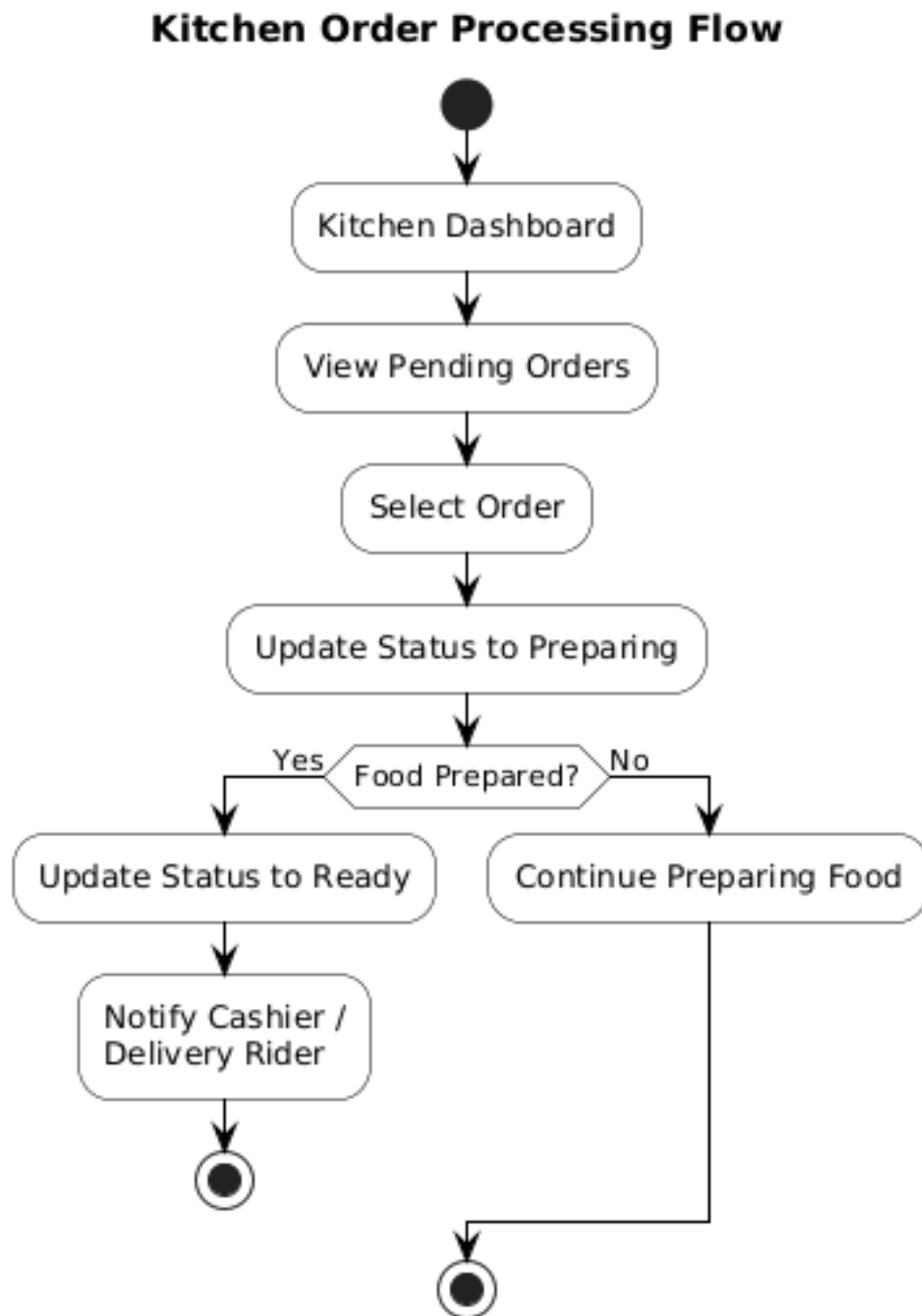


Figure 3.13: Delivery Rider Flow

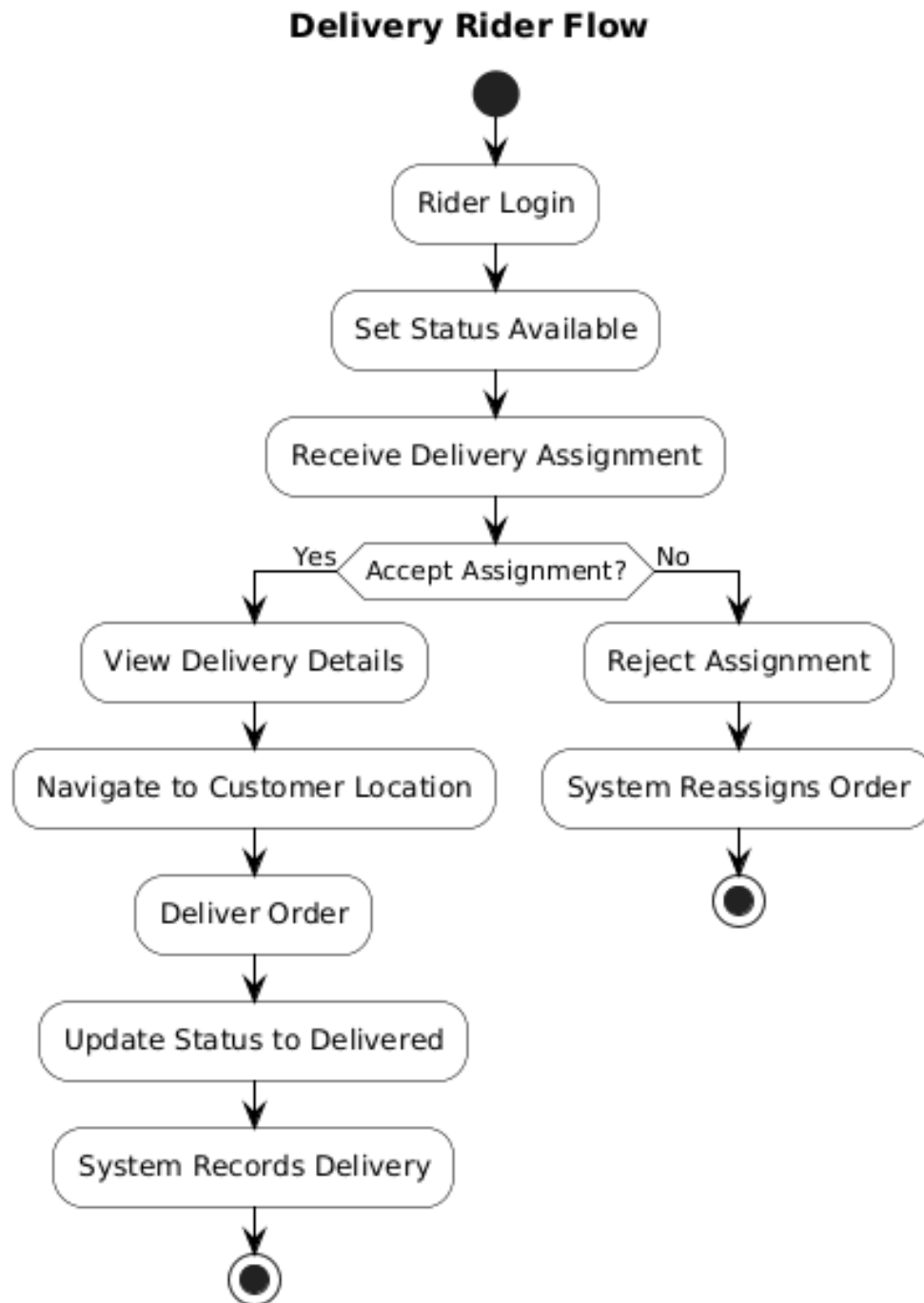


Figure 3.14: Inventory Management Flow

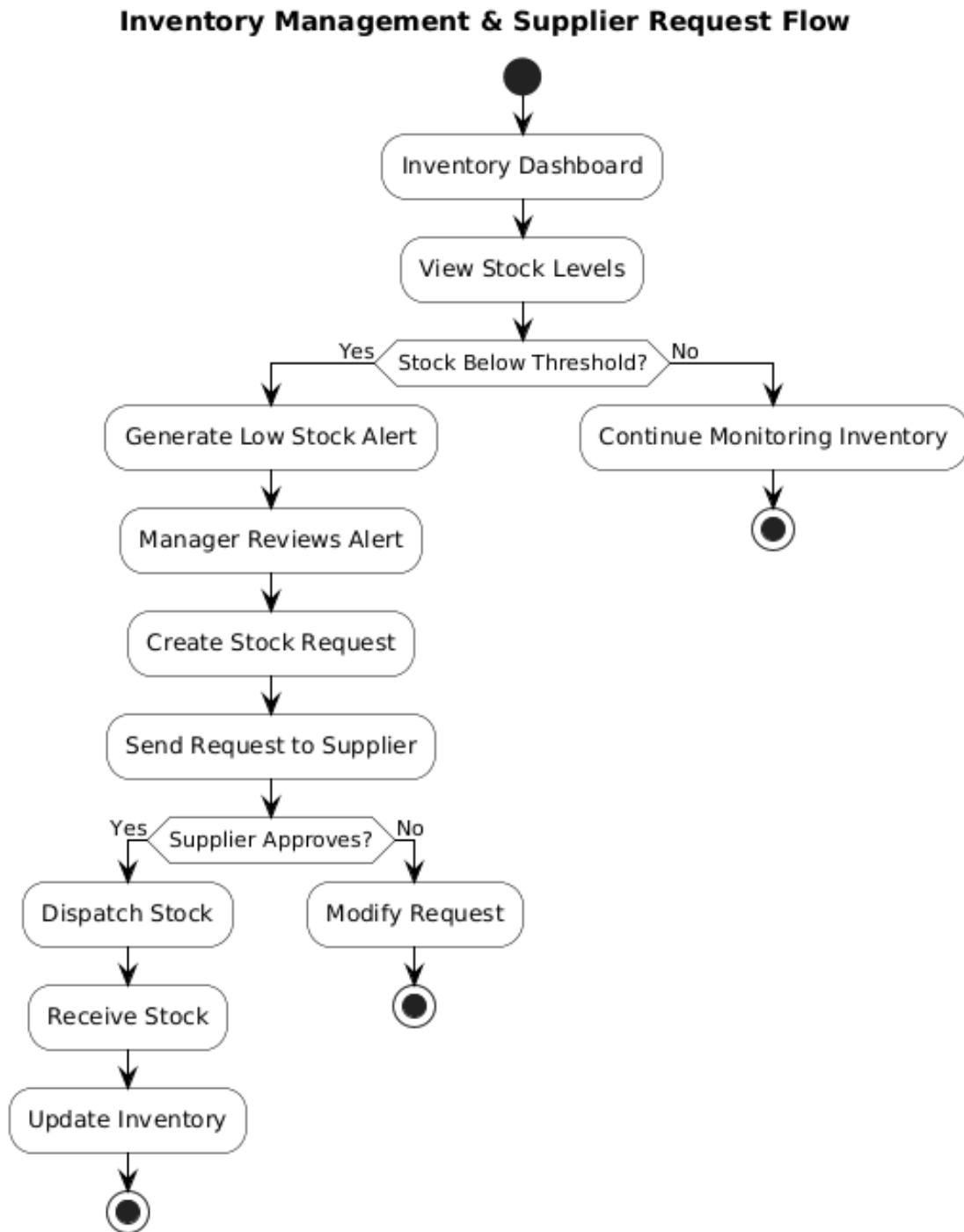
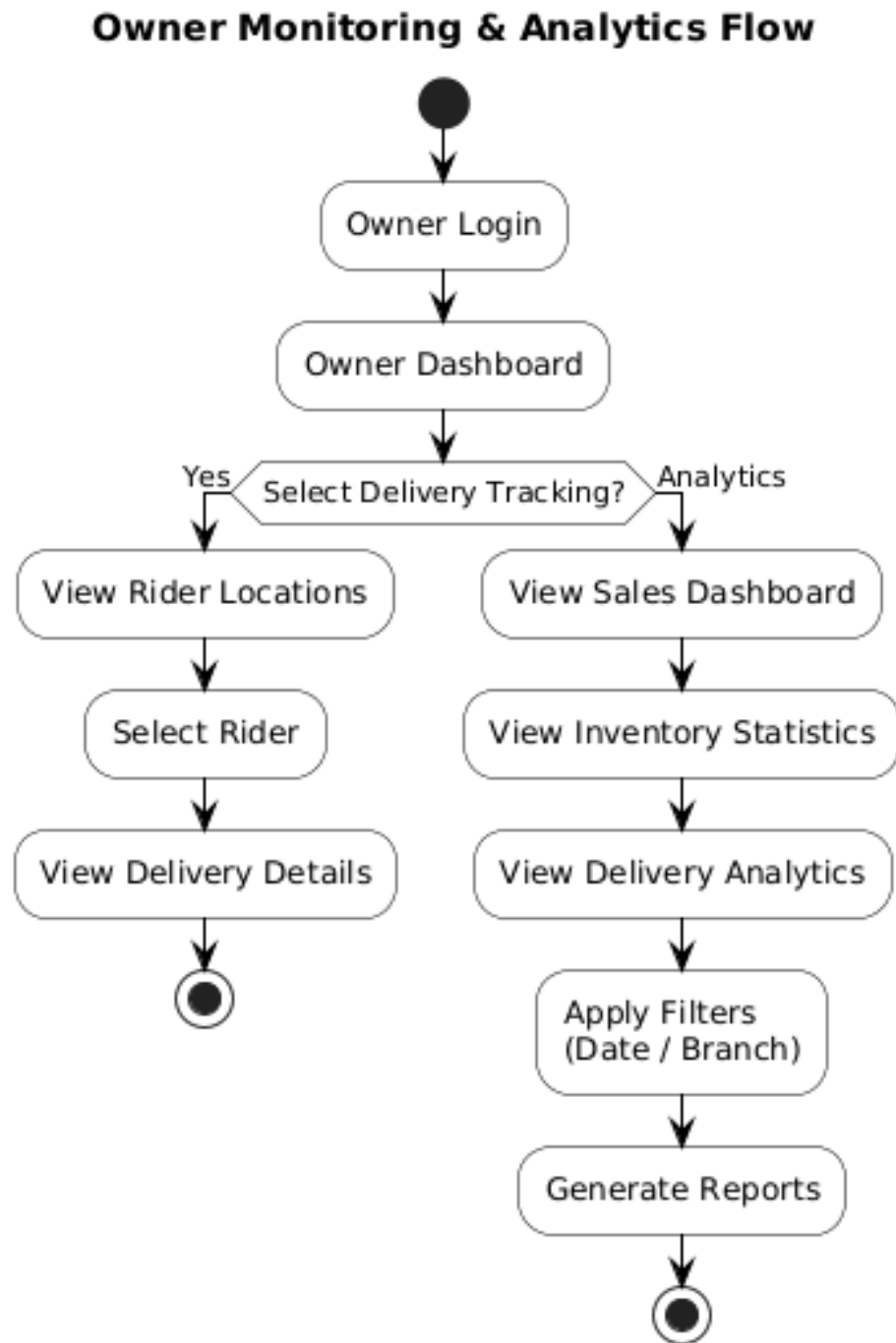


Figure 3.15: Owner Monitoring Flow



3.5.2 UI Screen Mockups

Figure 3.16: UI - Staff Registration

Staff Registration

BK Management
Sri Lankan Flavors

Dashboard

Staff Management

Inventory

Schedules

Leave Requests

Analytics

Settings

Support

Staff Management

Search staff...

Registration Progress

1 Personal Details
IDENTITY INFORMATION

2 Work Placement
ROLE & BRANCH

3 Security Access
LOGIN CREDENTIALS

Your data is encrypted and stored in compliance with restaurant security protocols.

New Staff Profile
Step 1: Define the basic identity

First Name
e.g. Aruni

Last Name
e.g. Perera

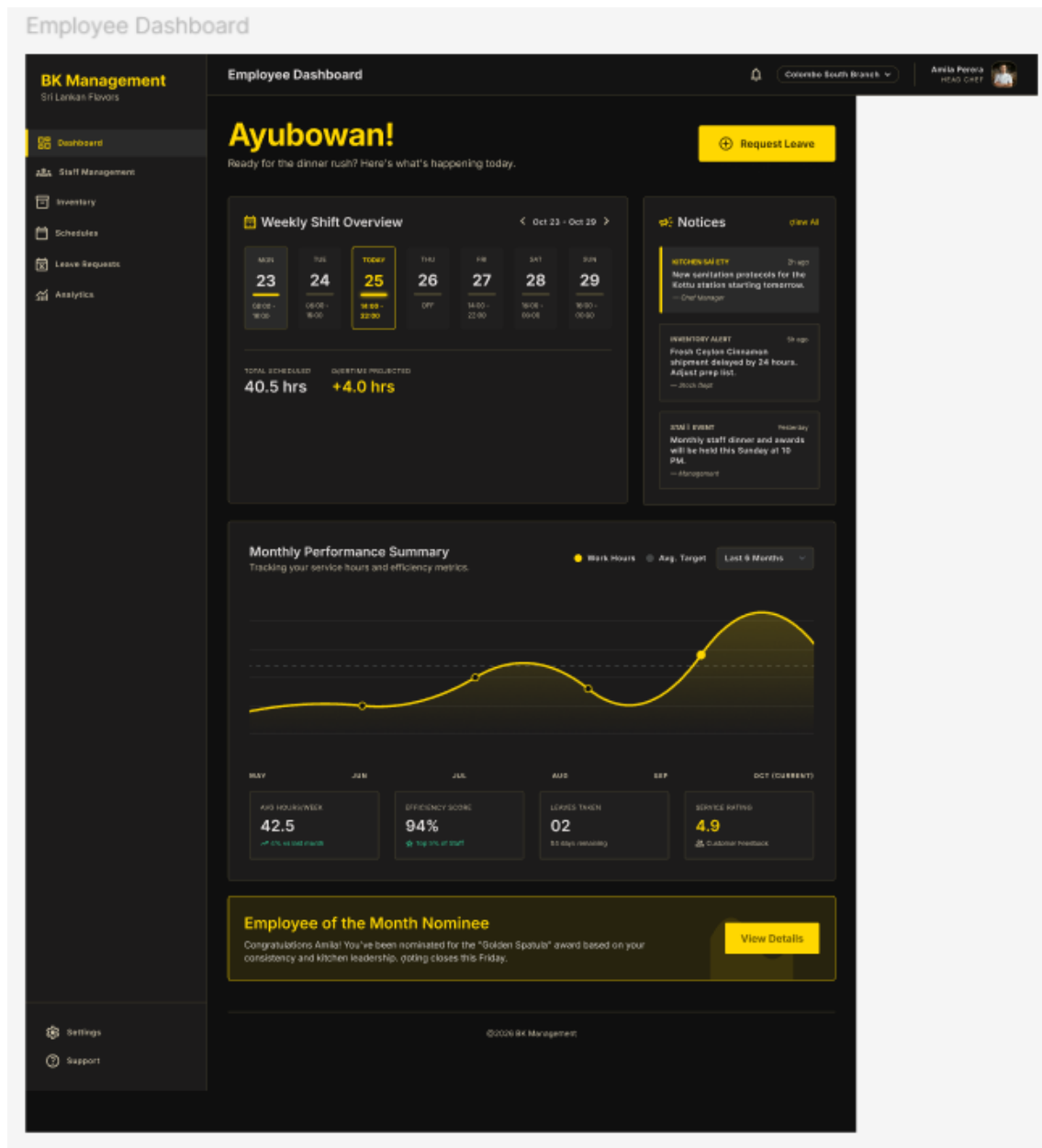
Phone Number
+94 77 123 4567

NEXT STEP ->

SERVER STATUS: OPERATIONAL

© 2026 BK RESTAURANT GROUP. SECURED BY AES-256 ENCRYPTION.

Figure 3.17: UI - Employee Dashboard



UI - Employee Schedule

BK Management

Sri Lankan Flavors

Dashboard

Staff Management

Inventory

Schedules

Leave Requests

Analytics

New Entry

Settings

Support

Schedules

Columbo Port Branch

Search staff or shift...

Logout

Shift Timetable

August 2024 • Week 34

Weekly Monthly Team View

Today

ACTIVE NOW

Morning Shift

06:00 AM — 02:00 PM

On duty with you

+2.5h this week

42 Hours

Scheduled Total (Week 34)

Calculated base

LKR 28,400

Estimated Weekly Earnings

MON 19	TUE 20	WED 21 (TODAY)	THU 22	FRI 23	SAT 24	SUN 25
MORNING 06:00 - 14:00 Male Kitchen	EVENING 14:00 - 22:00 Floor Staff	ACTIVE NOW 06:00 - 14:00 Pastry Section	NIGHT 22:00 - 06:00 Inventory Check	MORNING 06:00 - 14:00 Male Kitchen	REQUIRED OFF	STORE CLOSED

Team on Duty - Today

Arjun Mendis

Sous Chef

Nimal Perera

Head Server

Borana Kuruppu

Evening Shift

Prithi Silva

Night Shift

Using the Hustle

"Success is served hot. Manage your time like a master chef manages his kitchen - with precision and passion."

UI - Leave Requests

Leave Requests

BK Management
Sri Lankan Flavours

Dashboard

Staff Management

Inventory

Schedules

Leave Requests

Analytics

Settings

Support

Leave Management

Search requests...

Annual Leave Balance

14 / 21 days

Sick Leave Used

03 / 07 days

Casual Leave

02 / 05 days

Next Holiday

Poya Day - Aug 19

New Request

Leave Type

Annual Leave

From Date

mm/dd/yyyy

To Date

mm/dd/yyyy

Reason

Briefly explain the reason for your leave...

Submit Request

Status Tracking

View All

TYPE	DATES	REASON	STATUS
Annual	Oct 12 - Oct 15 4 Days	Family trip to Kar	APPROVED
Sick	Sep 04 - Sep 05 2 Days	Flu symptoms	APPROVED
Casual	Aug 28 - Aug 28 1 Day	Urgent bank wor	PENDING
Annual	Aug 10 - Aug 12 3 Days	Wedding celebra	REJECTED
Casual	Jul 20 - Jul 20 1 Day	Personal matters	APPROVED

Leave Policy

Please ensure all leave requests are submitted at least 48 hours in advance for planning purposes. Emergency sick leave should be notified by phone before the shift start.

Download Full Policy PDF →

Upcoming Public Holidays

Nikini Full Moon Poya Day

Aug 19

Milad-un-Nabi

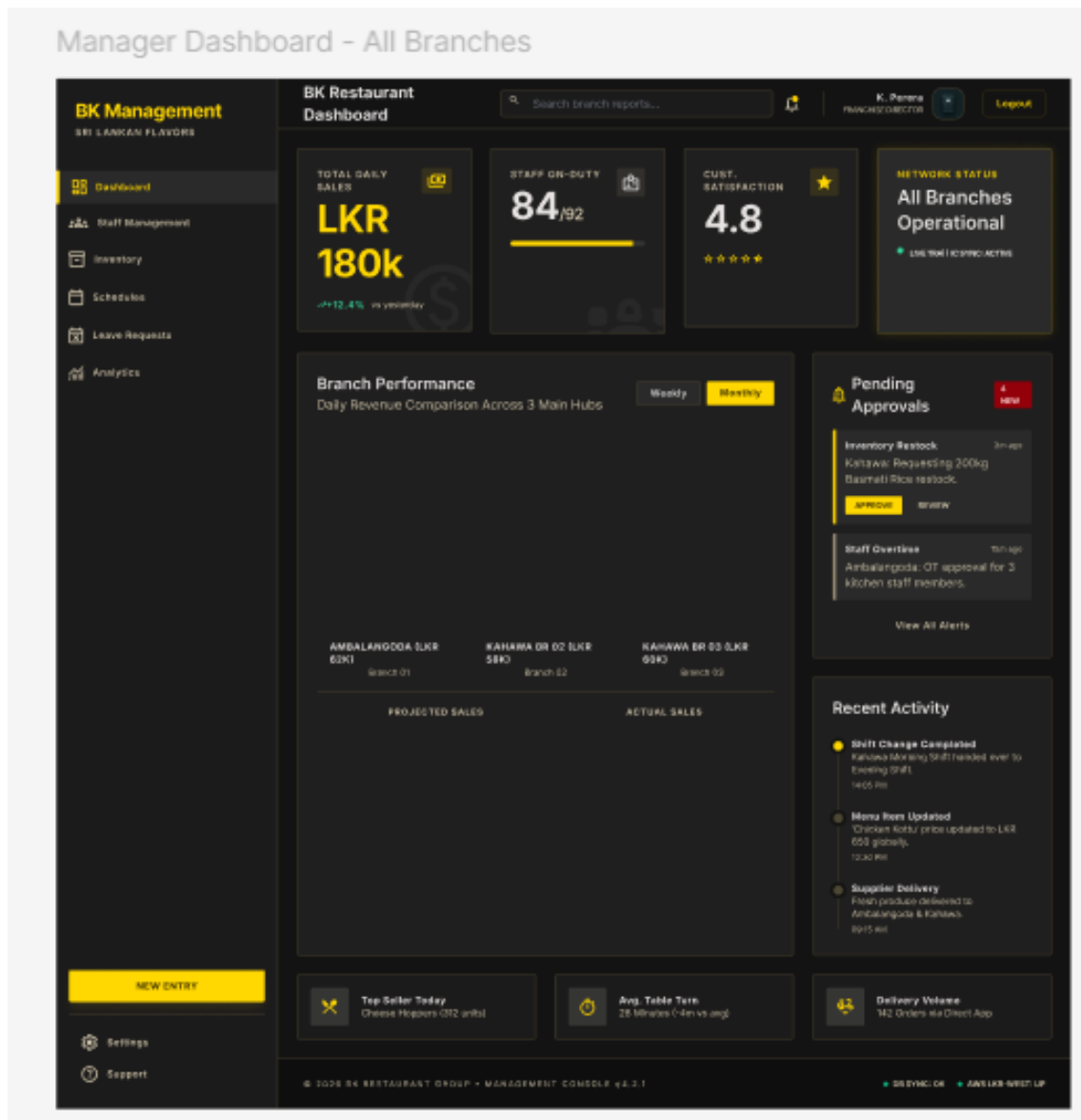
Sep 16

Binara Full Moon Poya Day

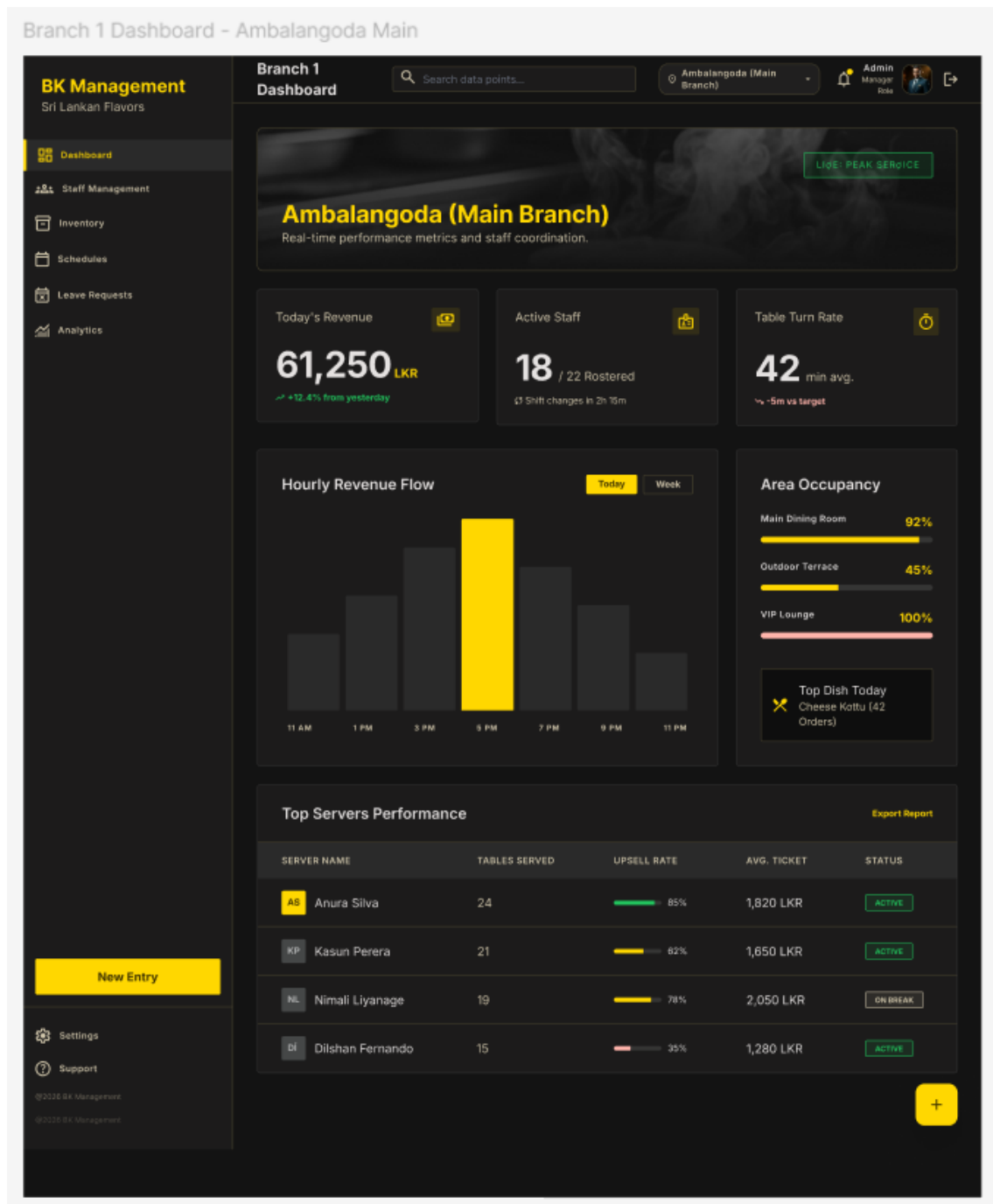
Sep 17

©2026 BK MANAGEMENT

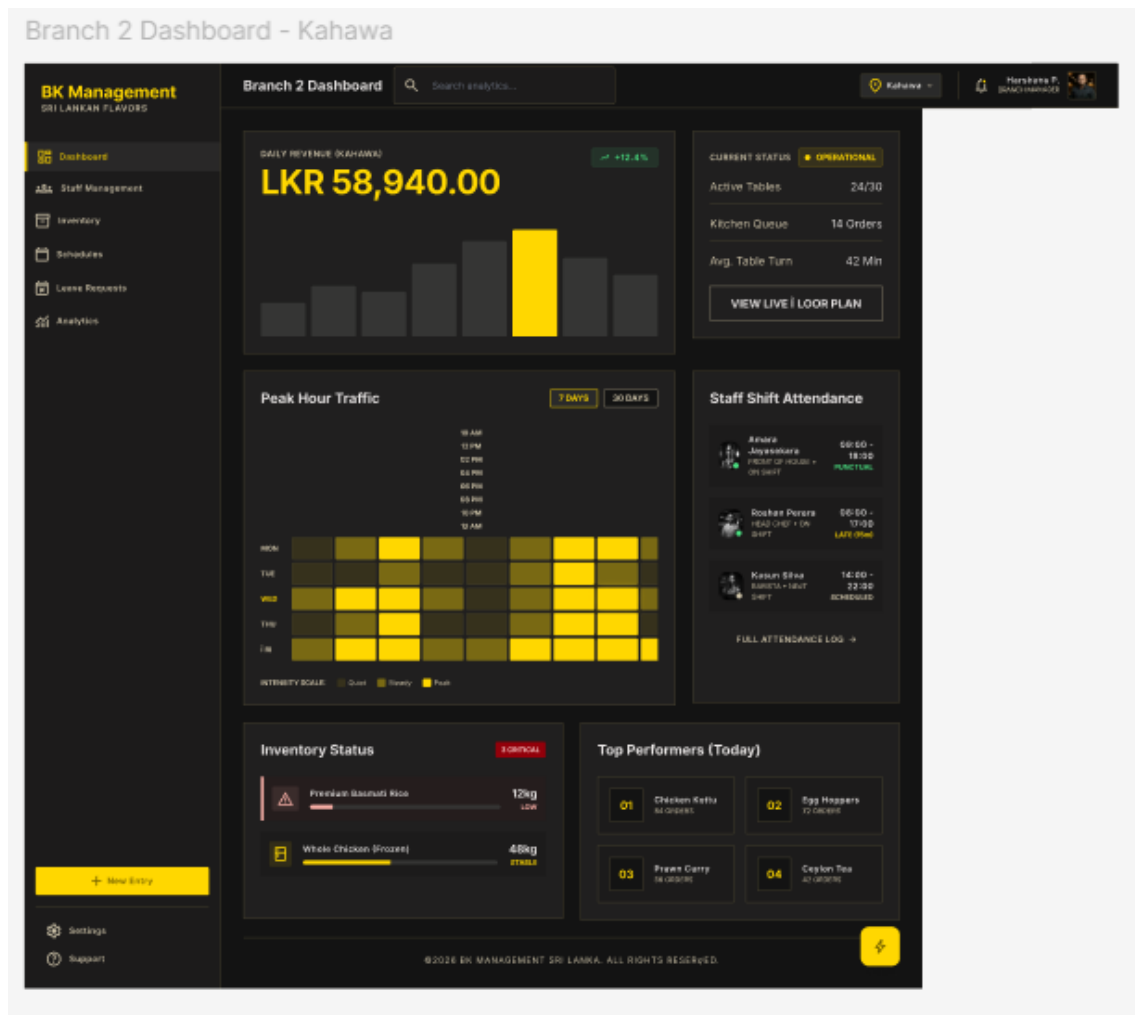
Figure 3.18: UI - Manager Dashboard (Global)



UI - Branch 1 Dashboard



UI - Branch 2 Dashboard



UI - Branch 3 Dashboard

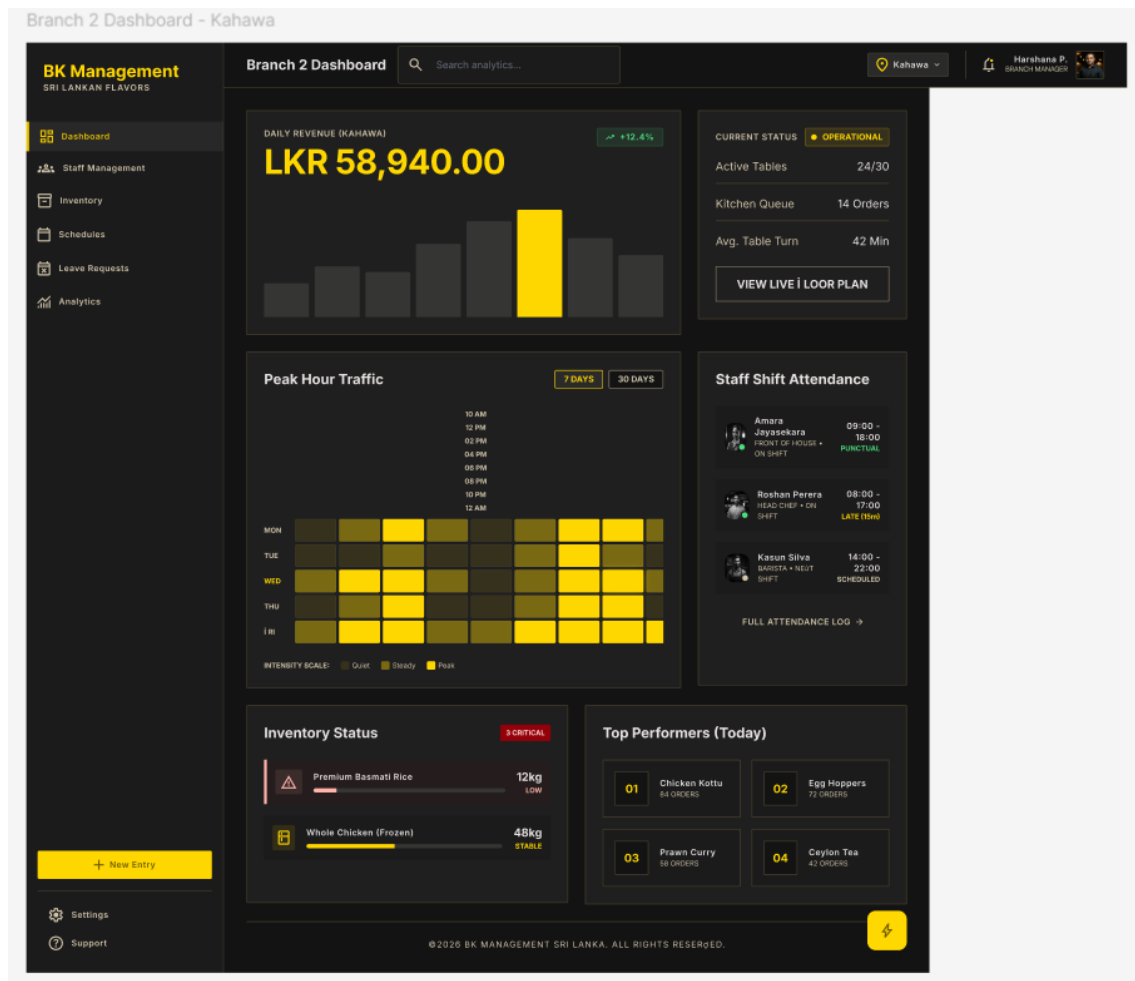


Figure 3.19: UI - Overall Analytics

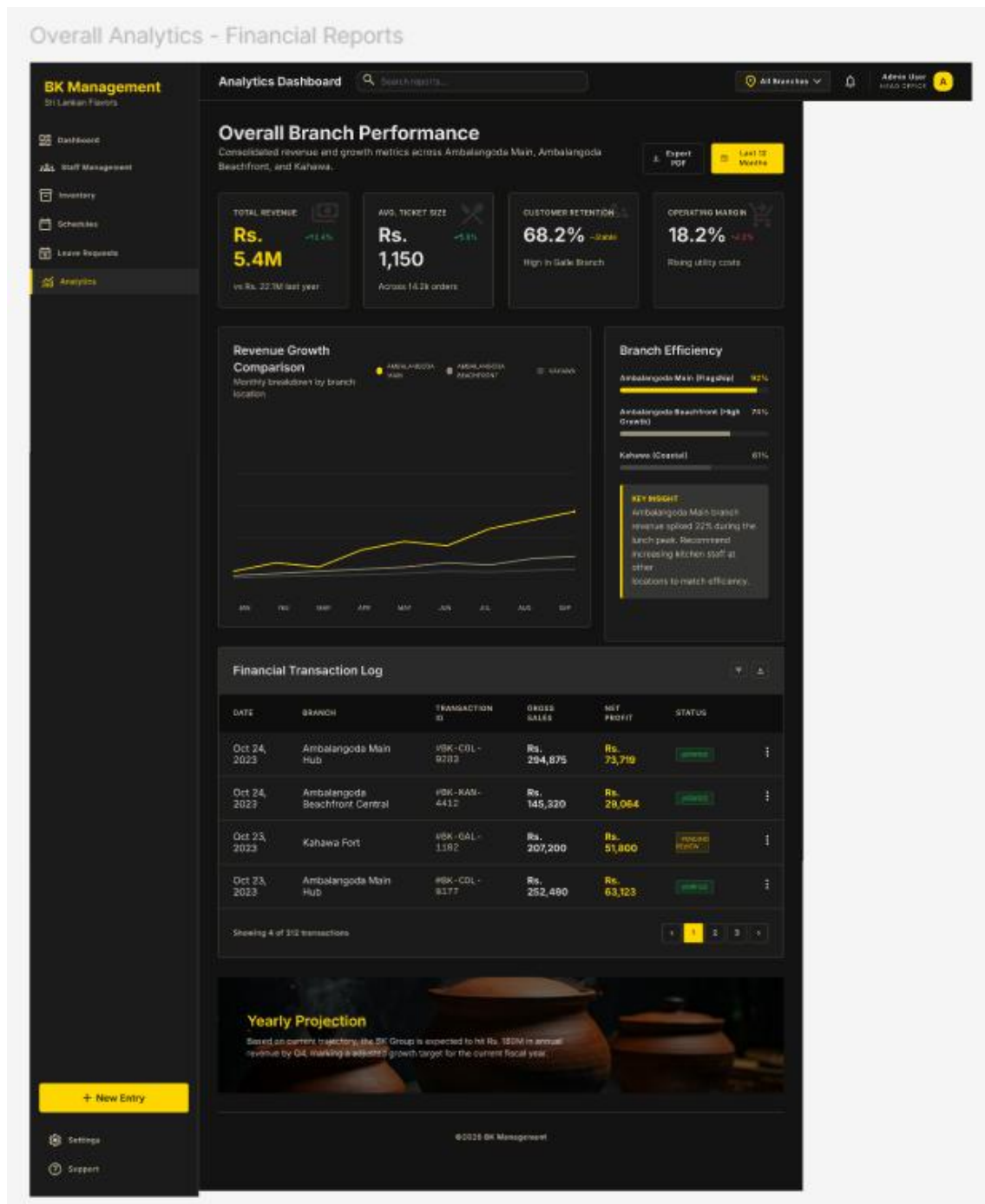
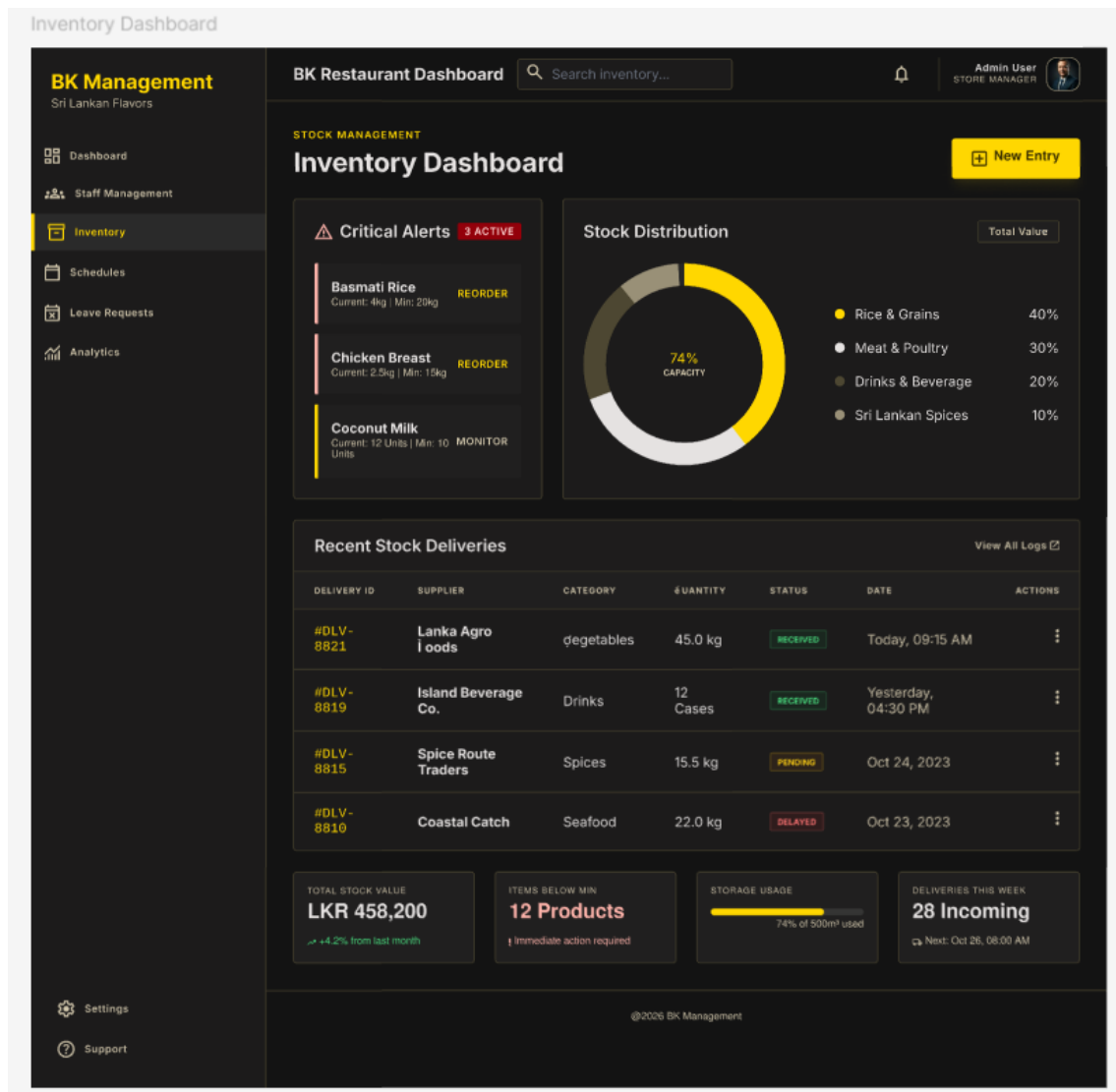


Figure 3.20: UI - Inventory Dashboard



UI - Stock Management

Stock Management & Control

BK Management
Sri Lankan Flavors

Dashboard

Staff Management

Inventory

Schedules

Leave Requests

Analytics

+ New Entry

Settings

Support

BK Restaurant Dashboard

Search inventory...

Inventory Management

Manage your stock levels, suppliers, and procurement alerts in real-time.

Total Categories
12

Low Stock Alerts
08

Out of Stock
03

Last Reorder Value
LKR 45,200

All Items

Perishables

Dry Goods

Liquids

ITEM NAME	CATEGORY	QTY	UNIT PRICE	STATUS	ACTIONS
Samba Rice (Premium)	Dry Goods	250 kg	LKR 210	IN STOCK	
Organic Tomatoes	Vegetables	5 kg	LKR 450	LOW STOCK	
Ceylon Cinnamon	Spices	15 kg	LKR 1,200	IN STOCK	
Chicken Breast	Meat	0 kg	LKR 1,150	OUT OF STOCK	
Ginger Beer (EGB)	Drinks	48 Units	LKR 180	IN STOCK	

Showing 5 of 124 items

1

2

3

UI - Stock Usage Analytics

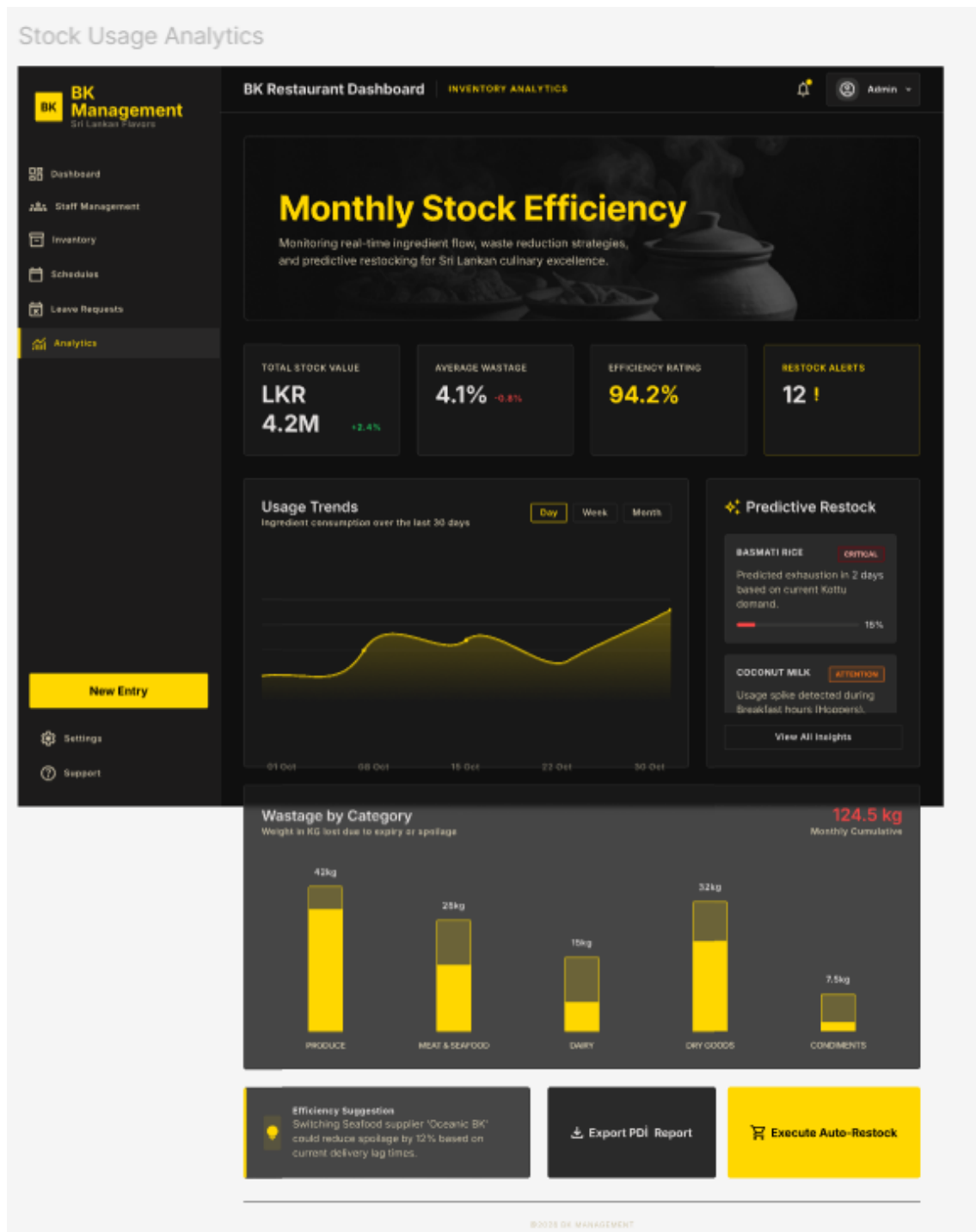
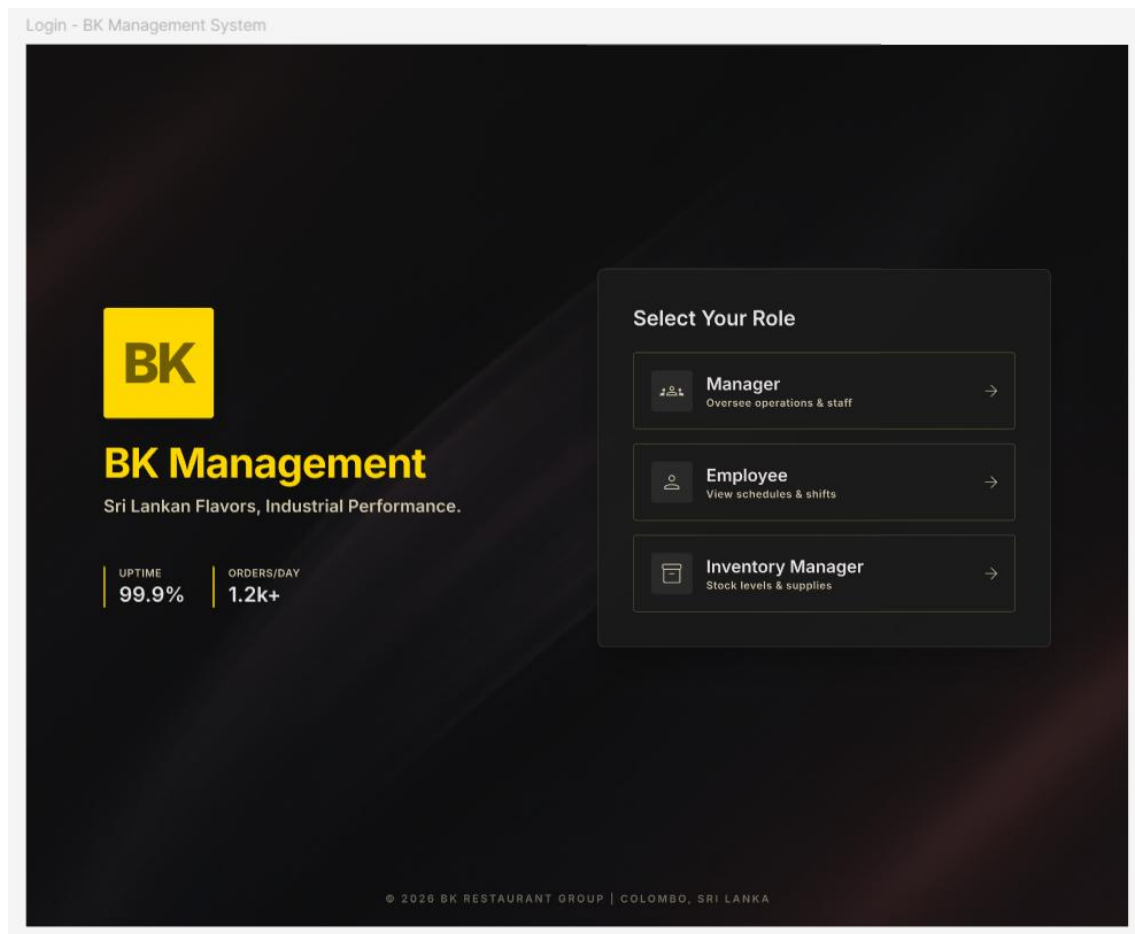


Figure 3.21: UI - Login Screen



UI - User Profile Settings


BK Management
SRI LANKAN FLAVORS

- Dashboard
- Staff Management
- Inventory
- Schedules
- Leave Requests
- Analytics
- Settings**
- Support

BK Restaurant Dashboard

Branch Selector
Logout



Dinesh Wickramasinghe
General Manager

Colombo Main Branch & Head Office
View Public Profile


Contact Information

FULL NAME

EMAIL ADDRESS

PHONE NUMBER

PREFERRED LANGUAGE

English (UK)

RESIDENTIAL ADDRESS

UPDATE PROFILE


Security & Access

Update Password

It's a good practice to change your password every 90 days to maintain high account security.

CHANGE PASSWORD

Two-Factor Authentication

Secure your account with SMS or Authenticator App.

☒

BRANCH ASSIGNMENT


Colombo HQ
 72 Galle Road, Colombo 03

Employee ID

BK-MO-0012

Joined Date

Mar 15, 2021

Reporting To

Dinesh Wickramasinghe


PRIMARY PERMISSIONS

FINANCIAL_READ

STAFF_WRITE

SYSTEM_ADMIN

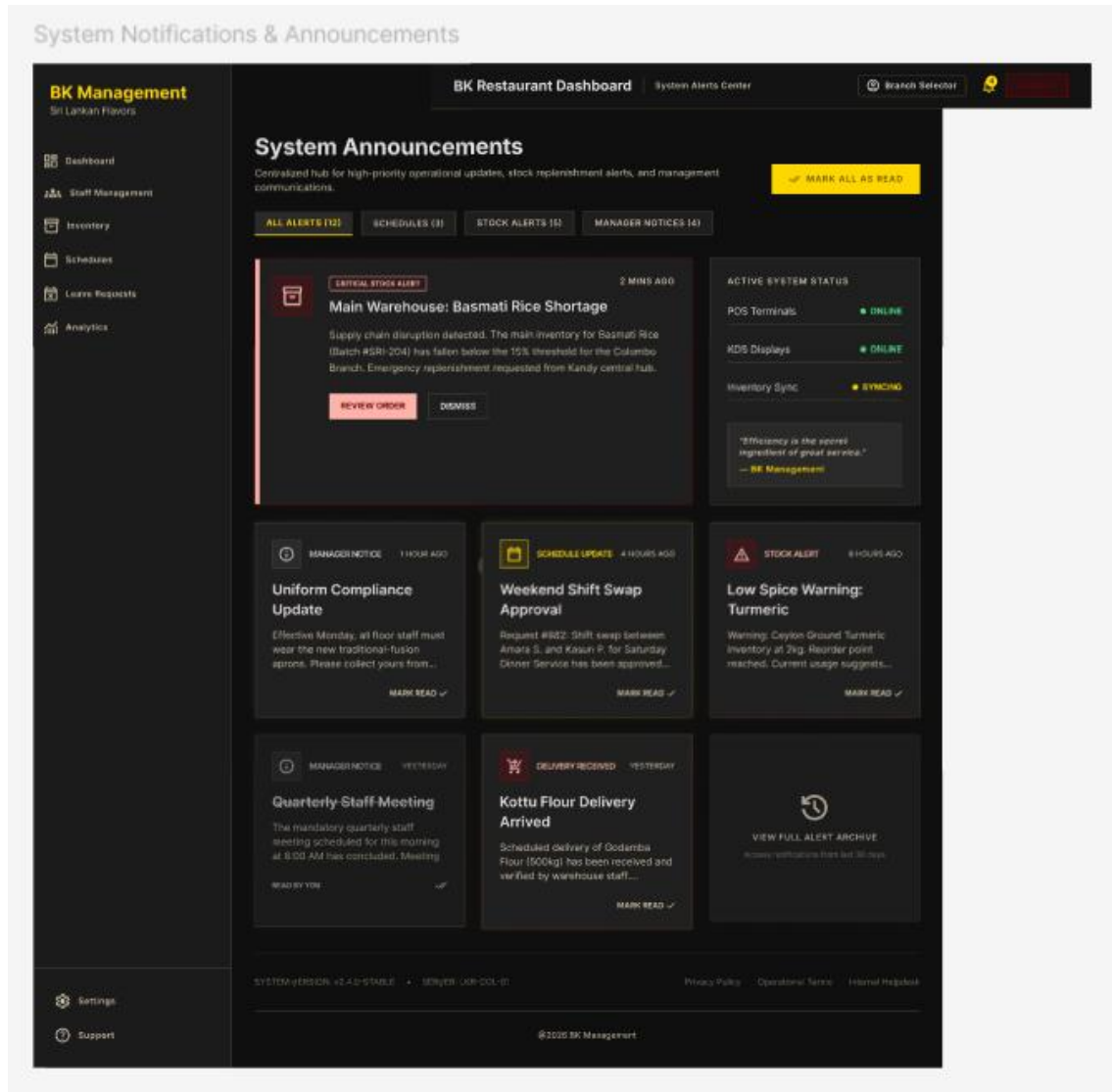
LOGIN SESSIONS
REVIEW ALL


MacBook Pro 16"
 Colombo • Current Session


iPhone 15 Pro
 Colombo • 2 hours ago


Windows Desktop
 Colombo • Yesterday

UI - System Notifications



3.5 Security Design (OWASP Top 10)

A secure system architecture was prioritized. The following OWASP Top 10 vulnerabilities were mitigated:

A01:2021-Broken Access Control: This was addressed via strict Role-Based Access Control (RBAC) and JWT validation on every API endpoint.

A02:2021-Cryptographic Failures: All data in transit is encrypted via TLS 1.3. User passwords are encrypted at rest using bcrypt with a salt factor of 10.

A03:2021-Injection: This was addressed by using a robust ORM which parameterizes all SQL queries, neutralizing SQL injection vectors.

3.6 Non-Functional Design Decisions Mapping

Non-functional requirements were successfully mapped to specific architectural implementations:

Performance (NFR-01): The requirement for the KDS to sync within 2 seconds was addressed by implementing WebSocket connections (Socket.io).

Reliability (NFR-02): The POS offline-first requirement was addressed using IndexedDB and Service Workers to cache transactions locally during outages.

3.7 UX Decisions and Rationale

To ensure the system is intuitive and rapid for restaurant staff, the user experience was driven by the following key decisions:

Large Touch Targets: The POS and Kitchen Display System (KDS) utilize large, tappable tiles to reduce input errors during fast-paced service, catering to staff with low-to-medium technical proficiency.

Color-Coded Indicators: Status indicators (e.g., green/red for stock levels) allow staff to instantly recognize urgent items at a glance.

Guided Rider Flow: The Delivery Rider App features a step-by-step guided flow with integrated navigation, minimizing decision fatigue while on the road.

Real-time Dashboards: The Owner Dashboard provides high-level visibility with pre-built charts, updating automatically in real-time without requiring manual page refreshes.

3.8 Interface & API Design

The system relies on a combination of internal and external interfaces to facilitate smooth operations:

External Integrations: The platform integrates with the Google Maps Platform for live GPS tracking of delivery riders, thermal receipt printers for physical POS invoicing, and cloud-based Email/SMS Gateways (e.g., SendGrid/Twilio) for automated notifications.

Internal API Structure: Internal communication between the React frontend, Flutter mobile app, and Node.js backend occurs via a versioned RESTful API (/api/v1/...). Payloads are exchanged in JSON format and secured using JWT-based authentication headers. Core backend modules communicate internally through direct service-layer calls.

3.9 Security Design

Robust security measures have been implemented to protect operational data and align with OWASP Top 10 guidelines:

Authentication & Authorization: All access is governed by a centralized Authentication Module utilizing salted bcrypt hashing and JWT tokens. Strict Role-Based Access Control (RBAC) middleware ensures users can only access endpoints and data pertinent to their specific branch and role.

Data Protection: Data in transit is strictly encrypted via HTTPS/TLS, preventing interception. Critical backend configurations and API keys are stored securely as environment variables.

Input Validation & Integrity: All incoming API requests are validated against strict schemas to reject malformed payloads. Status transitions (e.g., order dispatching) follow strict state machines to prevent invalid business operations. Comprehensive audit logs record all sensitive actions for traceability.

CHAPTER 4: CONCLUSION

4.1 Objectives Met

The BK Restaurant Management System was successfully designed to centralize branch operations and eliminate the inefficiencies of manual ledger tracking. The proposed system replaces ad-hoc WhatsApp communication with a robust Point of Sale (POS) and automated Kitchen Display System (KDS), thereby significantly reducing order preparation errors. Furthermore, the integration of a Flutter-based Rider Application

enables real-time GPS tracking of deliveries, fulfilling a critical requirement for operational oversight. By leveraging a centralized PostgreSQL database and React frontend, real-time analytics and inventory alerts are seamlessly provided to the business owner, directly addressing the pain point of sudden stock-outs.

4.2 Limitations

Despite the successful architectural design, certain limitations are acknowledged in the current proposed system. Direct integration with third-party delivery platforms such as UberEats and PickMe was excluded from the initial scope due to API licensing constraints. Consequently, orders originating from these platforms must still be manually entered into the POS by the cashiers. Additionally, the offline caching capability of the POS is limited to transactional data; heavy analytical queries cannot be executed until network connectivity is restored.

4.3 Future Work

Several enhancements are recommended for future iterations of the system. First, the development of a direct webhook integration for third-party food delivery services is proposed to fully automate external order ingestion. Second, a predictive analytics module powered by machine learning could be introduced to forecast inventory depletion rates based on historical seasonal data. Finally, a customer-facing web application could be deployed to allow patrons to place takeaway orders directly, further reducing the dependency on phone calls and WhatsApp messages.

CHAPTER 5: APPENDICES

Appendix A: Software Requirements Specification (SRS)

The complete and detailed Software Requirements Specification (SRS) document, which includes the full feasibility study, business processes, and comprehensive requirement breakdowns, has been attached as an appendix. Please navigate to the Appendix A: SRS bookmark in the PDF viewer to read it.

Appendix B: Software Design Specification (SDS)

The complete and detailed Software Design Specification (SDS) document, which includes the database schema, internal/external API specifications, detailed architecture, and security design, has been attached as an appendix. Please navigate to the Appendix B: SDS bookmark in the PDF viewer to read it.

BK Restaurant Management System

Software Requirements Specification

Course	SENG 31242 – System Design Project
Group	01
Team Members	SE/2022/020 – Sachin Lakshitha SE/2022/023 – Duvini Nimethra SE/2022/026 – Dinuja Ranaweera SE/2022/030 – Nethmi Navodya
Client	Mr. N. C. Sanjeewa De Silva, BK Restaurant & BK Bar
Document Status	Final

Table of Content

1. Introduction	4
1.1 Purpose and Scope.....	4
1.2 Definitions, Acronyms, and Abbreviations.....	4
1.3 References	5
1.4 Overview of Document	5
2. Overall Description.....	6
2.1 Product Perspective (Context Diagram).....	6
2.2 User Classes and Characteristics (Personas).....	7
2.3 Operating Environment	7
2.4 Design and Implementation Constraints	8
2.5 Assumptions and Dependencies	8
3. System Analysis	9
3.1 Fact-Gathering Techniques Used	9
3.2 Existing System Analysis (with Pain Points)	10
3.2.1 Current System Overview	10
3.2.2 Pain Points Identified	10
3.3 Use Case Diagram (System Boundary).....	11
3.4 Use Case Descriptions (one per use case)	12
UC-01: Log In and Access Role-Based Features.....	12
UC-02: Create Customer Orders	12
UC-03: Modify and Cancel Orders.....	13
UC-04: Update Order Status	13
UC-05: Generate Bills and Invoices	14
UC-06: Place External Orders	14
UC-07: Receive Delivery Assignments	15
UC-08: Update Delivery Status via Mobile	15
UC-09: Monitor Delivery Progress	16
UC-10: Manage Local Inventory	16
UC-11: Review Low-Stock Alerts.....	17
UC-12: Process Stock Requests.....	17
UC-13: Manage Employee Shifts and Schedules	18
UC-14: Record Employee Attendance	18
UC-15: Monitor Centralized Branch Operations & Analytics.....	19
3.5 Activity Diagrams (one per major use case)	20
AD-01: User Login and Role-Based Access	20
AD-02: Update Order Status	21
AD-03: Generate Bills and Invoices	22

AD-04: Receive Delivery Assignments	23
AD-05: Monitor Delivery Progress	24
AD-06: Manage Local Inventory	25
3.6 System Sequence Diagrams (High-Level)	26
SSD-01 — Log In and Access Role-Based Features	26
SSD-02 — Create Customer Orders	26
SSD-03 — Generate Bills and Invoices	27
SSD-04 — Place External Orders	28
SSD-05 — Receive Delivery Assignments	28
SSD-06 — Update Delivery Status via Mobile	29
SSD-07 — Monitor Delivery Progress	29
SSD-08 — Manage Employee Shifts and Record Attendance	30
SSD-09 — Review Low-Stock Alerts & Process Stock Requests	30
SSD-10— Monitor Centralized Branch Operations & Analytics	31
4. Functional Requirements	32
5. Non-Functional Requirements	34
6. Alternative Solutions & Feasibility Study	36
6.1 Alternative Solutions Analysed	36
Alternative 1 – Manual / Semi-Digital System (Current Approach)	36
Alternative 2 – Cloud-Based SaaS POS Solutions (e.g., Toast POS, Revel Systems)	36
Alternative 3 – Proposed Custom Web-Based System (BK Restaurant System)	37
6.2 Feasibility Study	37
6.2.1 Technical Feasibility	37
6.2.2 Economic Feasibility	37
6.2.3 Operational Feasibility	38
6.2.4 Schedule Feasibility	39
6.3 Justified Recommendation	39

1. Introduction

1.1 Purpose and Scope

The purpose of this document is to specify the complete software requirements for the BK Restaurant Management System. This system is designed to digitize and centralize operations for BK Restaurant and BK Bar across three branches located in Ambalangoda and Kahawa, Sri Lanka.

The scope of this system encompasses:

- A custom Point of Sale (POS) system for counter order management and billing
- A proprietary mobile application for delivery riders with live GPS tracking
- Automated inventory management with low-stock alerts and supplier notifications
- A centralized web dashboard for multi-branch monitoring and analytics
- An employee management module covering attendance, shift scheduling, and leave applications

This system directly replaces the current manual operations that rely on WhatsApp messaging, physical ledger books, and unmonitored delivery assignments.

1.2 Definitions, Acronyms, and Abbreviations

Definitions	
Restaurant Owner	The primary user who manages all three branches, oversees analytics, and approves business decisions.
Employee (Reception / Kitchen)	A supporting user who handles walk-in orders at the POS counter, prepares food based on real-time kitchen displays, and manages their attendance.
Delivery Rider	An operational user who receives delivery assignments on their mobile device and navigates to customer locations using GPS.
Supplier	A supporting external user who receives digital inventory requests and confirms stock availability.

Acronym	
POS	Point of Sale – the digital counter system used to process orders and generate bills.
GPS	Global Positioning System – used for live rider location tracking.
SRS	Software Requirements Specification
KDS	Kitchen Display System – a screen in the kitchen showing incoming orders.
UC	Use Case – a description of system interactions with actors.
FR	Functional Requirement – numbered FR-01, FR-02, etc.

Acronym	
NFR	Non-Functional Requirement – system quality attributes.
SME	Small and Medium Enterprise.
REST API	Representational State Transfer Application Programming Interface.
JWT	JSON Web Token – used for secure session authentication.

1.3 References

- Aggarwal, S. (2018). Modern Web Development with React. Packt Publishing.
- Boysen, N., Fedtke, S., & Schwerdfeger, S. (2021). Last-mile delivery concepts: a survey from an operational research perspective. *OR Spectrum*, 43, 1–58.
- Dixon, M., Freeman, K., & Toman, N. (2020). Digital POS systems and their impact on restaurant operations. *Cornell Hospitality Quarterly*, 61(2), 45–60.
- Google Developers. (2023). Flutter: Build apps for any screen. <https://flutter.dev>
- Gupta, S. (2016). The impact of technology on restaurant operations. *International Journal of Hospitality Management*, 55, 91–102.
- Heizer, J., Render, B., & Munson, C. (2019). *Operations Management: Sustainability and Supply Chain Management* (13th ed.). Pearson.
- Kasavana, M. L., & Cahill, J. J. (2007). *Managing Computers in the Hospitality Industry* (5th ed.). American Hotel & Lodging Educational Institute.
- Law, R., Buhalis, D., & Cobanoglu, C. (2013). Progress on information and communication technologies in hospitality and tourism. *International Journal of Contemporary Hospitality Management*, 26(5), 727–750.
- National Restaurant Association. (2023). *Restaurant Technology Landscape Report 2023*. <https://restaurant.org>
- Pigatto, G., Machado, J. G. C. F., Negreti, A. S., & Machado, L. M. (2017). Have you chosen your request? Analysis of online food delivery companies in Brazil. *British Food Journal*, 119(3), 639–657.
- Statista. (2024). Online food delivery market worldwide. <https://www.statista.com>

1.4 Overview of Document

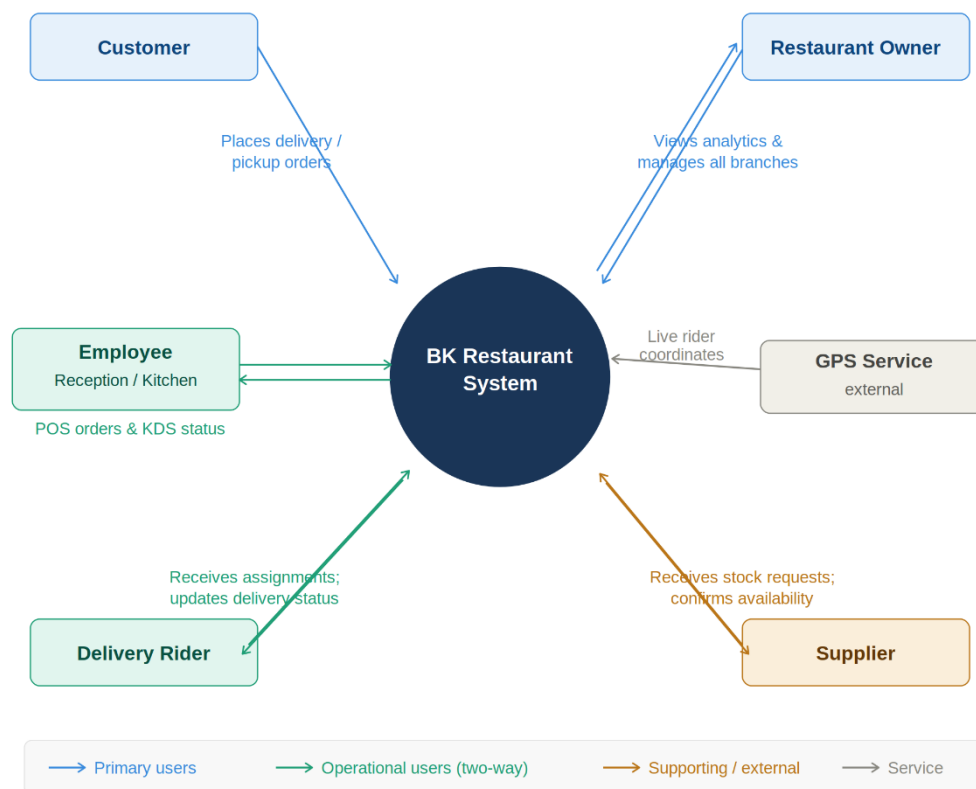
This document follows the standard SRS structure for SENG 31242:

1. Section 1 – Introduction: purpose, scope, definitions, acronyms, references.
2. Section 2 – Overall Description: product perspective, user classes, operating environment, constraints, and assumptions.
3. Section 3 – System Analysis: fact-gathering techniques, existing system analysis, use-case diagrams, use-case descriptions, activity diagrams, and system sequence diagrams.
4. Section 4 – Functional Requirements: numbered FR list (FR-01, FR-02, ...) with ID, title, description, priority, source, and verification method.
5. Section 5 – Non-Functional Requirements: performance, security, usability, reliability, scalability, and maintainability.
6. Section 6 – Alternative Solutions & Feasibility Study: at least two alternatives analysed, four-dimension feasibility assessment, and justified recommendation.

2. Overall Description

2.1 Product Perspective (Context Diagram)

The BK Restaurant Management System is a purpose-built, integrated platform replacing the existing fragmented manual operations. The system boundary encompasses three primary sub-systems: a web-based POS dashboard, a Flutter mobile application for riders, and a centralized backend API. The diagram below shows the system context and all external entities.



External Entity	Interaction	Direction
Customer	Places external orders; receives order confirmation	Input
Restaurant Owner	Monitors all branches; views analytics; manages staff and inventory	Both
Employee (Reception/Kitchen)	Processes walk-in orders via POS; updates kitchen status	Both
Delivery Rider	Receives delivery assignments on mobile app; updates delivery status	Both
Supplier	Receives automated stock requests; confirms availability	Both
GPS Service (ext.)	Provides real-time location data for rider tracking	Input

2.2 User Classes and Characteristics (Personas)

User Class	Persona	Tech Level	Primary Interface	Key Responsibilities
Restaurant Owner	Mr. Sanjeewa, 45, business owner across 3 branches. Comfortable with web apps.	Medium	Web Dashboard	Monitor branches, view financials, manage inventory, approve schedules
Reception Staff	Amali, 28, counter cashier. Limited tech experience.	Low–Medium	POS System	Create orders, generate bills, manage walk-ins
Kitchen Staff	Nishantha, 32, cook. Uses only the KDS screen.	Low	KDS Display	View and update order preparation status
Delivery Rider	Kusal, 24, motorcycle rider. Smartphone-literate.	Low	Mobile App	Receive assignments, navigate, update delivery status
Supplier	External vendor. Interacts via email/SMS notification.	Varies	Notification	Receive stock requests, confirm availability

2.3 Operating Environment

Component	Operating Environment
POS and Management System	Web-based application accessible through desktop computers, laptops, and tablets used by restaurant staff and management.
Client Platform	Modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.
Frontend Environment	React.js with Tailwind CSS for responsive and user-friendly interfaces.
Backend Environment	Node.js and Express.js running on a cloud-hosted server to support system functionality and API services.
Database Environment	PostgreSQL/MySQL database for storing orders, inventory, employee, customer, and financial data.
Hardware Environment	Desktop computers, tablets, POS terminals, receipt printers, and smartphones used for restaurant operations and delivery activities.
Network Environment	Internet connectivity is required for real-time synchronization between branches, online ordering, reporting, and external service integrations.
Hosting Environment	Cloud-based hosting platform such as AWS or Railway.app to provide system availability and scalability.

2.4 Design and Implementation Constraints

- System must function effectively in areas with poor internet connectivity (Kahawa/Ambalangoda), requiring local caching and synchronization mechanisms.
- Must remain cost-effective to avoid the high overhead of global SaaS competitors.
- UI must be extremely intuitive for non-technical staff, requiring minimal training.
- System shall be developed using a modular architecture (React.js frontend and Node.js backend) to support future enhancements and maintenance.
- System must support secure handling of customer, order, and business data through authentication and encryption mechanisms.
- System shall maintain reliable performance during peak restaurant operating hours when multiple users access the system simultaneously.

2.5 Assumptions and Dependencies

- Delivery riders possess personal smartphones with functioning GPS and internet connectivity for delivery tracking and navigation (if delivery module is enabled).
- Branches have the necessary hardware (PCs/Tablets and receipt printers) to support POS operations and order processing.
- A stable internet connection is available at restaurant branches to ensure real-time synchronization of orders, billing, and inventory data.
- Restaurant staff and management have basic digital literacy to operate the system with minimal training.
- The system depends on third-party services such as payment gateways, SMS/email notification services, and mapping APIs (if delivery tracking is included) for full functionality.

3. System Analysis

3.1 Fact-Gathering Techniques Used

The following methods were used to collect, understand, and confirm the requirements for the system:

Technique	Description	Output / Evidence Produced	Purpose in the Project
Structured Client Interviews	Conducted structured interviews with the client (Mr. N. C. Sanjeewa De Silva) using prepared questions to understand system requirements, business workflow, and pain points.	Interview transcript document and summarized notes, zoom meeting recording	To gather direct and accurate user requirements and real-world operational needs of the restaurant system
Competitor Analysis	Analysed existing restaurant management systems and compared their features, strengths, and weaknesses.	Feature comparison table	To identify industry standards, missing features, and improvement opportunities for the proposed system
Literature Review/ Document Analysis	Reviewed academic papers and industry sources related to restaurant management systems, POS systems, and digital ordering systems.	Literature review document	To understand existing technologies, best practices, and support system design decisions

Evidence Summary:

Evidence was gathered through a structured interview conducted with the owner of BK Restaurant to understand the challenges faced in managing operations across multiple branches. The interview revealed that restaurant activities, including order processing, inventory management, delivery coordination, and employee administration, are currently handled manually using WhatsApp messages, phone calls, and paper-based records. This has resulted in issues such as missed customer orders, lack of real-time delivery visibility, inconsistent inventory tracking, and limited oversight of employee attendance and schedules. Key findings highlighted the need for a centralized management solution that would enable the owner to monitor all branches from a single dashboard, digitize order processing, track deliveries in real time, automate inventory management, and streamline employee-related activities. The client also emphasized the importance of automated alerts, supplier request management, and comprehensive reporting capabilities. The findings were supported by the following evidence artifacts maintained within the project repository:

1. Client Interview Questions
2. Client Interview Session notes
3. Structured Interview Transcript Documentation
4. Competitor Analysis (Feature Comparison Table)
5. Literature Review (Academic and Industry Sources)

These artifacts provided the foundation for identifying business requirements, validating stakeholder needs, and defining the functional and non-functional requirements of the proposed BK Restaurant Management System.

3.2 Existing System Analysis (with Pain Points)

The current restaurant management process is largely manual and semi-digital, relying on WhatsApp messages, phone calls, physical registers, and spreadsheets. Each branch operates with limited integration, meaning data is not shared in real time across locations. This leads to inefficiencies in order handling, inventory tracking, and financial reporting.

3.2.1 Current System Overview

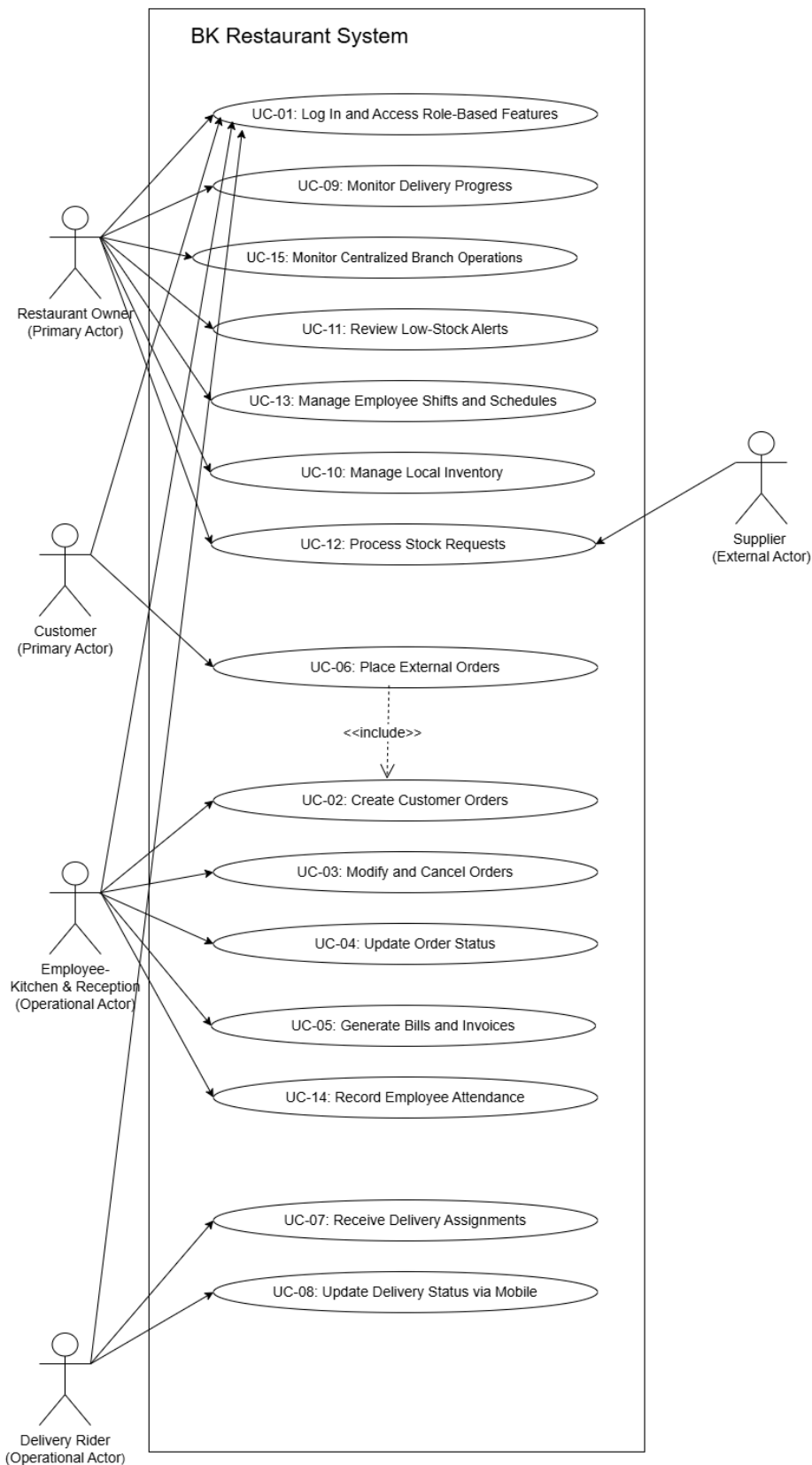
The existing system at the restaurant primarily uses WhatsApp for order communication between customers and staff, while kitchen operations and billing are managed through manual notes or basic POS tools. Delivery operations are handled without real-time tracking, and riders are dispatched based on availability without system assistance. Inventory and supplier management are done manually through physical logs and phone calls. Financial reporting across multiple branches is done at the end of each day using spreadsheets or paper records.

3.2.2 Pain Points Identified

Pain Point	Description	Impact on Business	Proposed Solution (Our System)
Missed or duplicated orders	Orders received via WhatsApp are manually tracked, leading to confusion during busy hours	Customer dissatisfaction and loss of revenue	Centralized order management system where all orders are placed and tracked in real time through a unified POS dashboard
Lack of delivery visibility	Delivery riders are dispatched without real-time tracking or status updates	Delays in delivery and poor customer experience	Real-time delivery tracking system with GPS-enabled rider updates and live order status monitoring
Inventory stockouts	Stock levels are updated manually, leading to delayed awareness of low inventory	Frequent stockouts and disruption in service	Automated inventory management system with low-stock alerts and real-time stock updates across branches
Manual financial reconciliation	Cross-branch financial reports are prepared manually using spreadsheets	Time-consuming and error-prone financial analysis	Automated financial reporting system with real-time dashboards and cross-branch analytics
Paper-based staff management	Staff schedules and shifts are managed using paper or informal communication	Scheduling conflicts and inefficient workforce allocation	Digital staff management and scheduling module with centralized shift planning and updates

3.3 Use Case Diagram (System Boundary)

The use case diagram below illustrates the system boundary and the interactions between all actors and the BK Restaurant Management System. The system covers five primary actors: Restaurant Owner, Customer, Employee (Reception/Kitchen), Delivery Rider, and Supplier.



3.4 Use Case Descriptions (one per use case)

The following use cases define the complete scope of system interactions. Full detailed descriptions for each use case are provided below.

UC-01: Log In and Access Role-Based Features

Use Case ID	UC-01
Use Case Name	Log In and Access Role-Based Features
Actor(s)	Restaurant Owner, Branch Managers, Employees/Cashiers, Delivery Riders, Suppliers
Preconditions	User is registered in the system and is not currently logged in.
Postconditions – Success	User is authenticated and redirected to their role-specific dashboard.
Postconditions – Failure	Authentication fails; user remains on the login page.
Main Flow	1. User navigates to the login screen. 2. User enters username and password. 3. System validates credentials. 4. System retrieves user role. 5. System grants access to role-based features.
Alternative Flow 1	At step 3: If credentials are invalid, system displays an 'Invalid login' error message.
Alternative Flow 2	At step 2: User clicks 'Forgot Password', redirecting to password reset flow.
Business Rules	Users can only access modules permitted by their assigned role.

UC-02: Create Customer Orders

Use Case ID	UC-02
Use Case Name	Create Customer Orders
Actor(s)	Employees/Cashiers
Preconditions	Employee is logged in to the POS module.
Postconditions – Success	Order is recorded, inventory is marked for update, and kitchen is notified.
Postconditions – Failure	Order is discarded; no inventory changes occur.
Main Flow	1. Employee initiates new order. 2. Employee selects order type (Dine-in, Takeaway, Delivery). 3. Employee adds menu items. 4. System calculates total. 5. Employee confirms order and accepts payment. 6. System saves order.
Alternative Flow 1	At step 3: If an item is out of stock, system prevents adding it and displays an alert.
Alternative Flow 2	None
Business Rules	Order total must be greater than zero.

UC-03: Modify and Cancel Orders

Use Case ID	UC-03
Use Case Name	Modify and Cancel Orders
Actor(s)	Employees/Cashiers
Preconditions	Employee is logged in; an active order exists.
Postconditions – Success	Order is successfully updated or cancelled; system logs the change.
Postconditions – Failure	Order remains in its previous state.
Main Flow	1. Employee searches for an active order. 2. System displays order details. 3. Employee selects 'Modify' or 'Cancel'. 4. Employee makes changes or confirms cancellation. 5. System updates order status and recalculates totals if modified.
Alternative Flow 1	At step 3: If order is already in 'Dispatched' or 'Completed' status, system denies modification.
Alternative Flow 2	None
Business Rules	Orders cannot be cancelled once they are dispatched.

UC-04: Update Order Status

Use Case ID	UC-04
Use Case Name	Update Order Status
Actor(s)	Employees/Cashiers
Preconditions	Employee is logged in; active orders exist in the queue.
Postconditions – Success	Order status is updated (e.g., Preparing, Ready, Completed).
Postconditions – Failure	Status update fails; error logged.
Main Flow	1. Employee views active order queue. 2. Employee selects an order. 3. Employee changes status from 'Pending' to 'Preparing'. 4. System saves the new status. 5. Status updates on real-time displays.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	Status progression must follow a logical sequence (Pending -> Preparing -> Ready).

UC-05: Generate Bills and Invoices

Use Case ID	UC-05
Use Case Name	Generate Bills and Invoices
Actor(s)	Employees/Cashiers
Preconditions	Order has been created and confirmed.
Postconditions – Success	Digital bill/invoice is generated and stored in order history.
Postconditions – Failure	Invoice generation fails; user is prompted to retry.
Main Flow	1. Employee finalizes an order. 2. System calculates final amount including taxes/fees. 3. System generates a digital invoice. 4. System offers option to print or email the invoice. 5. System archives invoice in order history.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	Invoices must include a unique transaction ID and timestamp.

UC-06: Place External Orders

Use Case ID	UC-06
Use Case Name	Place External Orders
Actor(s)	Customers
Preconditions	Customer has access to phone or WhatsApp.
Postconditions – Success	Order is received by the restaurant and entered into the system.
Postconditions – Failure	Order is not placed (e.g., outside delivery hours).
Main Flow	1. Customer contacts restaurant via Phone/WhatsApp. 2. Customer provides order details and delivery address. 3. Employee enters details into the system (invoking UC-02). 4. Customer receives confirmation.
Alternative Flow 1	At step 2: If address is outside delivery zone, customer is informed.
Alternative Flow 2	None
Business Rules	Delivery orders require a valid contact number and address.

UC-07: Receive Delivery Assignments

Use Case ID	UC-07
Use Case Name	Receive Delivery Assignments
Actor(s)	Delivery Riders
Preconditions	Rider is logged into the mobile app and marked as 'Available'.
Postconditions – Success	Order is assigned to the rider; order status becomes 'Dispatched'.
Postconditions – Failure	Assignment fails; order remains 'Ready for Delivery'.
Main Flow	1. System identifies a 'Ready' delivery order. 2. System assigns order to an available rider. 3. Rider receives notification on mobile app. 4. Rider accepts assignment. 5. System updates order status to 'Dispatched'.
Alternative Flow 1	At step 4: Rider rejects assignment; system reassigns to the next available rider.
Alternative Flow 2	None
Business Rules	Riders can only be assigned orders from their designated branch.

UC-08: Update Delivery Status via Mobile

Use Case ID	UC-08
Use Case Name	Update Delivery Status via Mobile
Actor(s)	Delivery Riders
Preconditions	Rider has an active assigned delivery.
Postconditions – Success	Delivery is marked 'Completed'; customer notified.
Postconditions – Failure	Status remains unchanged; rider prompted to retry.
Main Flow	1. Rider arrives at delivery location. 2. Rider hands over the order. 3. Rider opens mobile app. 4. Rider marks order as 'Delivered'. 5. System logs completion time and updates central database.
Alternative Flow 1	At step 2: Customer is unavailable; rider marks order as 'Delivery Failed'.
Alternative Flow 2	None
Business Rules	Riders must be within reasonable GPS proximity to the delivery address to mark as delivered.

UC-09: Monitor Delivery Progress

Use Case ID	UC-09
Use Case Name	Monitor Delivery Progress
Actor(s)	Restaurant Owner
Preconditions	Owner is logged in; deliveries are currently active.
Postconditions – Success	Owner successfully views real-time delivery locations.
Postconditions – Failure	Map or tracking data fails to load.
Main Flow	1. Owner navigates to 'Delivery Tracking' dashboard. 2. System retrieves GPS coordinates of active riders. 3. System overlays rider locations and estimated delivery times on a map view. 4. Owner selects a specific rider for detailed order info.
Alternative Flow 1	At step 2: If a rider's GPS is off, system displays their last known location and a warning.
Alternative Flow 2	None
Business Rules	GPS data must be updated at least every 60 seconds.

UC-10: Manage Local Inventory

Use Case ID	UC-10
Use Case Name	Manage Local Inventory
Actor(s)	Branch Managers
Preconditions	Branch Manager is logged into the inventory module.
Postconditions – Success	Inventory counts are accurately adjusted and saved.
Postconditions – Failure	Inventory update fails; previous counts remain.
Main Flow	1. Manager navigates to 'Inventory Management'. 2. System displays current stock levels. 3. Manager performs manual stock count. 4. Manager enters adjustments into the system. 5. System saves updated inventory records.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	Inventory updates automatically after sales; manual adjustments require a logged reason.

UC-11: Review Low-Stock Alerts

Use Case ID	UC-11
Use Case Name	Review Low-Stock Alerts
Actor(s)	Branch Managers
Preconditions	Inventory item falls below the defined threshold.
Postconditions – Success	Manager acknowledges alert and initiates a stock request.
Postconditions – Failure	Manager ignores alert; item risks stocking out.
Main Flow	1. System detects an item count below minimum threshold. 2. System generates a low-stock alert. 3. Manager reviews alerts on dashboard. 4. Manager acknowledges the alert. 5. Manager proceeds to create a stock request.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	Alerts remain active until inventory count is increased above the threshold.

UC-12: Process Stock Requests

Use Case ID	UC-12
Use Case Name	Process Stock Requests
Actor(s)	Branch Managers, Suppliers
Preconditions	Manager has identified a need for supplies.
Postconditions – Success	Stock request is submitted and fulfilled by supplier.
Postconditions – Failure	Stock request is rejected or fails to send.
Main Flow	1. Manager creates a new stock request. 2. Manager adds items and quantities. 3. System sends request to corresponding Supplier. 4. Supplier logs in and reviews request. 5. Supplier approves and dispatches items. 6. Manager confirms receipt.
Alternative Flow 1	At step 5: Supplier lacks stock and rejects/modifies the request.
Alternative Flow 2	None
Business Rules	Received stock automatically updates branch inventory counts.

UC-13: Manage Employee Shifts and Schedules

Use Case ID	UC-13
Use Case Name	Manage Employee Shifts and Schedules
Actor(s)	Branch Managers
Preconditions	Manager is logged into the employee module.
Postconditions – Success	Weekly schedule is published and accessible to employees.
Postconditions – Failure	Schedule is not saved.
Main Flow	1. Manager opens the scheduling tool. 2. Manager assigns employees to specific shifts. 3. System checks for scheduling conflicts. 4. Manager finalizes the schedule. 5. System notifies employees of their upcoming shifts.
Alternative Flow 1	At step 3: System detects an overlapping shift and prompts manager to resolve it.
Alternative Flow 2	None
Business Rules	Employees cannot be scheduled for overlapping shifts.

UC-14: Record Employee Attendance

Use Case ID	UC-14
Use Case Name	Record Employee Attendance
Actor(s)	Employees/Cashiers
Preconditions	Employee is physically at the branch and has system access.
Postconditions – Success	Clock-in/clock-out time is recorded in the system.
Postconditions – Failure	Attendance record fails to save.
Main Flow	1. Employee accesses the attendance module. 2. Employee enters unique pin or credentials. 3. Employee selects 'Clock In' (or 'Clock Out'). 4. System logs the timestamp and employee ID. 5. System displays confirmation.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	Clock-in must occur within the designated branch network or location.

UC-15: Monitor Centralized Branch Operations & Analytics

Use Case ID	UC-15
Use Case Name	Monitor Centralized Branch Operations & Analytics
Actor(s)	Restaurant Owner
Preconditions	Owner is logged into the central admin dashboard.
Postconditions – Success	Owner successfully generates and views performance reports.
Postconditions – Failure	Data fails to aggregate; dashboard displays an error.
Main Flow	1. Owner opens the analytics dashboard. 2. System aggregates data from all branches. 3. System displays sales, inventory, and delivery metrics. 4. Owner applies filters (e.g., date range, specific branch). 5. System updates visual charts and data tables.
Alternative Flow 1	None
Alternative Flow 2	None
Business Rules	Analytics dashboard is strictly restricted to the Restaurant Owner role.

3.5 Activity Diagrams (one per major use case)

The following activity diagrams illustrate the detailed workflows of the major use cases in the BK Restaurant Management System. Each diagram shows the interactions between actors and the system, including decision points, validations, notifications, and exception handling scenarios.

AD-01: User Login and Role-Based Access

Description:

This activity diagram illustrates the authentication process of the BK Restaurant Management System. The user enters login credentials, and the system validates them. If the credentials are valid, the system retrieves the user's role and redirects the user to the appropriate dashboard with role-based features. Invalid credentials result in access denial and a request to re-enter credentials.

Actors Involved:

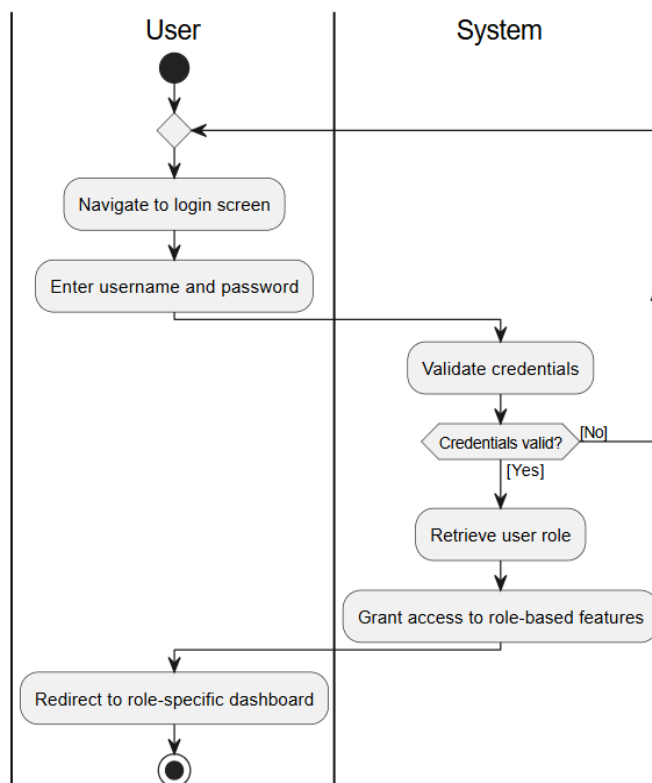
- User
- System

Use Cases Covered:

- UC-01 Log In and Access Role-Based Features

Activity Diagram:

Activity Diagram: UC-01 Log In and Access Role-Based Features



AD-02: Update Order Status

Description:

This activity diagram illustrates how kitchen staff manage the order preparation workflow. The kitchen selects pending orders, updates their status to “Preparing,” completes food preparation, and marks orders as “Ready.” The system automatically notifies the cashier or delivery rider. If issues such as missing ingredients occur, the system records the issue and allows cancellation or refund procedures.

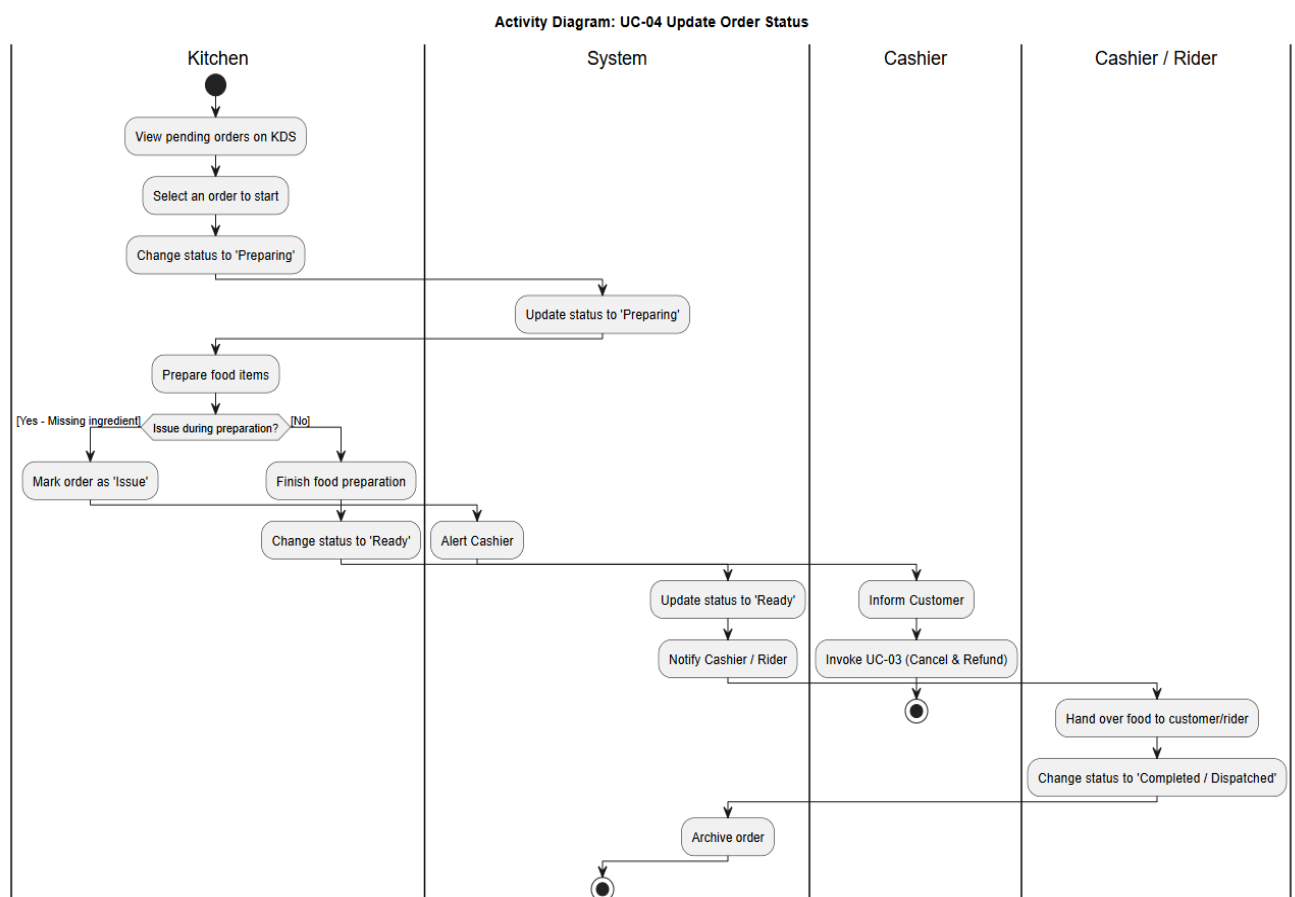
Actors Involved:

- Kitchen Staff
- System
- Cashier
- Rider

Use Cases Covered:

- UC-04 Update Order Status

Activity Diagram:



AD-03: Generate Bills and Invoices

Description:

This activity diagram describes the invoice generation process after an order is finalized. The system calculates taxes and fees, generates a digital invoice, and allows the cashier to either print or email the invoice. Error handling mechanisms are included for invoice generation failures and printer connectivity issues.

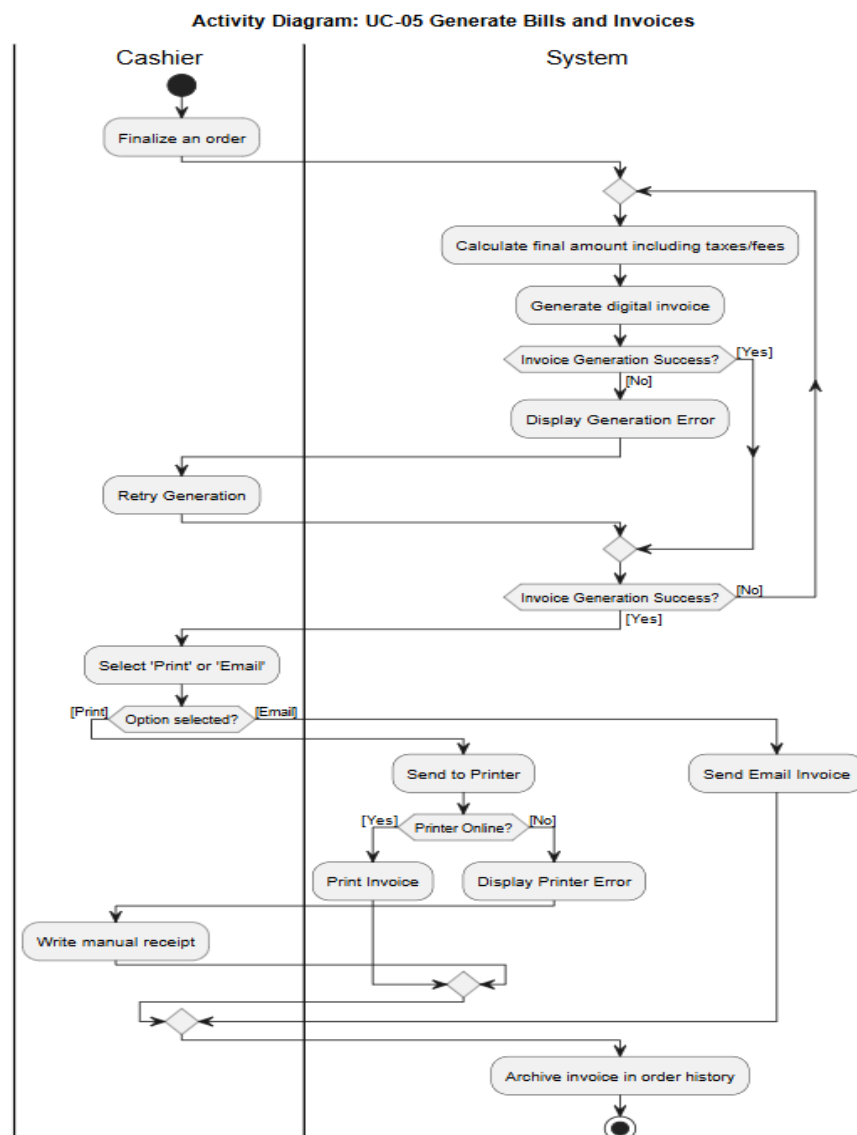
Actors Involved:

- Cashier
- System

Use Cases Covered:

- UC-05 Generate Bills and Invoices

Activity Diagram:



AD-04: Receive Delivery Assignments

Use Case: UC-07 Receive Delivery Assignments

Description:

This activity diagram illustrates the delivery assignment process. The system identifies a ready order and checks rider availability. Available riders receive delivery notifications through the mobile application and may accept or reject assignments. If no riders are available, the system generates alerts and supports alternative delivery handling procedures.

Actors Involved:

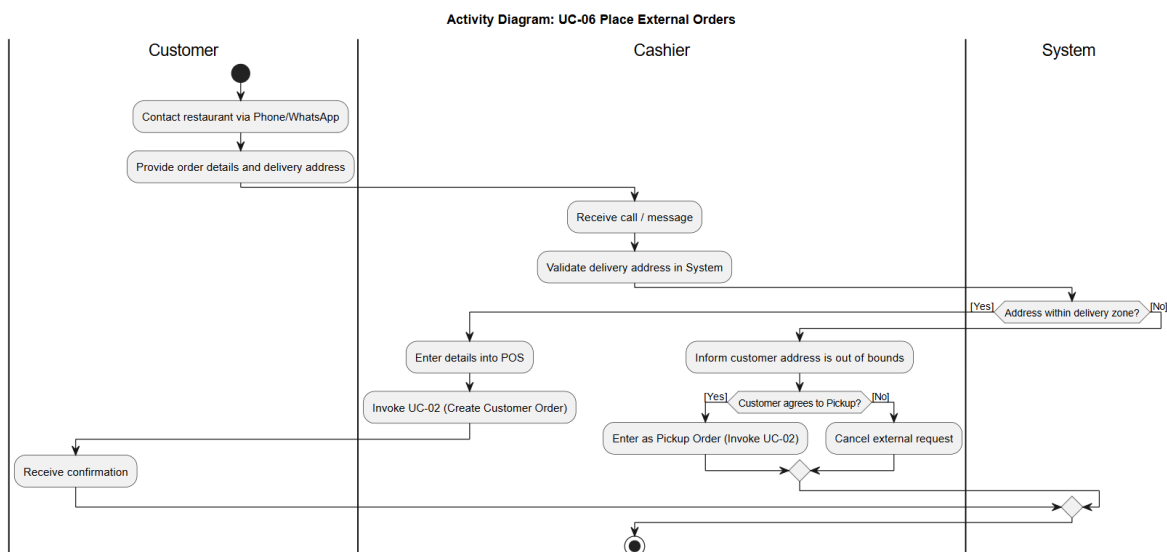
- System
- Delivery Rider
- Branch Manager

Use Cases Covered:

- UC-07 Receive Delivery Assignments

Activity Diagram:

Figure 3.5.4: Activity Diagram for Receive Delivery Assignments (UC-07)



AD-05: Monitor Delivery Progress

Description:

This activity diagram illustrates how restaurant owners monitor ongoing deliveries. The system continuously tracks rider GPS locations and updates estimated delivery times. If GPS tracking becomes unavailable, the system displays warnings and the rider's last known location to support delivery supervision.

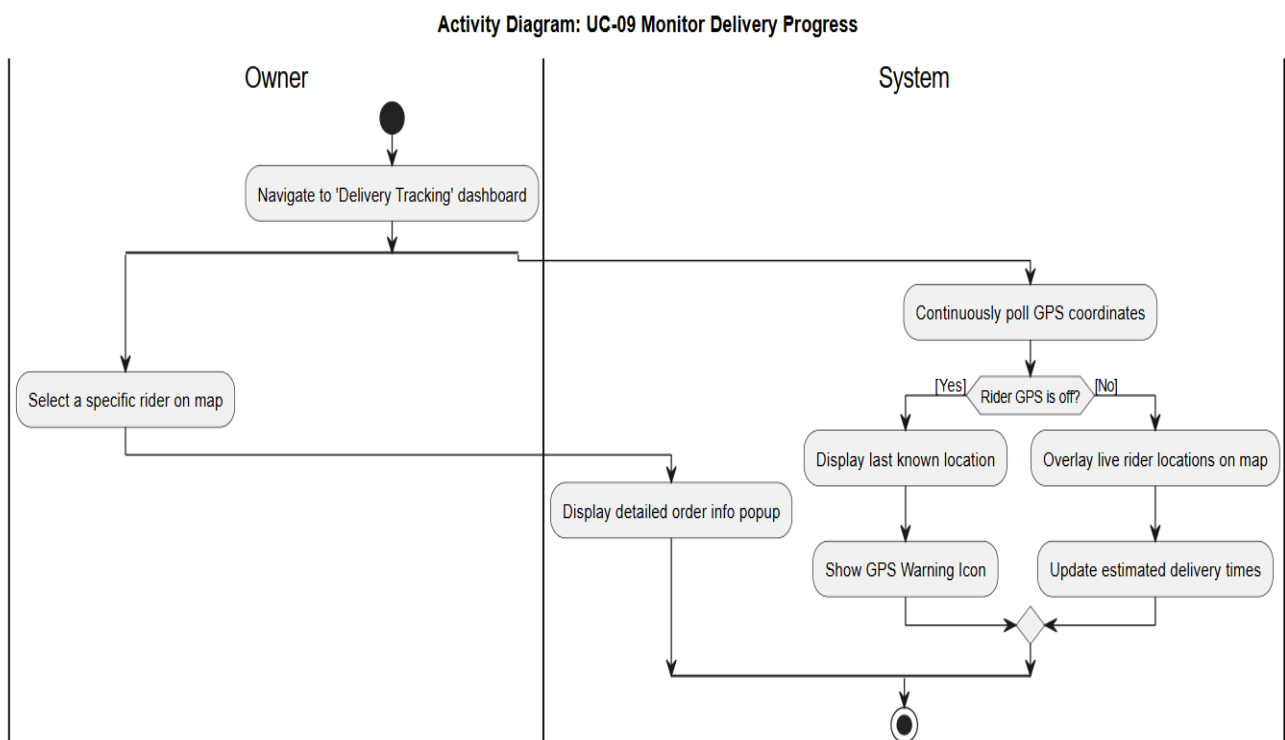
Actors Involved:

- Restaurant Owner
- System

Use Cases Covered:

- UC-09 Monitor Delivery Progress

Activity Diagram:



AD-06: Manage Local Inventory

Description:

This activity diagram illustrates inventory management activities performed by branch managers. Managers review inventory levels, perform physical stock counts, compare counts with system records, and make stock adjustments when discrepancies are identified. The system records adjustment reasons, timestamps, and manager information for auditing purposes.

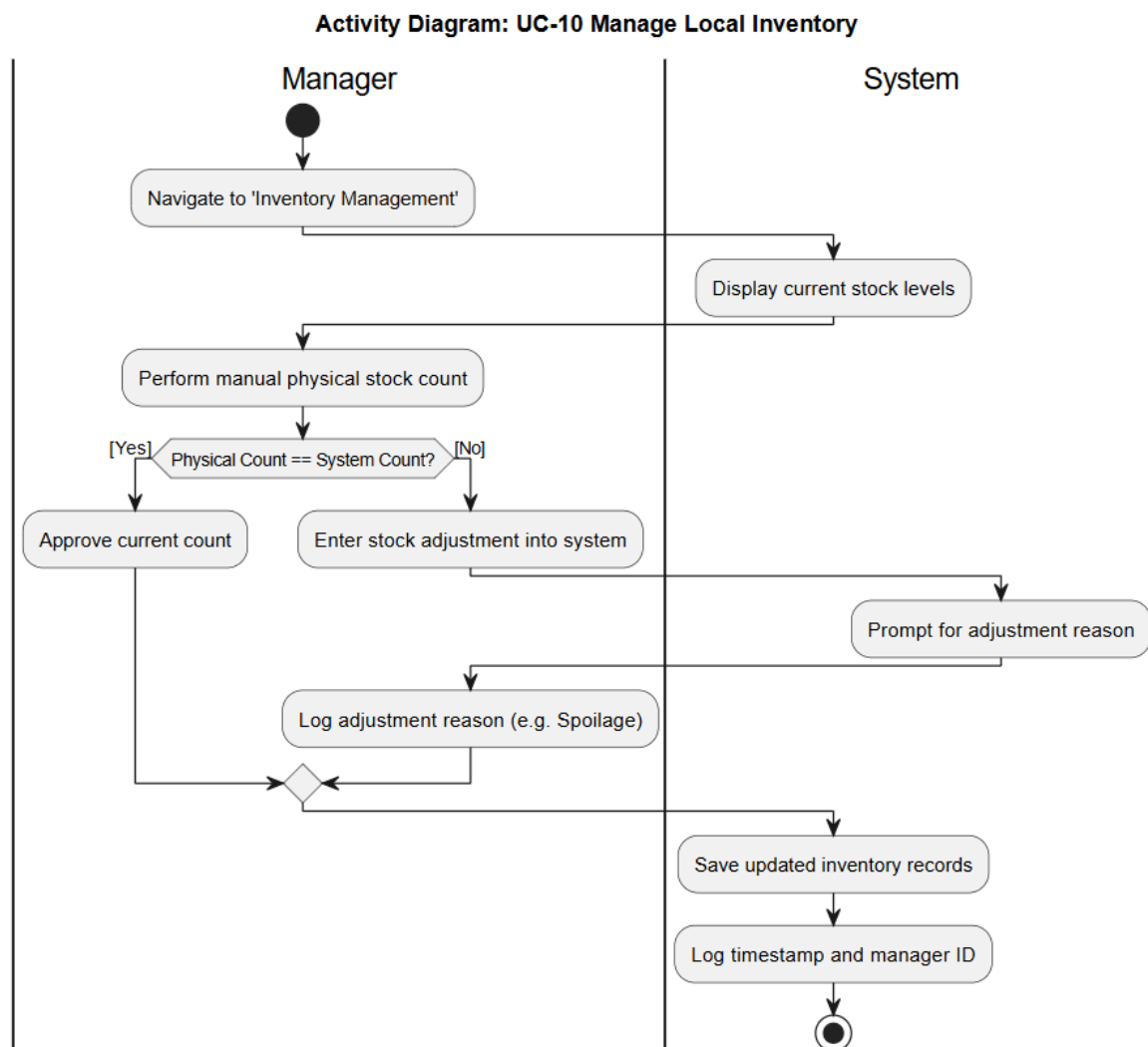
Actors Involved:

- Branch Manager
- System

Use Cases Covered:

- UC-10 Manage Local Inventory

Activity Diagram:

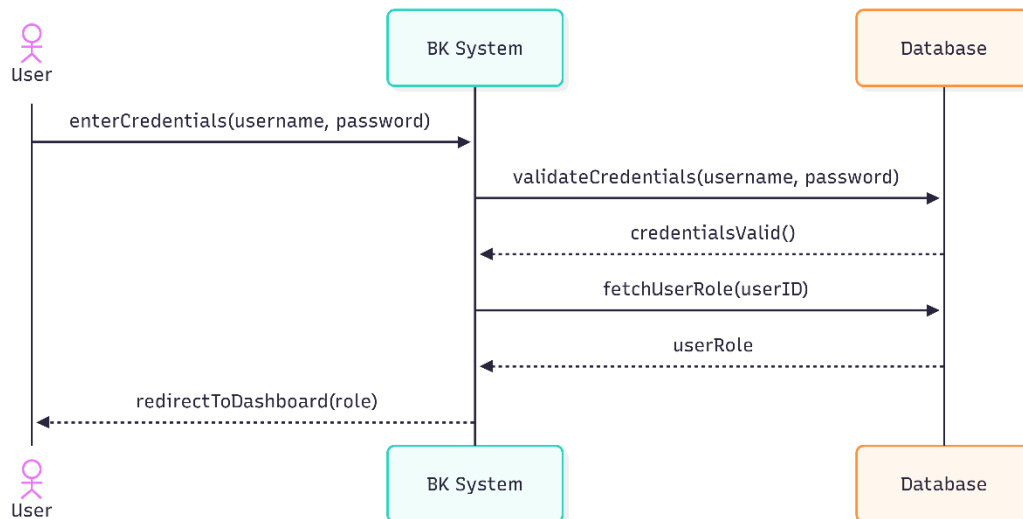


3.6 System Sequence Diagrams (High-Level)

System Sequence Diagrams show the high-level interactions between actors and the BK Restaurant System boundary for the key use cases.

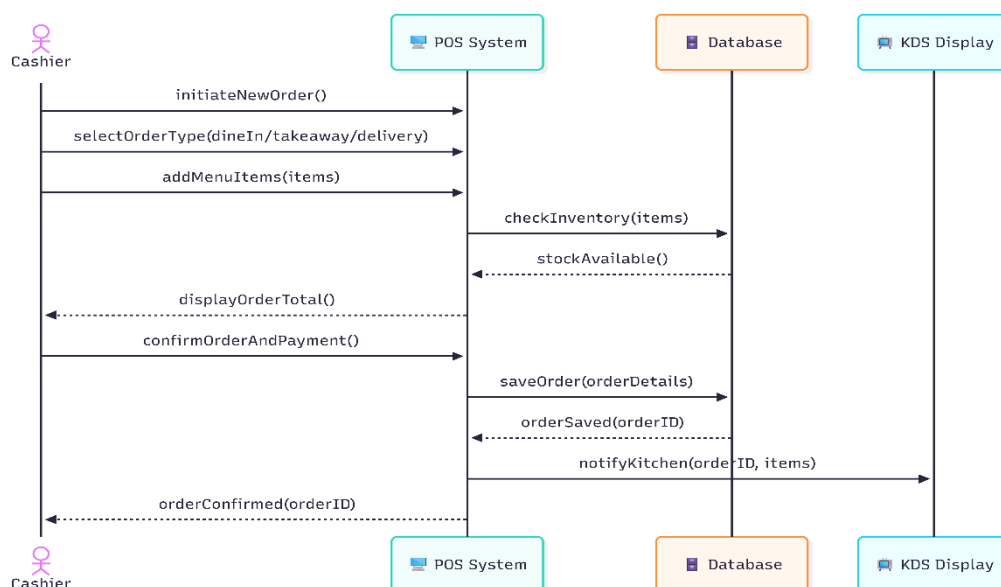
SSD-01 — Log In and Access Role-Based Features

This diagram shows the user authentication process in the BK Restaurant Management System. After login, the system verifies the user's credentials, identifies their role, and redirects them to the appropriate dashboard with role-based access and permissions.



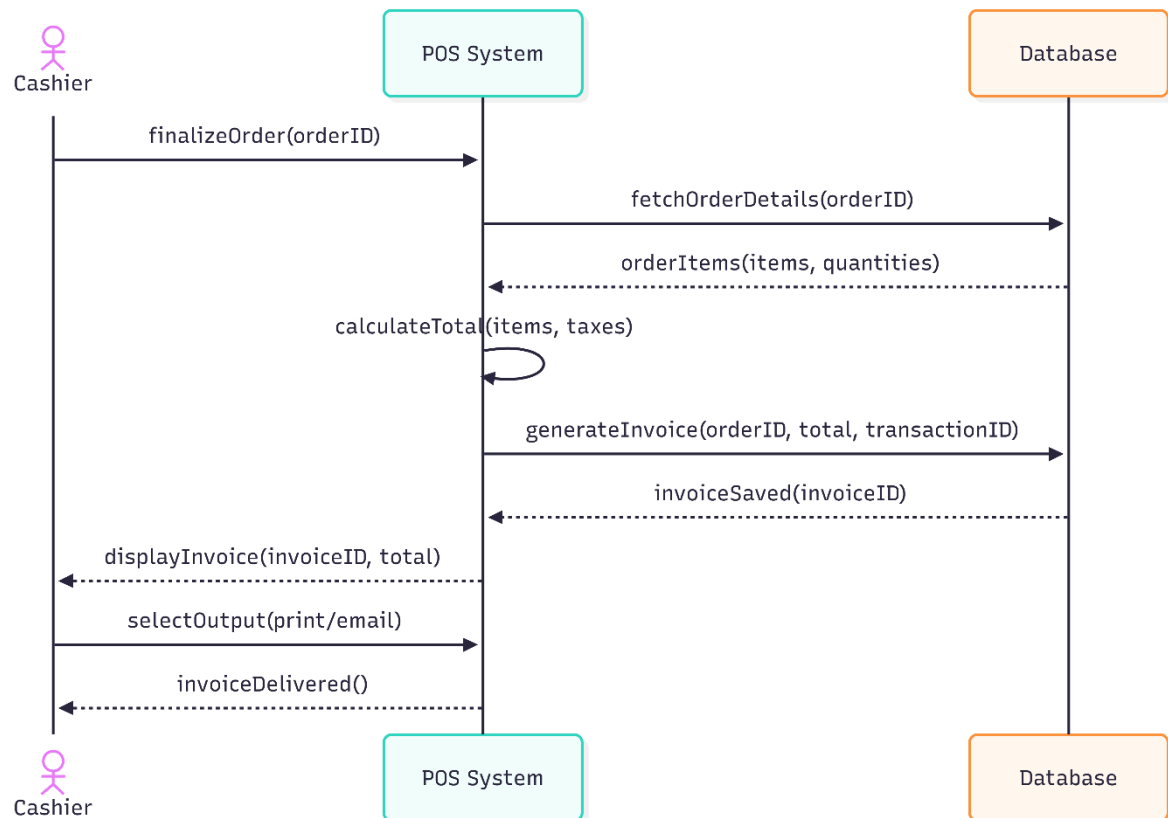
SSD-02 — Create Customer Orders

This diagram illustrates the order creation process in the BK Restaurant Management System. The Cashier selects the order type, adds menu items, and confirms the order after payment. The system verifies inventory availability, saves the order, and instantly notifies the Kitchen Display System (KDS) for preparation.



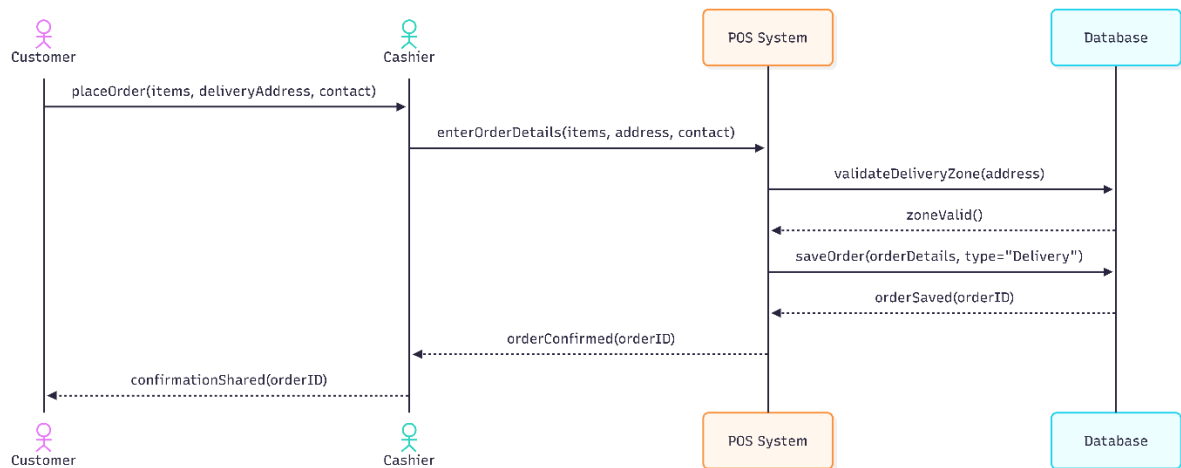
SSD-03 — Generate Bills and Invoices

This diagram illustrates the billing process in the BK Restaurant Management System. The Cashier finalizes the order, the system calculates the total amount and generates an invoice, then saves it with a unique reference. The invoice is delivered to the customer either as a printed copy or via email, completing the transaction.



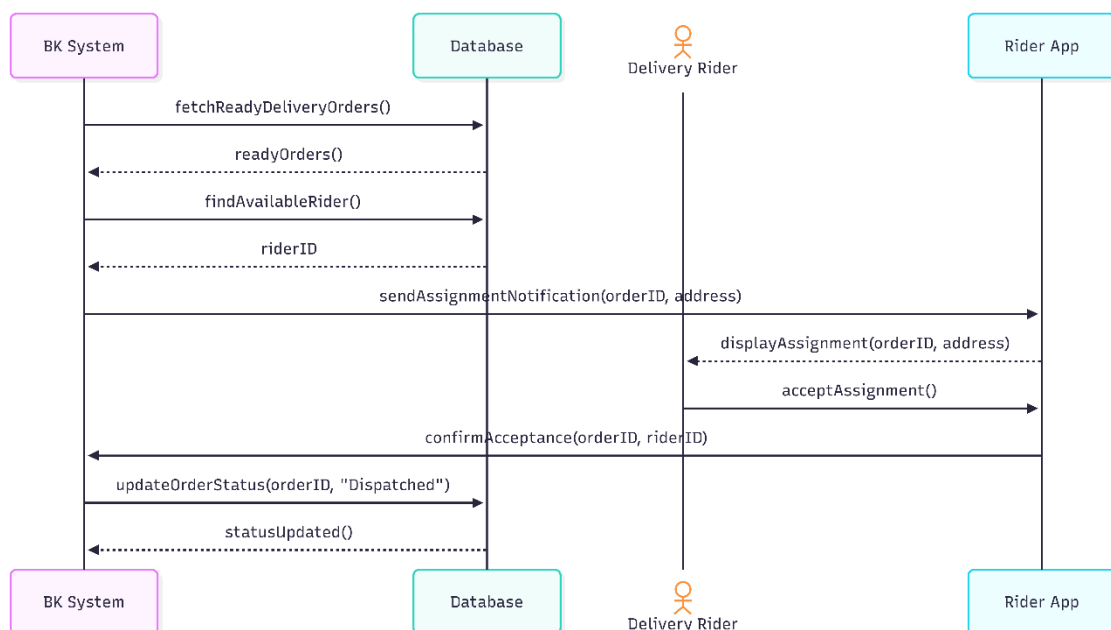
SSD-04 — Place External Orders

This diagram illustrates the delivery order process in the BK Restaurant Management System. The Cashier records the customer's order and delivery details, and the system verifies that the address is within the delivery zone. Once validated, the order is saved as a delivery order, and the customer receives an order ID for tracking purposes.



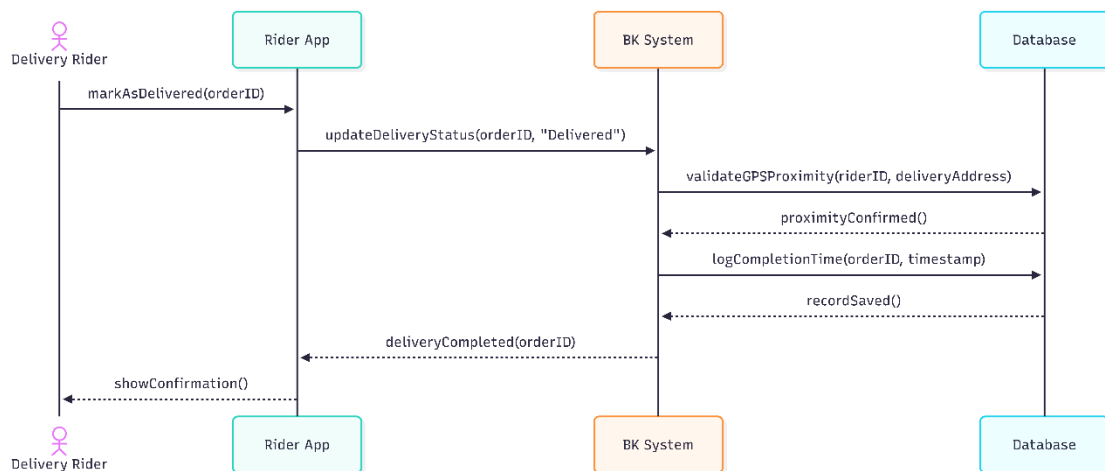
SSD-05 — Receive Delivery Assignments

This diagram illustrates the rider assignment process in the BK Restaurant Management System. When an order is marked as ready, the system automatically assigns it to an available rider and sends the delivery details. After the rider accepts the assignment, the system updates the order status to "Dispatched," initiating the delivery process.



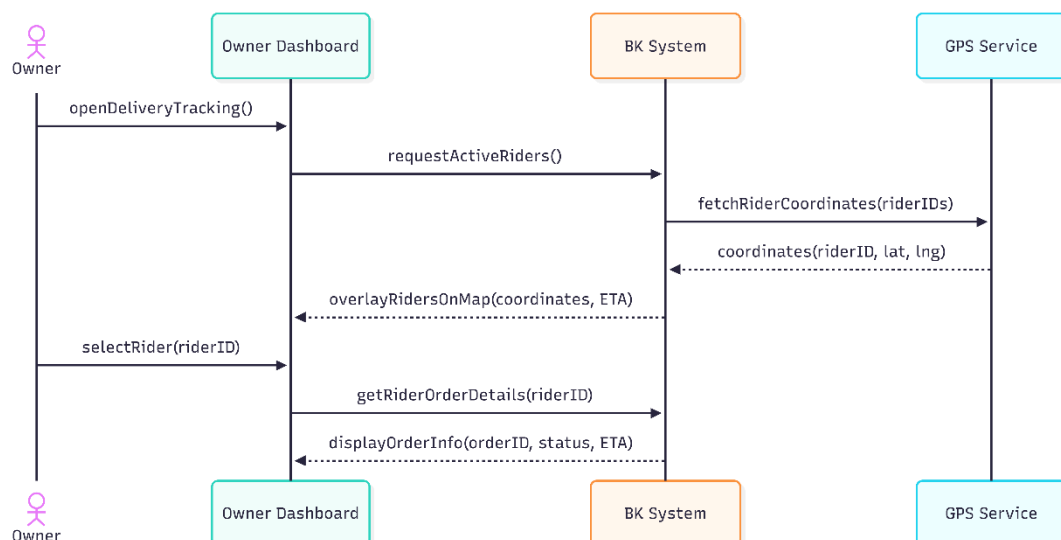
SSD-06 — Update Delivery Status via Mobile

This diagram illustrates the delivery completion process in the BK Restaurant Management System. After delivering an order, the rider updates its status to "Delivered" through the mobile app. The system verifies the rider's location, records the delivery completion time, and confirms the successful update, completing the delivery process.



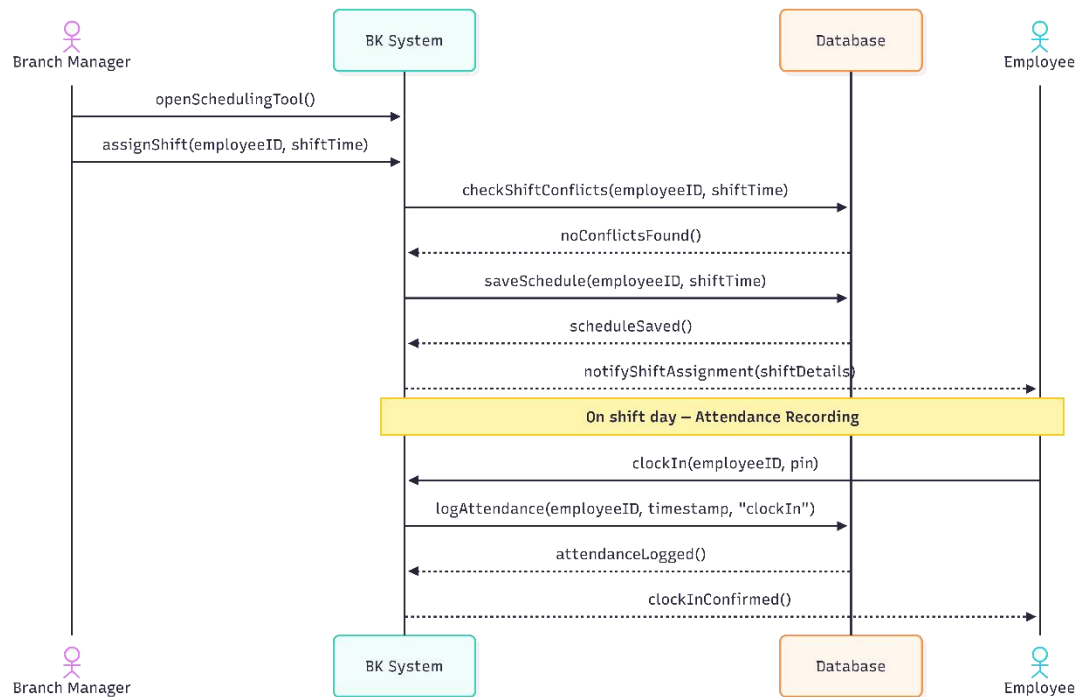
SSD-07 — Monitor Delivery Progress

This diagram shows the Owner's real-time delivery tracking dashboard. It pulls live GPS data to display rider locations on a map with ETAs, and allows the Owner to view detailed information for each rider and order, ensuring full visibility of all active deliveries.



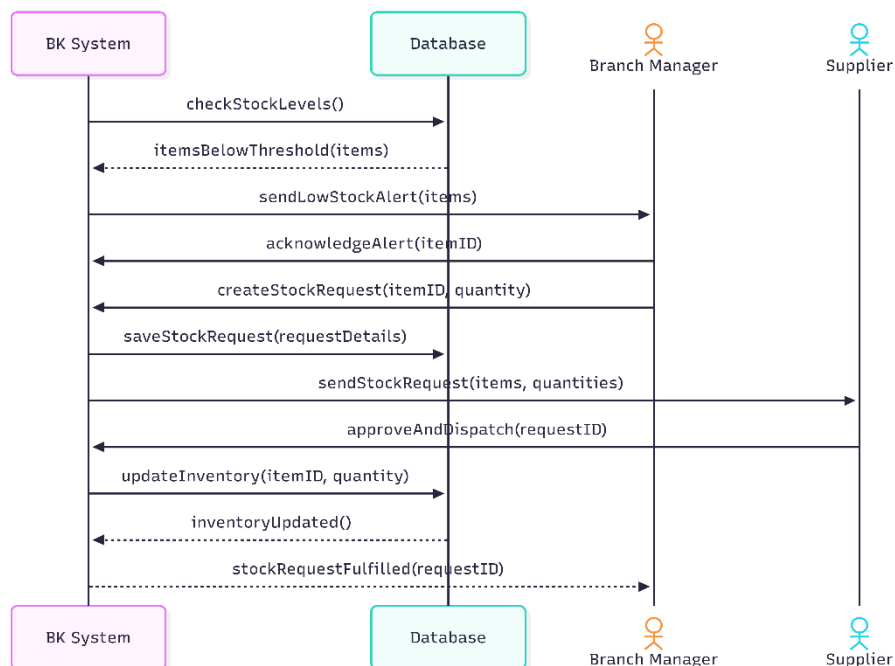
SSD-08 — Manage Employee Shifts and Record Attendance

This diagram illustrates shift scheduling and attendance tracking in the system. The Branch Manager assigns employee shifts, the system checks for conflicts, saves the schedule, and notifies employees.



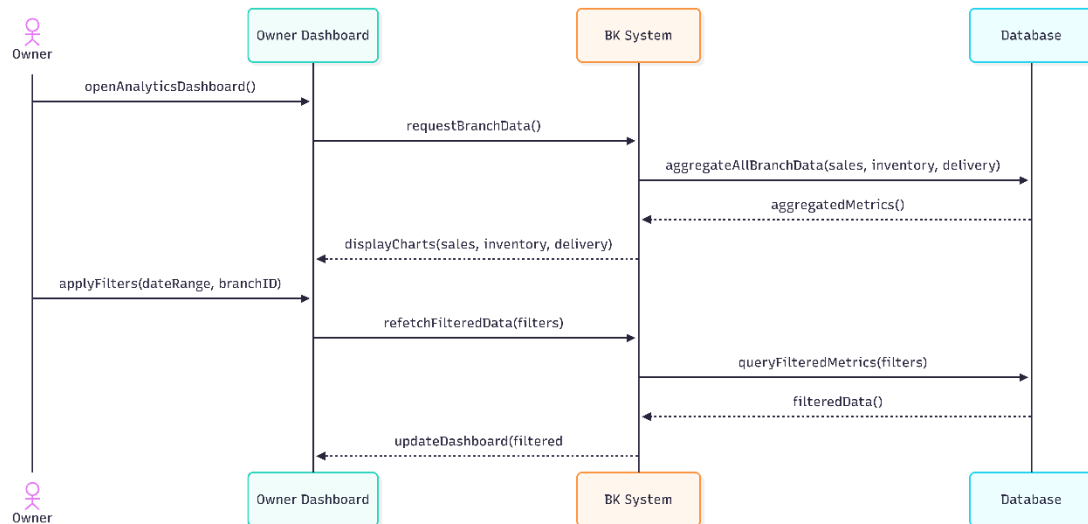
SSD-09 — Review Low-Stock Alerts & Process Stock Requests

This diagram illustrates the low-stock detection and replenishment process in the system. When inventory drops below set thresholds, the system alerts the Branch Manager, who creates a stock request that is sent to the Supplier. After approval and dispatch, inventory is updated and the Manager is notified once the request is fulfilled.



SSD-10— Monitor Centralized Branch Operations & Analytics

This diagram shows the Owner's analytics dashboard, which aggregates data from all branches, including sales, inventory, and delivery metrics. The system retrieves and displays this data as charts and tables, and allows filtering by date or branch to refresh the results for better decision-making.



4. Functional Requirements

BK Restaurant Management System — Functional Requirements			
ID	Requirement	Description	Priority
FR-01	Centralized Order Management	The system shall capture and store all online and walk-in customer orders in a centralized database, replacing the current WhatsApp and paper-based order management process.	High
FR-02	Digital Billing and Invoice Generation	The system shall automatically calculate order totals and generate digital bills and invoices through the Point of Sale (POS) system.	High
FR-03	Real-Time Kitchen Order Display	The system shall display customer orders on a digital kitchen screen in real time to reduce communication errors between reception staff and kitchen staff.	High
FR-04	Order Modification and Cancellation	The system shall allow authorized staff to modify or cancel customer orders before order completion and update all related system records accordingly.	High
FR-05	Delivery Assignment Management	The system shall allow restaurant staff to assign delivery orders to available riders through the system.	High
FR-06	Rider Mobile Application	The system shall provide a mobile application for delivery riders to receive delivery assignments and view customer location details.	High
FR-07	GPS-Based Delivery Tracking	The system shall provide real-time GPS tracking of delivery riders and display delivery progress on the owner dashboard.	High
FR-08	Delivery Status Updates	The system shall allow riders to update delivery statuses such as Assigned, Dispatched, In Progress, and Delivered.	High
FR-09	Inventory Management	The system shall automatically update inventory quantities after each completed sale.	High
FR-10	Low Stock Alert Generation	The system shall generate automatic alerts when stock levels fall below predefined minimum thresholds.	High
FR-11	Supplier Request Management	The system shall allow branch managers to create and send digital inventory requests to suppliers and maintain request records.	Medium

FR-12	Employee Attendance Management	The system shall record employee attendance digitally and maintain attendance history.	Medium
FR-13	Leave Request Management	The system shall allow employees to submit leave requests and allow managers to approve or reject requests through the system.	Medium
FR-14	Employee Schedule Management	The system shall allow managers to create, update, and monitor employee work schedules.	Medium
FR-15	Multi-Branch Monitoring Dashboard	The system shall provide a centralized dashboard that allows the restaurant owner to monitor all branches simultaneously.	High
FR-16	Sales and Revenue Reporting	The system shall generate daily, weekly, and monthly sales reports for each branch and for all branches combined.	High
FR-17	Role-Based Access Control	The system shall enforce role-based access control for Restaurant Owners, Branch Managers, Reception Staff, Kitchen Staff, and Delivery Riders.	High
FR-18	Secure User Authentication	The system shall require users to authenticate themselves before accessing system functions.	High

5. Non-Functional Requirements

BK Restaurant Management System — Non-Functional Requirements			
(i) Performance			
ID	Category	Description	Priority
NFR-01	Response Time	The system shall display POS order updates within 2 seconds.	High
NFR-02	Concurrent Access	The system shall support concurrent access from all restaurant branches without significant performance degradation.	High
NFR-03	Real-Time Processing	The system shall process inventory updates and delivery status updates in real time.	High
(ii) Security			
ID	Category	Description	Priority
NFR-04	User Authentication	The system shall require secure user authentication for all users.	High
NFR-05	Password Encryption	Passwords shall be stored using secure encryption techniques.	High
NFR-06	Role-Based Restriction	The system shall restrict access to features based on user roles.	High
NFR-07	Audit Logs	The system shall maintain audit logs of important user activities.	Medium
(iii) Usability			
ID	Category	Description	Priority
NFR-08	Simple UI	The user interface shall be simple and easy to use for non-technical staff.	High
NFR-09	Clear Navigation	The system shall provide clear navigation and user-friendly screens.	High
NFR-10	Rider App Usability	The rider mobile application shall be easy to learn and operate with minimal training.	High
(iv) Reliability			
ID	Category	Description	Priority
NFR-11	Offline Sync	The system shall prevent data loss during network failures through offline synchronization mechanisms.	High

NFR-12	Data Consistency	The system shall maintain accurate and consistent records across all branches.	High
NFR-13	Backup & Recovery	The system shall provide backup and recovery mechanisms for critical business data.	High
(v) Scalability			
ID	Category	Description	Priority
NFR-14	Branch Expansion	The system shall support the addition of new restaurant branches without major architectural changes.	Medium
NFR-15	Load Scalability	The system shall support increasing numbers of users, orders, and inventory records while maintaining acceptable performance.	Medium
(vi) Maintainability			
ID	Category	Description	Priority
NFR-16	Modular Architecture	The system shall use a modular architecture to simplify future enhancements and maintenance.	Medium
NFR-17	Coding Standards	The system shall follow standard coding practices and documentation guidelines.	Medium
NFR-18	Non-Disruptive Updates	The system shall allow software updates without significant disruption to restaurant operations.	Medium
(vii) Compatibility			
ID	Category	Description	Priority
NFR-19	Browser Support	The web application shall support modern web browsers including Google Chrome, Microsoft Edge, and Mozilla Firefox.	High
NFR-20	Android Support	The rider mobile application shall support Android smartphones with GPS capabilities.	High
(viii) Availability			
ID	Category	Description	Priority
NFR-21	Operating Hours	The system shall be available during restaurant operating hours with minimal downtime.	High
NFR-22	Continuous Operation	The system shall support continuous operation across all restaurant branches.	High

6. Alternative Solutions & Feasibility Study

6.1 Alternative Solutions Analysed

Alternative 1 – Manual / Semi-Digital System (Current Approach)

Aspect	Details
Description	Uses WhatsApp for orders, phone calls for delivery, and paper/spreadsheet-based records for billing, inventory, and employee management.
Advantages	No implementation cost, simple to use, no technical setup required.
Disadvantages	High chance of errors, missed orders, no real-time tracking, inefficient reporting, and poor scalability across multiple branches.
Cost	Very low initial cost but high long-term operational inefficiency cost.
Overall Assessment	Not suitable for scaling or improving efficiency.

Alternative 2 – Cloud-Based SaaS POS Solutions (e.g., Toast POS, Revel Systems)

Aspect	Details
Description	Utilize an existing cloud-based restaurant management and POS platform such as Toast POS or Revel Systems for order processing, billing, inventory management, and reporting.
Advantages	Quick deployment, vendor support, proven reliability, built-in reporting features, and reduced development effort.
Disadvantages	High recurring subscription costs, limited customization, dependency on third-party vendors, and some advanced features may require additional paid modules.
Cost	Ongoing monthly or annual subscription fees, plus potential costs for additional terminals and premium features.
Overall Assessment	Suitable for established businesses with larger budgets, but less cost-effective and flexible compared to a custom-built solution tailored to BK Restaurant's specific requirements.

Alternative 3 – Proposed Custom Web-Based System (BK Restaurant System)

Aspect	Details
Description	A custom-built system using React.js (frontend), Node.js (backend), and MySQL/PostgreSQL for database with modules for POS, inventory, orders, and reporting.
Advantages	Fully customized to business needs, supports multi-branch operations, reduces manual work, improves accuracy, and enables future expansion.
Disadvantages	Requires development time and maintenance responsibility.
Cost	One-time development cost & minimal hosting/maintenance cost.
Overall Assessment	Recommended solution as it best fits the business requirements and solves identified operational issues.

6.2 Feasibility Study

6.2.1 Technical Feasibility

All required technologies are mature, well-documented, and proven in production restaurant and delivery management systems:

Factor	Analysis
Frontend Technologies	React.js and Tailwind CSS are mature, widely adopted technologies suitable for developing responsive web interfaces and POS dashboards.
Backend Technologies	Node.js and Express provide a scalable and efficient platform for handling orders, inventory, reporting, and API services.
Database Technologies	PostgreSQL offers reliable relational data management, while Redis can be used for caching and performance optimization.
Integration Support	The system can support integrations such as thermal receipt printers and Google Maps services for delivery tracking and navigation.
Conclusion	The proposed system is technically feasible as the required technologies are well-established and suitable for restaurant management operations.

6.2.2 Economic Feasibility

Factor	Analysis
Development Cost	The system will be developed as part of the project using existing team resources, reducing software acquisition costs.
Software Cost	Most development tools and frameworks are open-source and free to use.

Factor	Analysis
Hosting Cost	Cost-effective hosting options such as AWS Free Tier or Railway.app can be used during deployment.
Long-Term Benefits	The system reduces manual work, minimizes operational errors, and eliminates recurring subscription fees associated with commercial SaaS solutions.
Conclusion	The project is economically feasible due to its low implementation cost and potential operational savings.

6.2.3 Operational Feasibility

Factor	Analysis
User Readiness	Restaurant staff and management possess basic computer and smartphone literacy required to operate the system.
Ease of Use	The system is designed with a simple and intuitive user interface to minimize training requirements.
Business Compatibility	The proposed solution aligns with existing restaurant workflows while improving efficiency through automation.
User Acceptance	The client has expressed interest in a centralized solution to address current operational challenges.
Conclusion	The system is operationally feasible and can be adopted with minimal disruption to current business processes.

The system is designed for low-technical-proficiency users across all three branches:

- **POS (Reception/Kitchen):** Large touch targets, visual KDS display, minimal typing. New staff require no more than 30 minutes of training.
- **Rider Mobile App:** Step-by-step guided delivery flow, integrated navigation, simple status update buttons. Designed for riders with basic smartphone literacy.
- **Owner Dashboard:** Standard web conventions. Single-page analytics with pre-built charts requiring no manual data entry.
- **Training Plan:** On-site training sessions planned at the Ambalangoda branch for staff rollout before live deployment. BK branches will serve as UAT testing locations.

6.2.4 Schedule Feasibility

Phase	Duration	Timeline	Key Deliverables
Phase 1	1 week	Weeks 1–4	Idea discovery, problem identification, initial research
Phase 2	1 week	Weeks 1-2	System design, architecture diagrams, database schema, UI wireframes
Phase 3	1 week	Weeks 2	Project kick-off, team role allocation, repository setup
Phase 4	2 weeks	Weeks 3-4	Requirements engineering, stakeholder interviews, competitor analysis, literature review
Phase 5	1 weeks	Weeks 5	SRS preparation, requirements validation, SRS finalization
Phase 6	2 weeks	Weeks 5-7	System design, architecture design, database schema, use case diagrams, wireframes
Phase 7	1 week	Weeks 7–8	Design review, documentation consolidation, final submission preparation
Phase 8	4 Months	Month 2- 6	System development, implementation, testing, UI refinement, deployment preparation
Total	Approximately 6 Months	Approximately 2 Semesters	Complete restaurant management system delivery

Conclusion: The project schedule is feasible as the work is divided into manageable phases aligned with the academic timeline. The sprint-based approach allows requirements analysis, design, and development activities to be completed within the allocated project duration.

6.3 Justified Recommendation

The **custom web-based restaurant management system (BK Restaurant System)** is the most suitable solution because it directly addresses the operational problems of the current system while remaining cost-effective and scalable.

Unlike manual processes, it eliminates errors and improves efficiency. Compared to SaaS solutions, it avoids high recurring costs and provides full customization for local restaurant workflows, including multi-branch operations.

Therefore, the proposed system is recommended as it offers the best balance between cost, usability, and functionality for the BK Restaurant environment.

BK Restaurant Management System

Software Design Specification

Course	SENG 31242 – System Design Project
Group	01
Team Members	SE/2022/020 – Sachin Lakshitha SE/2022/023 – Duvini Nimethra SE/2022/026 – Dinuja Ranaweera SE/2022/030 – Nethmi Navodya
Client	Mr. N. C. Sanjeewa De Silva, BK Restaurant & BK Bar
Related Document	Software Requirements Specification (SRS) – BK Restaurant Management System
Document Status	Draft

Contents

1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 References	4
1.4 Overview of Document	4
2. System Overview	6
2.1 Summary of the System to be Built	6
2.2 Relationship to the SRS	6
3. Architectural Design	7
3.1 Architecture Pattern Selected	7
3.2 Justification (Quality Attributes)	7
3.3 Component / Package Diagram	8
3.4 Deployment Diagram	8
3.5 Technology Stack with Justification	8
4. Detailed Design	10
4.1 Class Diagram	10
4.2 Sequence Diagrams	10
4.3 State Machine Diagrams	10
4.4 Database / Data Model (Entity-Relationship Diagram)	10
5. UI Design	11
5.1 Wireframes for All Major Screens	11
5.1.1 Employee Related Screens	11
5.1.2 Manager Related Screens	15
5.1.3 Inventory Related Screens	20
5.1.4 Common Screens	23
5.2 Navigation Flow Diagram	26
5.2.1 User Login & Role-Based Access Flow	26
5.2.2 Customer Order Creation & Billing Flow	27
5.2.3 Kitchen Order Processing Flow	28
5.2.4 Delivery Rider Flow	29
5.2.5 Inventory Management & Supplier Request Flow	30
5.2.6 Owner Monitoring & Analytics Flow	31
5.3 UX Decisions and Rationale	32
6. Interface Design	33
6.1 External Interfaces (APIs, Hardware, Third-Party Services)	33
6.2 Internal Module Interfaces	34
7. Security Design	35

7.1 Authentication and Authorisation Strategy.....	35
7.2 Data Protection (At Rest, In Transit)	35
7.3 Input Validation Strategy	35
7.4 OWASP Top 10 Considerations	36
8. Non-Functional Design Decisions.....	38

1. Introduction

1.1 Purpose

The purpose of this Software Design Specification (SDS) is to define the technical design of the BK Restaurant Management System in sufficient detail to guide its implementation, testing, and future maintenance. This document translates the requirements captured in the Software Requirements Specification (SRS) into a concrete architecture, component structure, data model, interface design, and security design. It is intended for use by the development team during implementation, by reviewers assessing design quality against the requirements, and by future maintainers extending the system.

1.2 Scope

This document covers the design of all major subsystems identified in the SRS, namely:

- The Point of Sale (POS) and Kitchen Display System (KDS) used by reception and kitchen staff
- The centralised backend API and database serving all branches
- The Flutter-based mobile application used by delivery riders
- The centralised web dashboard used by the Restaurant Owner for multi-branch monitoring and analytics
- The inventory management, supplier request, and employee management modules

The design described here directly addresses the Functional Requirements (FR-01 to FR-18) and Non-Functional Requirements (NFR-01 to NFR-22) defined in the SRS. Implementation-level detail such as exact source code, configuration files, and deployment scripts is outside the scope of this document and will be produced during the development phase.

1.3 References

- IEEE Std 1016–1998 – IEEE Recommended Practice for Software Design Descriptions.
- ISO/IEC/IEEE 42010:2011 – Systems and Software Engineering – Architecture Description.
- OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks.
<https://owasp.org/Top10/>
- React.js Documentation. <https://react.dev>
- Node.js and Express.js Documentation. <https://nodejs.org>, <https://expressjs.com>
- PostgreSQL Documentation. <https://www.postgresql.org/docs/>
- Flutter Documentation. <https://flutter.dev>

1.4 Overview of Document

This document follows the SDS structure agreed for SENG 31242:

- Section 2 – System Overview: a summary of the system to be built and its relationship to the SRS.
- Section 3 – Architectural Design: the selected architecture pattern, justification, component/package diagram, deployment diagram, and technology stack.
- Section 4 – Detailed Design: class diagram, sequence diagrams (one per SRS use case), state machine diagrams for stateful entities, and the database/ER diagram.

- Section 5 – UI Design: wireframes for all major screens, the navigation flow diagram, and UX decisions and rationale.
- Section 6 – Interface Design: external interfaces (APIs, hardware, third-party services) and internal module interfaces.
- Section 7 – Security Design: authentication and authorisation strategy, data protection, input validation, and OWASP Top 10 considerations.
- Section 8 – Non-Functional Design Decisions: how each NFR from the SRS is addressed by the design.

2. System Overview

2.1 Summary of the System to be Built

The BK Restaurant Management System is a custom, web- and mobile-based platform that digitises and centralises operations for BK Restaurant and BK Bar across its branches in Ambalangoda and Kahawa, Sri Lanka. The system replaces the existing manual, WhatsApp- and paper-based workflow with an integrated set of modules:

- A web-based POS system for counter order entry, billing, and invoice generation.
- A Kitchen Display System (KDS) that shows incoming orders to kitchen staff in real time.
- A Flutter mobile application for delivery riders, providing assignment notifications, navigation, live GPS tracking, and delivery status updates.
- An inventory management module with automatic stock deduction, low-stock alerts, and digital supplier requests.
- An employee management module covering attendance, shift scheduling, and leave requests.
- A centralised owner dashboard for multi-branch monitoring, sales/revenue reporting, and delivery tracking analytics.

The system is built around a single backend API and a shared PostgreSQL database, with role-based access control ensuring that Restaurant Owners, Branch Managers, Reception/Kitchen Staff, and Delivery Riders each see only the features relevant to their role.

2.2 Relationship to the SRS

This SDS implements the requirements defined in the SRS as follows:

SRS Reference	How it is Addressed in this Design
FR-01, FR-02, FR-03, FR-04 (Order Management, Billing, KDS, Modification)	Implemented by the POS/KDS module, the Order and Invoice entities in the data model, and SSD-02/SSD-03 sequence diagrams (Section 4.2).
FR-05, FR-06, FR-07, FR-08 (Delivery Assignment and Tracking)	Implemented by the Delivery module, the rider mobile app, the GPS integration described in Section 6.1, and SSD-05/SSD-06/SSD-07 sequence diagrams.
FR-09, FR-10, FR-11 (Inventory and Supplier Requests)	Implemented by the Inventory module, the Inventory Item and Stock Request entities, and the SSD-09 sequence diagram.
FR-12, FR-13, FR-14 (Employee Attendance, Leave, Scheduling)	Implemented by the Employee Management module and the SSD-08 sequence diagram.
FR-15, FR-16 (Multi-Branch Dashboard and Reporting)	Implemented by the Owner Dashboard module and the SSD-10 sequence diagram.
FR-17, FR-18 (Role-Based Access Control, Secure Authentication)	Implemented by the security design in Section 7 and the SSD-01 sequence diagram.
NFR-01 to NFR-22 (Performance, Security, Usability, Reliability, Scalability, Maintainability, Compatibility, Availability)	Addressed individually in Section 8, with traceability to the corresponding design decision.

3. Architectural Design

3.1 Architecture Pattern Selected

The BK Restaurant Management System adopts a layered (n-tier) client–server architecture combined with a modular, feature-based organisation of the backend codebase. The system is structured into the following layers:

- **Presentation Layer** – the React.js web application (POS, KDS, and Owner Dashboard) and the Flutter mobile application (Rider App).
- **Application / API Layer** – a Node.js and Express.js REST API that exposes endpoints for each functional module (orders, inventory, delivery, employees, reporting, authentication).
- **Business Logic Layer** – service modules within the backend that implement validation rules, status transitions, and calculations (e.g., bill totals, inventory deductions, low-stock thresholds).
- **Data Access Layer** – an ORM-based data access layer that maps domain entities to the PostgreSQL database.
- **Data Layer** – a single PostgreSQL database shared across all branches, with Redis used as a caching layer for frequently accessed data such as menu items and dashboard summaries.

Within the Application Layer, the backend is further divided into independent modules (Authentication, Orders, Inventory, Delivery, Employees, Reporting), each exposing its own set of routes, controllers, and services. This gives the system the maintainability benefits of a modular/feature-based design while keeping the overall deployment simple enough for a small team to manage, which is appropriate given the project's scale and timeline.

3.2 Justification (Quality Attributes)

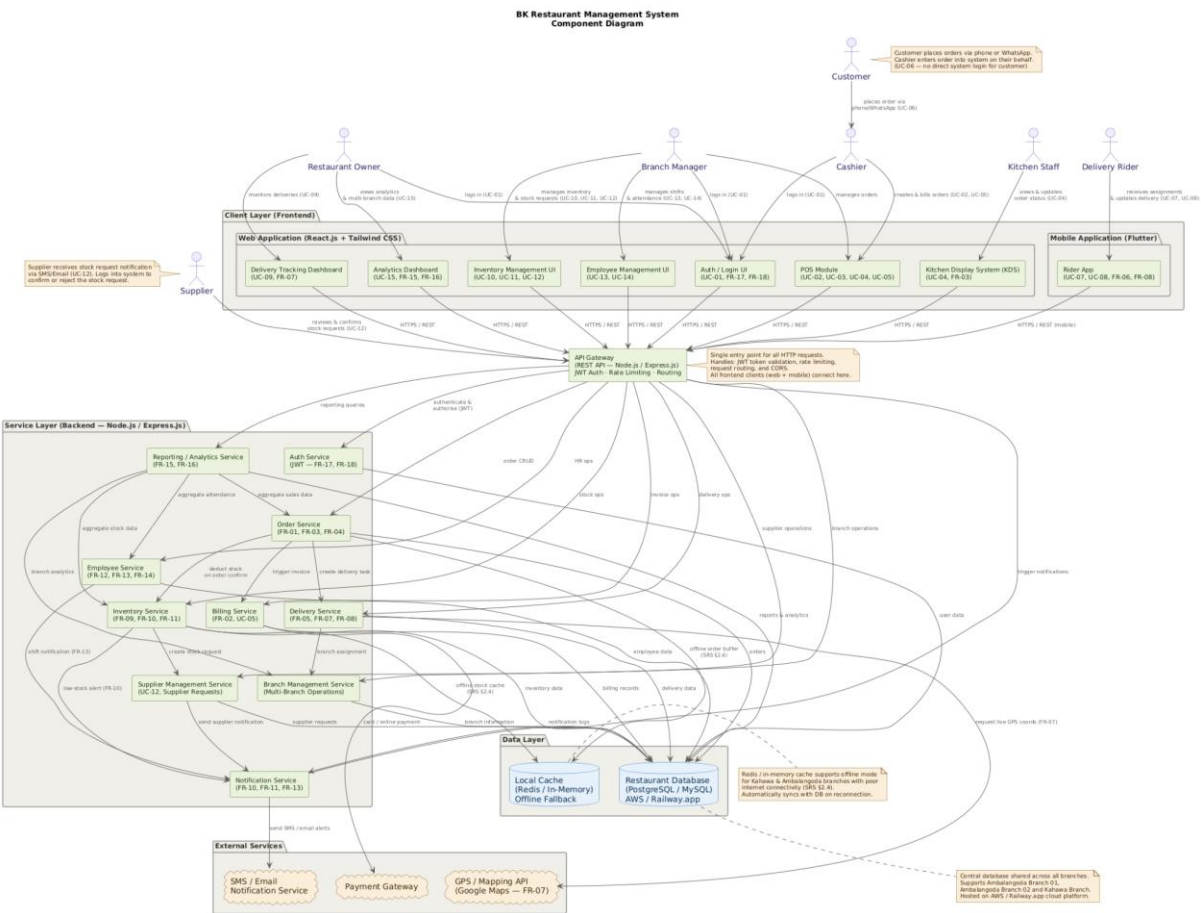
Quality Attribute	How the Architecture Supports It
Maintainability	Each functional module (orders, inventory, delivery, employees, reporting, authentication) is implemented as a separate package with its own routes, controllers, and services, allowing developers to modify one module without affecting others. This directly supports NFR-16 and NFR-17.
Scalability	The stateless API layer can be horizontally scaled behind a load balancer, and Redis caching reduces database load for read-heavy dashboard queries. New branches can be onboarded by adding records rather than changing the architecture, supporting NFR-14 and NFR-15.
Reliability	A layered design isolates business logic from data access, making it easier to add transaction handling, retries, and offline synchronisation for POS terminals and the rider app, supporting NFR-11, NFR-12, and NFR-13.
Performance	The API layer is kept thin and stateless, with caching for frequently read data (menu, dashboard summaries) to meet the 2-second response target in NFR-01 and to support concurrent multi-branch access (NFR-02).

Security	Centralising authentication and authorisation in a single module within the Application Layer (Section 7) ensures consistent enforcement of role-based access control across web and mobile clients, supporting NFR-04 to NFR-07.
Cost-Effectiveness	The chosen stack (React, Node.js/Express, PostgreSQL, Flutter) is open-source and well supported on low-cost cloud platforms, aligning with the design and implementation constraints identified in the SRS (Section 2.4).

3.3 Component / Package Diagram

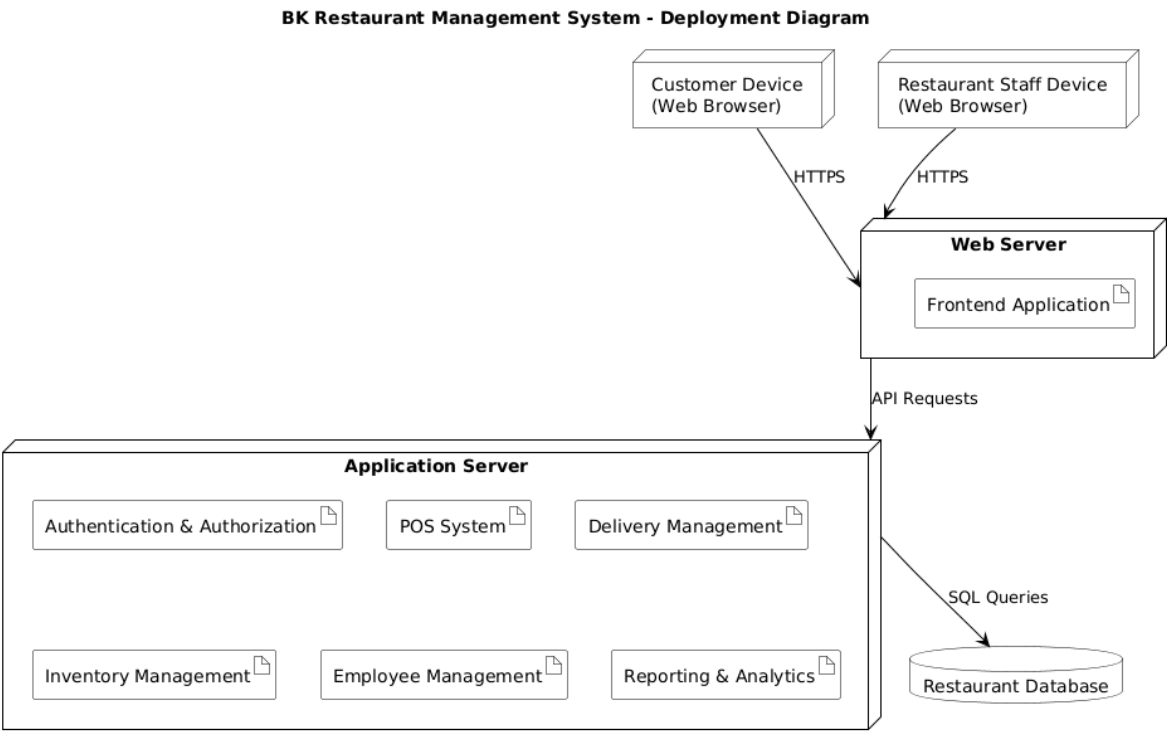
The Component Diagram illustrates the high-level architecture of the BK Restaurant Management System. It identifies the major system components, including the frontend applications, API Gateway, backend services, database, cache, and external services. The diagram shows how these components interact to support restaurant operations such as order processing, inventory management, delivery tracking, employee management, reporting, and authentication.

Figure 3.3.1: BK Restaurant Management System Component Diagram



3.4 Deployment Diagram

The deployment diagram below shows how the software components are mapped onto physical and virtual infrastructure nodes across the three restaurant branches and the cloud hosting environment.



3.5 Technology Stack with Justification

Layer / Component	Technology	Justification
Web Frontend (POS, KDS, Owner Dashboard)	React.js with Tailwind CSS	Mature, component-based library suited to building responsive POS and dashboard interfaces; Tailwind CSS speeds up consistent styling across screens (per SRS Section 2.3).
Mobile Frontend (Rider App)	Flutter	Cross-platform framework allowing a single codebase to target Android devices used by delivery riders, with good support for maps and GPS APIs.
Backend / API	Node.js with Express.js	Lightweight, widely adopted framework for building REST APIs; non-blocking I/O suits

		the real-time order and delivery update requirements (NFR-01, NFR-03).
Database	PostgreSQL	Reliable relational database suitable for transactional data such as orders, invoices, inventory, and employee records, with strong support for referential integrity across branches.
Caching	Redis	In-memory data store used to cache menu data and dashboard summaries, reducing database load and improving response times (NFR-01, NFR-02).
Authentication	JSON Web Tokens (JWT)	Stateless token-based authentication suitable for both the web and mobile clients, enabling role-based access control (FR-17, FR-18, NFR-04 to NFR-06).
Maps / GPS	Google Maps Platform (Maps SDK and Directions API)	Provides live location tracking and navigation for delivery riders and the owner's delivery monitoring dashboard (FR-07, UC-09).
Hosting	AWS Free Tier or Railway.app	Cost-effective cloud hosting options identified as economically feasible in the SRS feasibility study (Section 6.2.2).
Notifications	Email/SMS gateway (e.g., SendGrid, Twilio)	Used for sending invoices to customers and notifying suppliers of stock requests (FR-05, UC-12).

4. Detailed Design

4.1 Class Diagram

The class diagram below shows the core domain classes of the BK Restaurant Management System, their attributes, key methods, and the relationships between them. It forms the basis for the database schema described in Section 4.4.

4.2 Sequence Diagrams

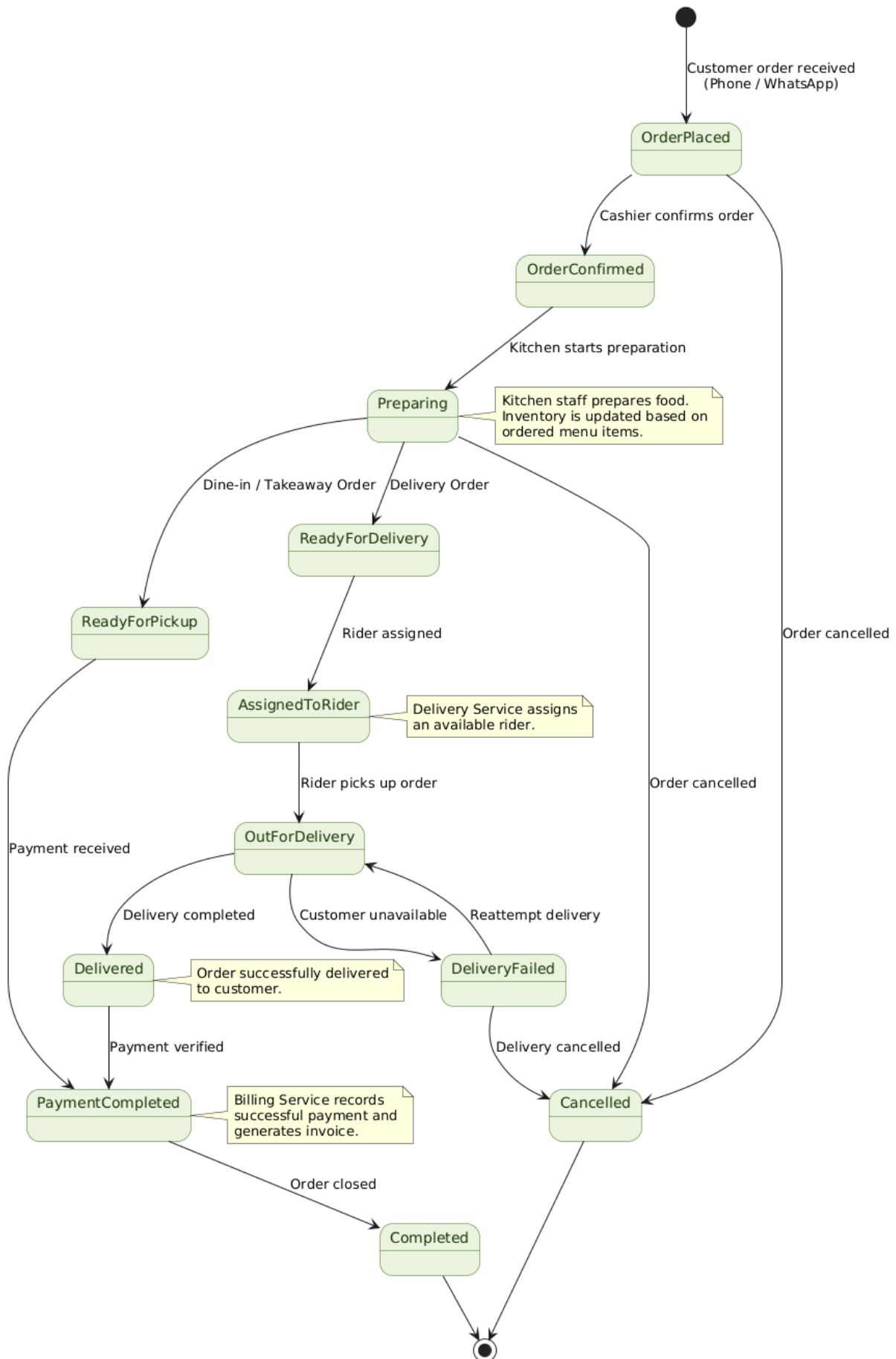
This sub-section provides one sequence diagram for each use case defined in the SRS (Section 3.4), showing the detailed message exchange between the actor, the client application, the API, and the database/services. Each diagram corresponds to a System Sequence Diagram (SSD) from the SRS, expanded to show internal system interactions.

4.3 State Machine Diagrams

The State Machine Diagram represents the lifecycle of an order within the BK Restaurant Management System. It shows the valid state transitions from order creation through preparation, delivery, completion, cancellation, and refund processing. The diagram ensures that order status changes follow the defined business rules and prevents invalid transitions between states.

Figure 4.3.1: BK Restaurant Management System Order State Machine Diagram

BK Restaurant Management System Order State Machine Diagram



4.4 Database / Data Model (Entity-Relationship Diagram)

The entity-relationship diagram below defines the relational database schema implementing the class diagram in Section 4.1. All entities are stored in a single shared PostgreSQL database, with a `branchId` foreign key used to partition data by branch where applicable.

Each row above maps directly to a table in the implementation database, with primary keys (PK) as auto-incrementing integers or UUIDs and foreign keys (FK) enforcing referential integrity. Indexes should be added on frequently queried columns such as `Order.branch_id`, `Order.status`, `InventoryItem.branch_id`, and `Delivery.status` to support the performance requirements in NFR-01 and NFR-02.

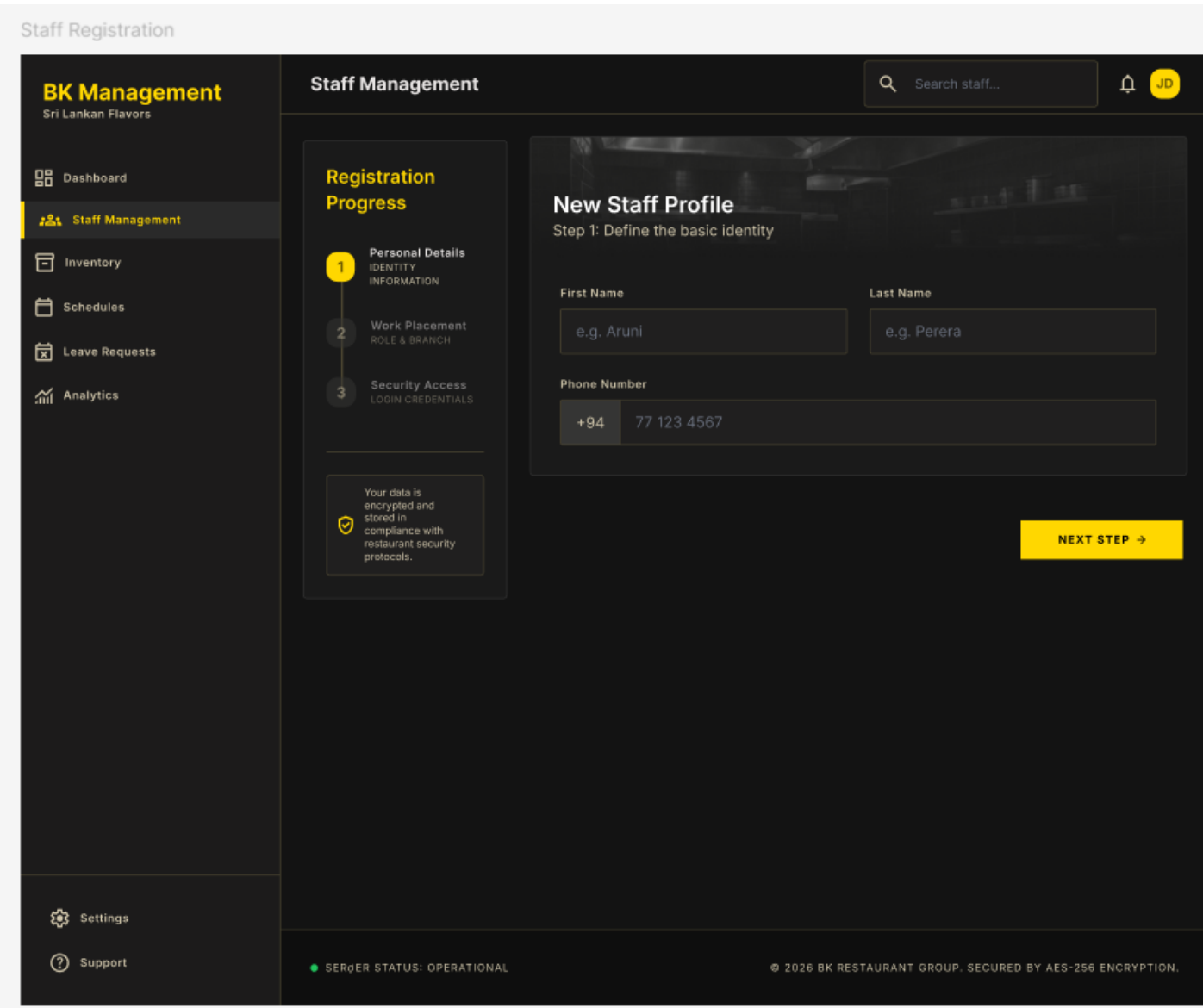
5. UI Design

5.1 Wireframes for All Major Screens

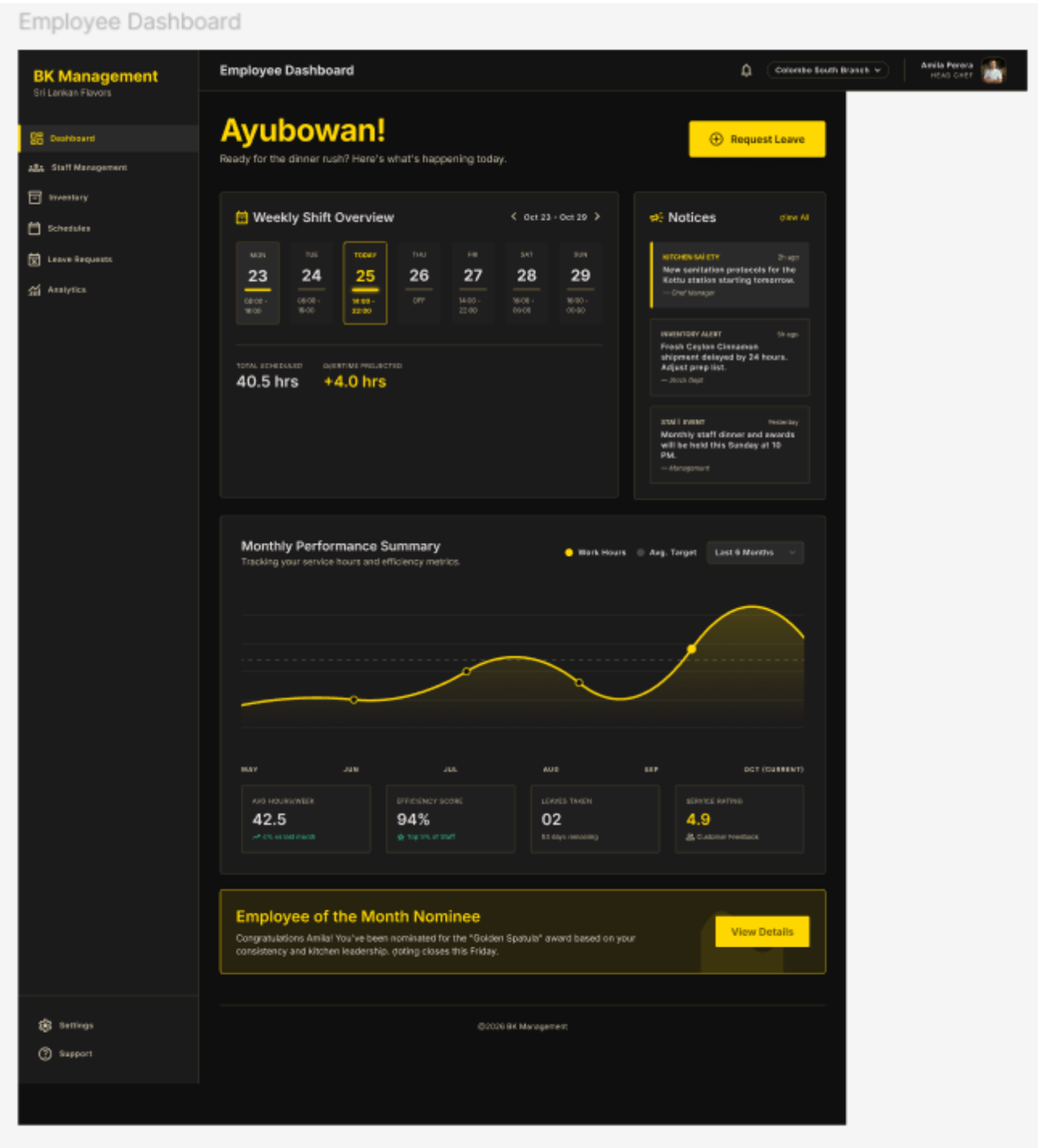
Wireframes are provided for each major screen of the web (POS, KDS, Owner Dashboard) and mobile (Rider App) applications. Each wireframe should reflect the simple, low-training-overhead design described in the SRS feasibility study (Section 6.2.3).

5.1.1 Employee Related Screens

5.1.1.1 Staff Registration



5.1.1.2 Employee Dashboard



5.1.1.3 Employee Schedule

Employee Schedule

BK Management
Sri Lankan Flavors

Dashboard

Staff Management

Inventory

Schedules

Leave Requests

Analytics

Schedules

Colombo Fort Branch

Search staff or shifts...

Logout

Shift Timetable

August 2024 • Week 34

WeeklyMonthlyTeam View

< Today >

ACTIVE NOW

Morning Shift

06:00 AM — 02:00 PM

On duty with you

+2.5h this week

42 Hours

Scheduled Total (Week 34)

Calculated base

LKR 28,400

Estimated Weekly Earnings

MON 19	TUE 20	WED 21 (TODAY)	THU 22	FRI 23	SAT 24	SUN 25
MORNING 06:00 - 14:00 Walk Kitchen	EVENING 14:00 - 22:00 Floor Staff	ACTIVE NOW 06:00 - 14:00 Pastry Section	MORNING 06:00 - 14:00 Walk Kitchen EVENING 14:00 - 22:00 Dishier Desk NIGHT 22:00 - 06:00 Inventory Check	MORNING 06:00 - 14:00 Walk Kitchen EVENING 14:00 - 22:00 Dishier Desk	REQUESTED OFF	STORE CLOSED
		Overbook Full				

Team on Duty - Today

Arun Mendis

Sous Chef

Rinal Pereira

Head Server

Burhan Kuttars

Evening Shift

Priya Sene

Night Shift

New Entry

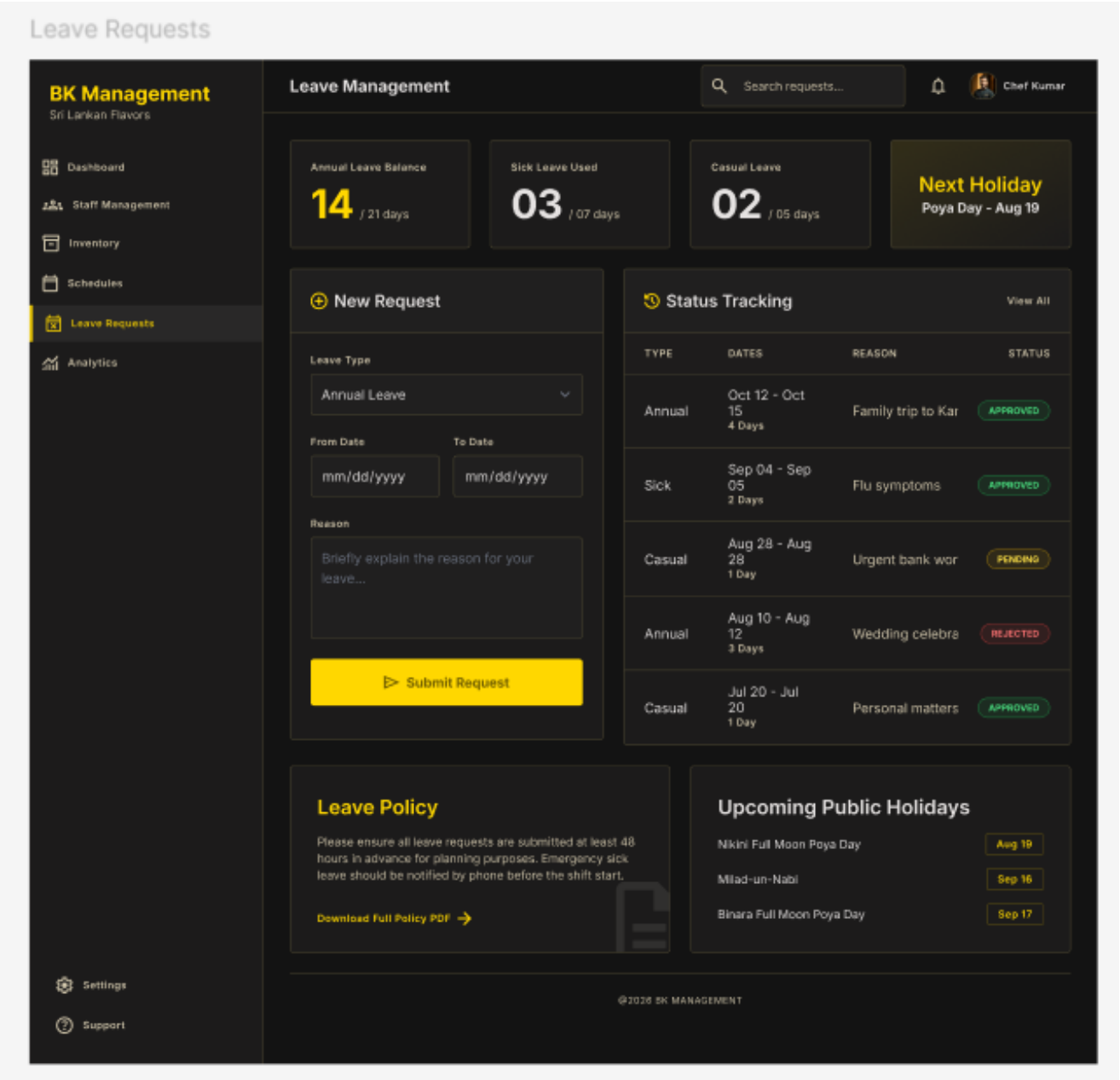
Settings

Support

Embracing the Hustle

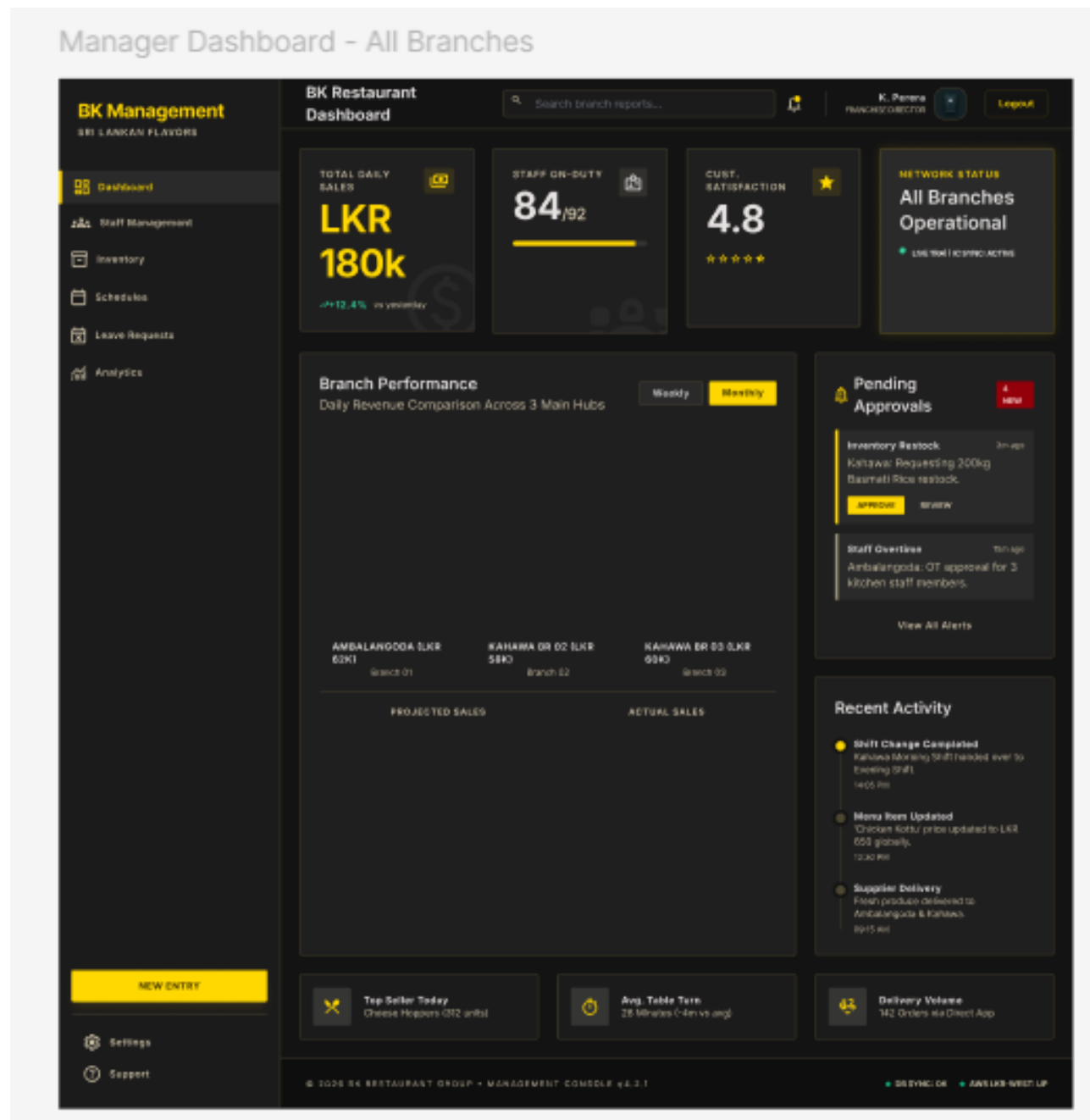
"Success is served hot. Manage your time like a master chef manages his kitchen - with precision and passion."

5.1.1.4 Leave Requests

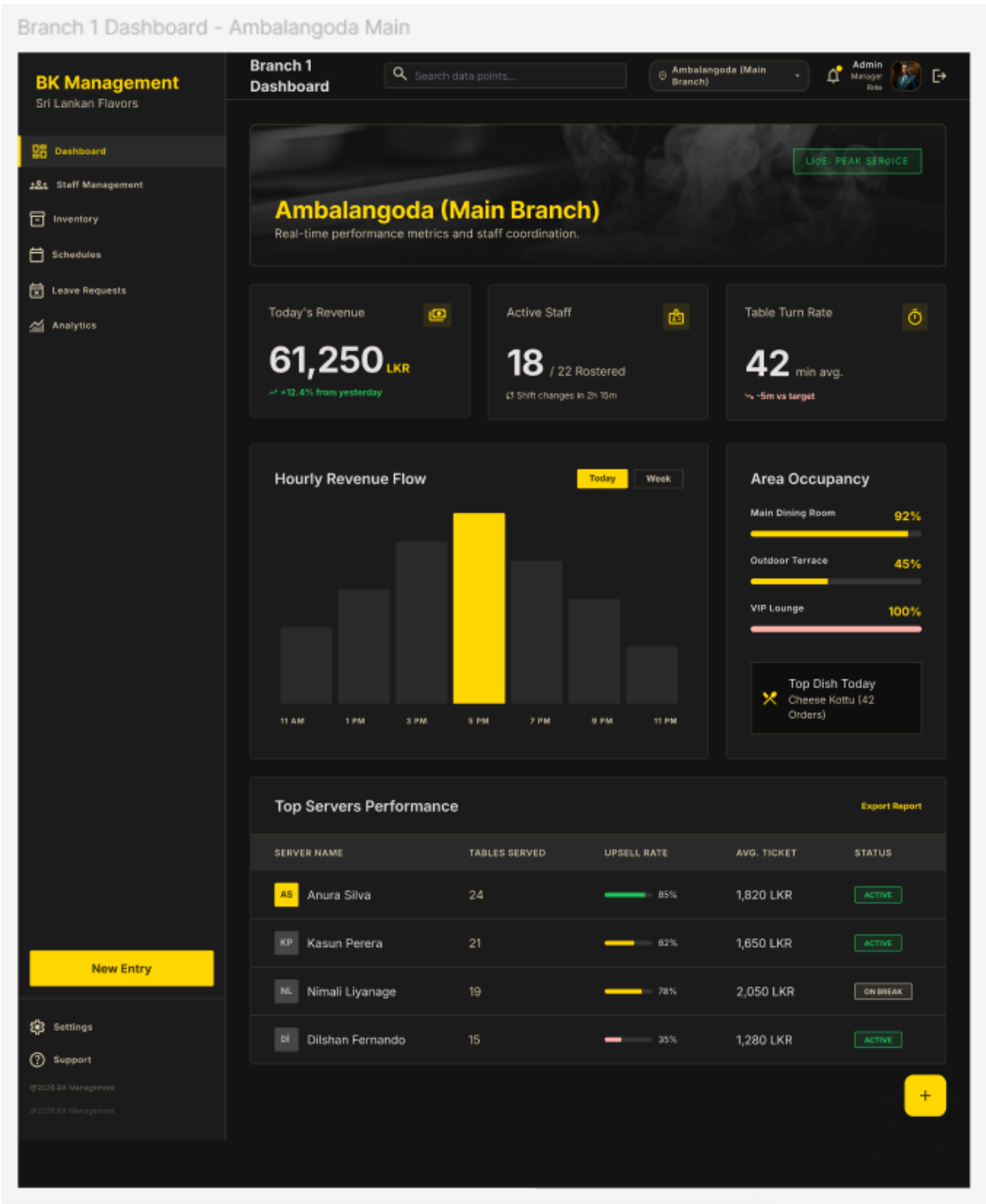


5.1.2 Manager Related Screens

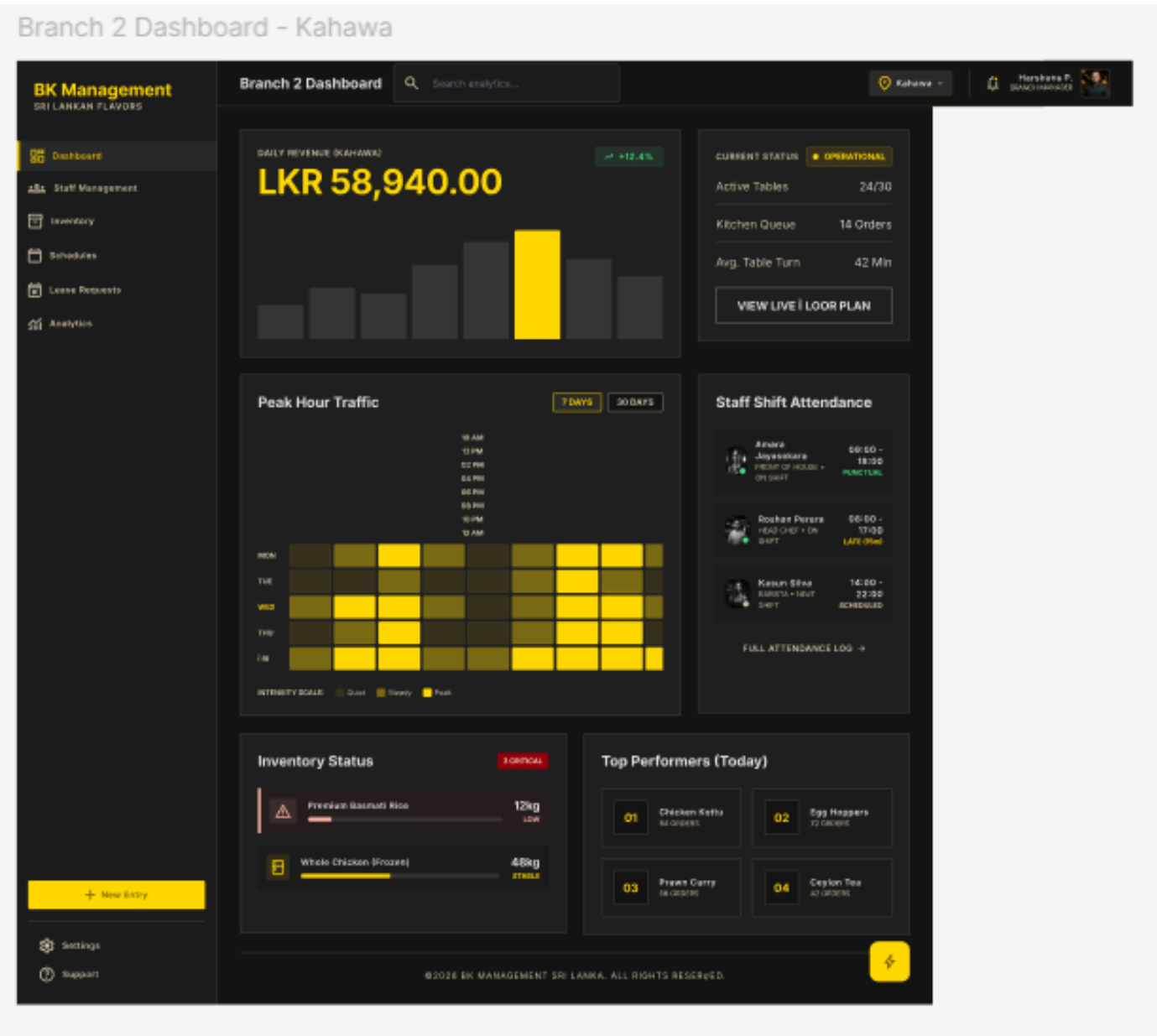
5.1.2.1 Manager Dashboard - All Branches



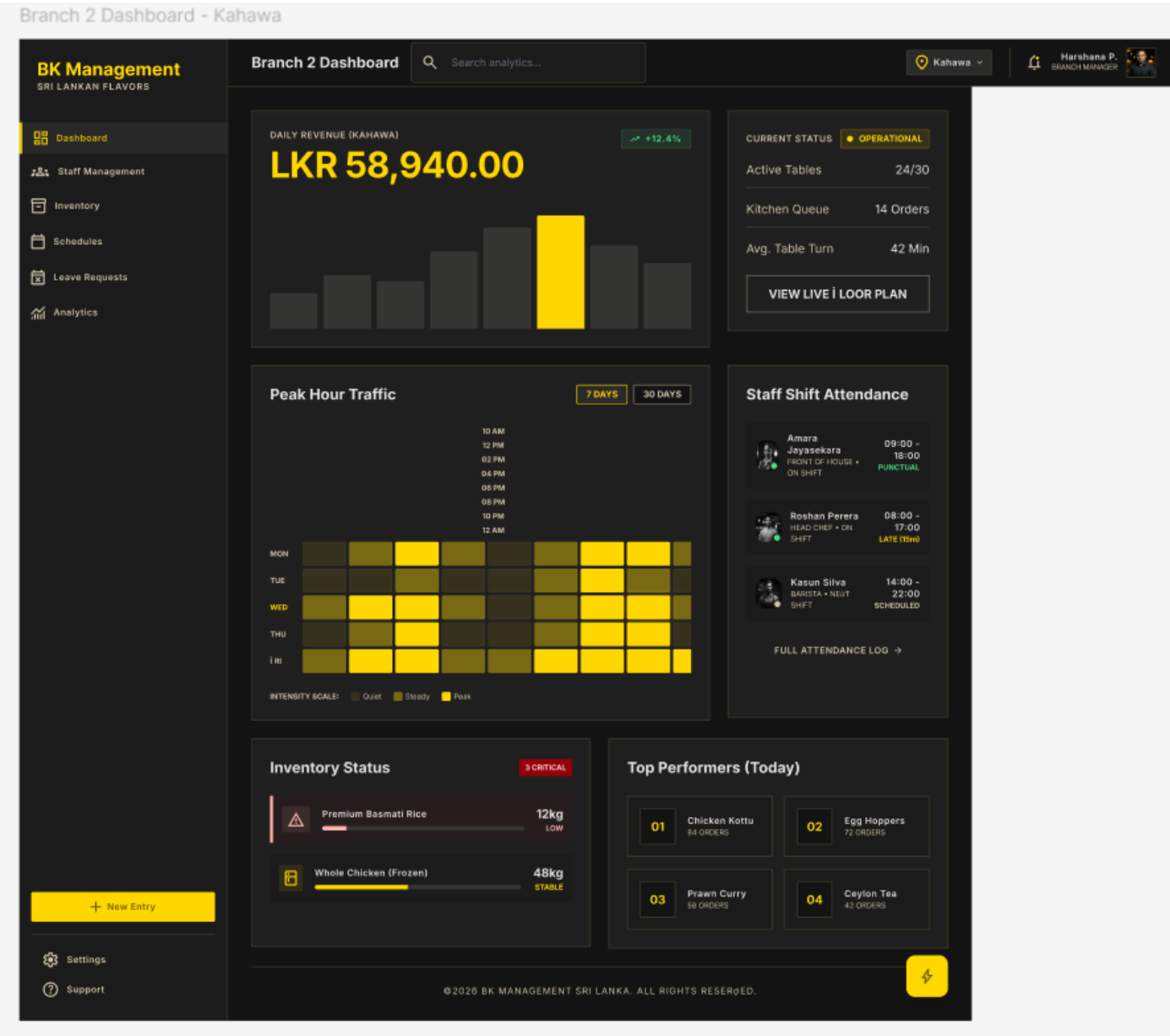
5.1.2.2 Branch 1 Dashboard - Ambalangoda Main



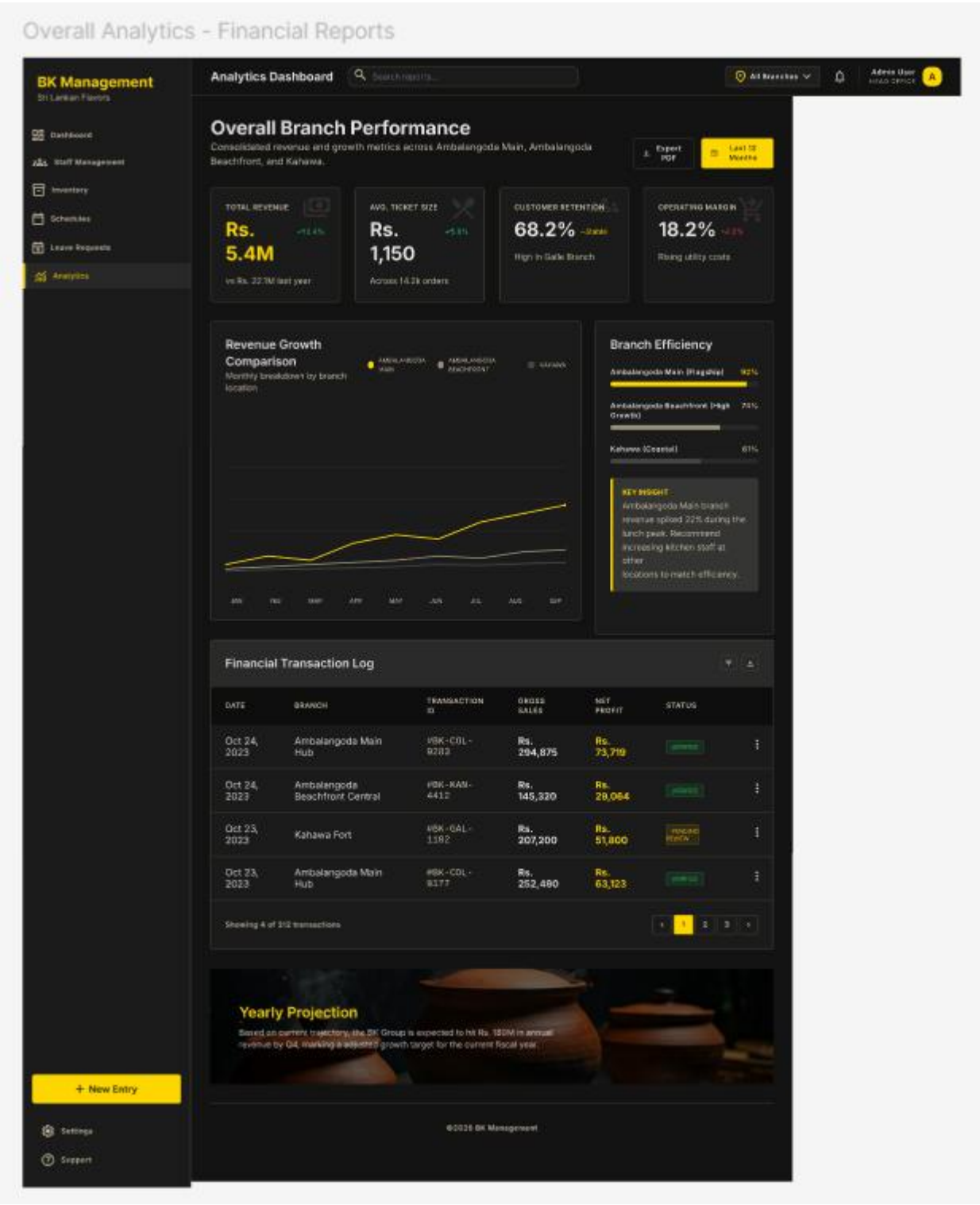
5.1.2.3 Branch 2 Dashboard – Kahawa



5.1.2.4 Branch 3 Dashboard – Kahawa

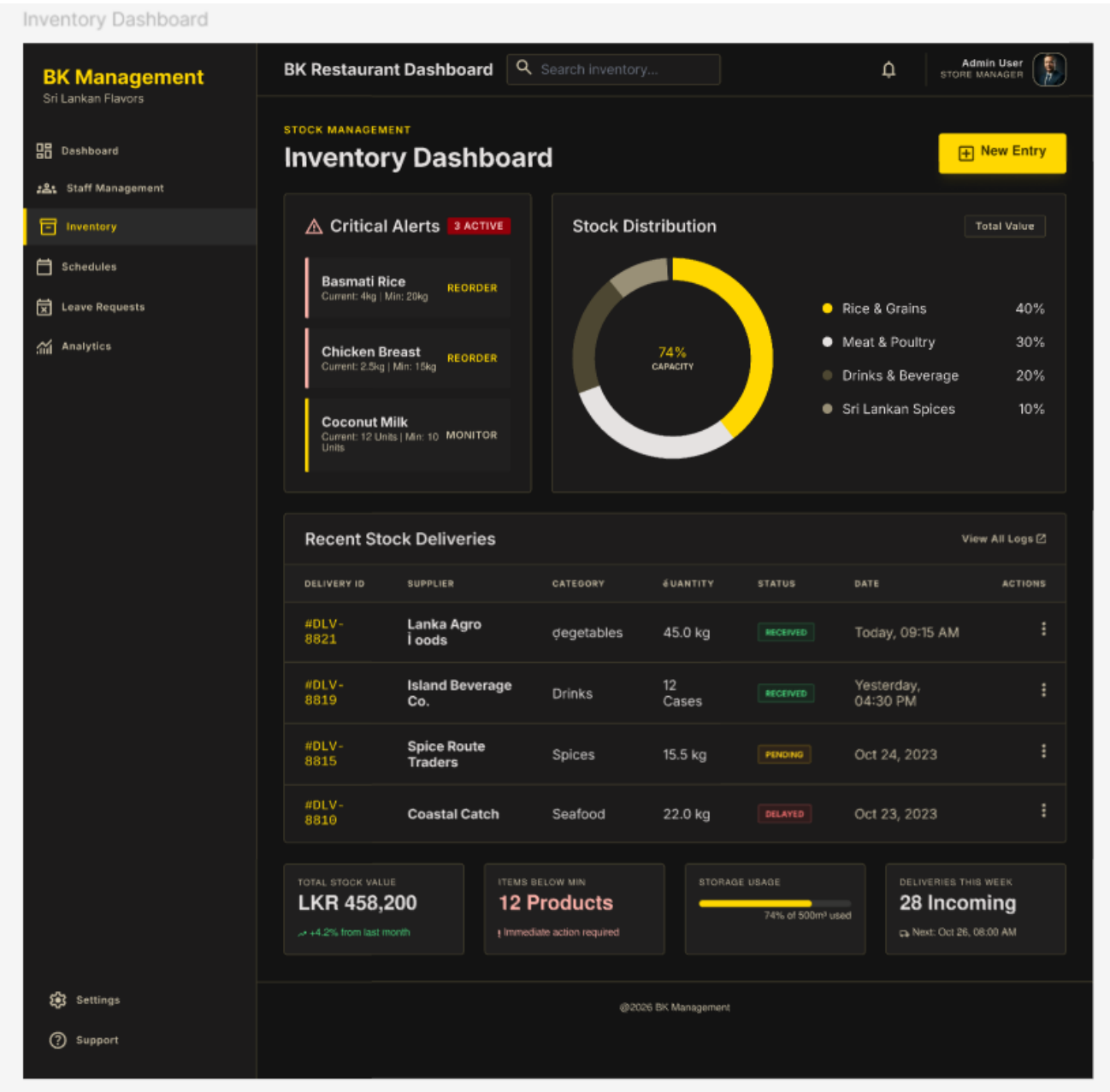


5.1.2.5 Overall Analytics - Financial Reports

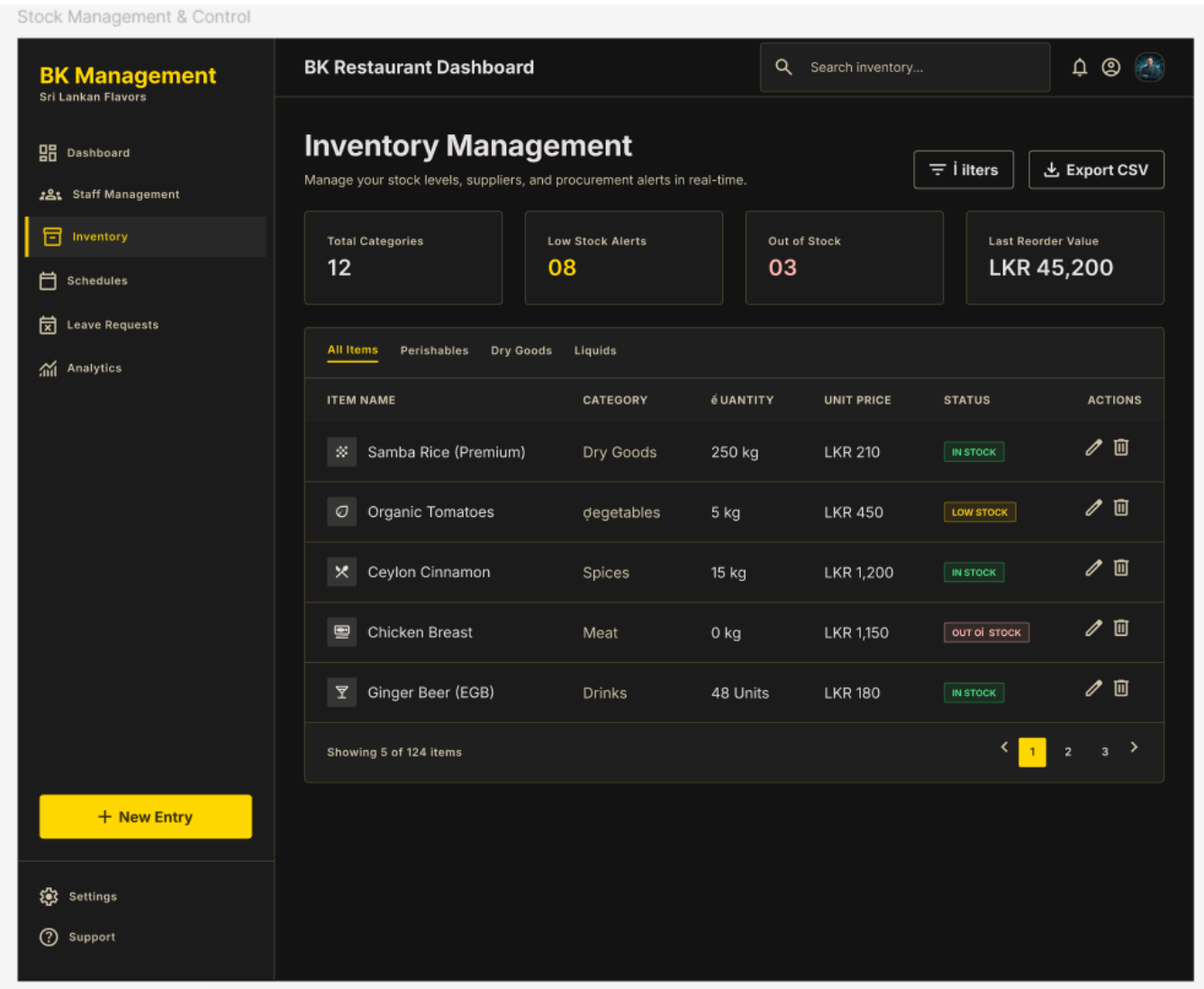


5.1.3 Inventory Related Screens

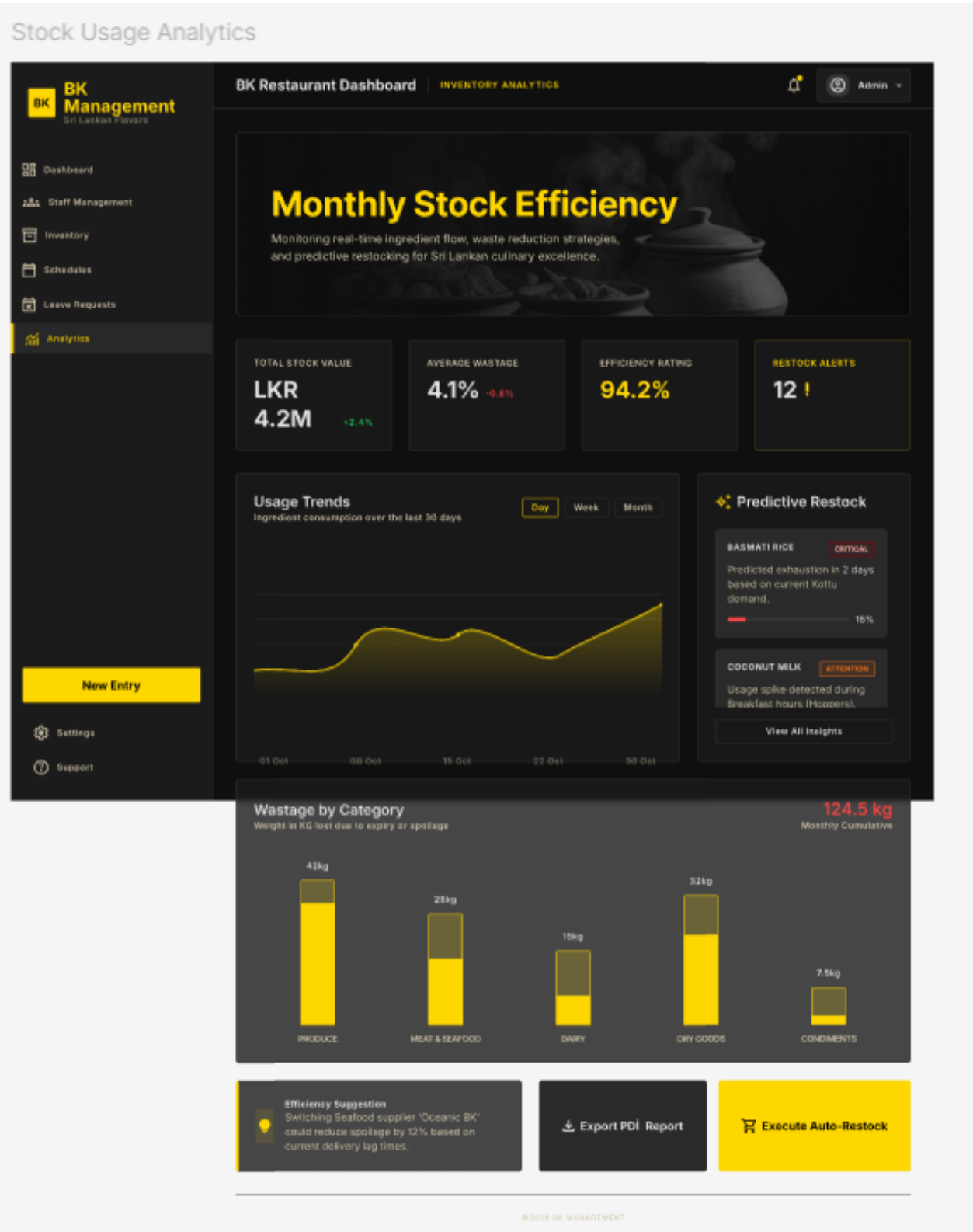
5.1.3.1 Inventory Dashboard



5.1.3.2 Stock Management & Control

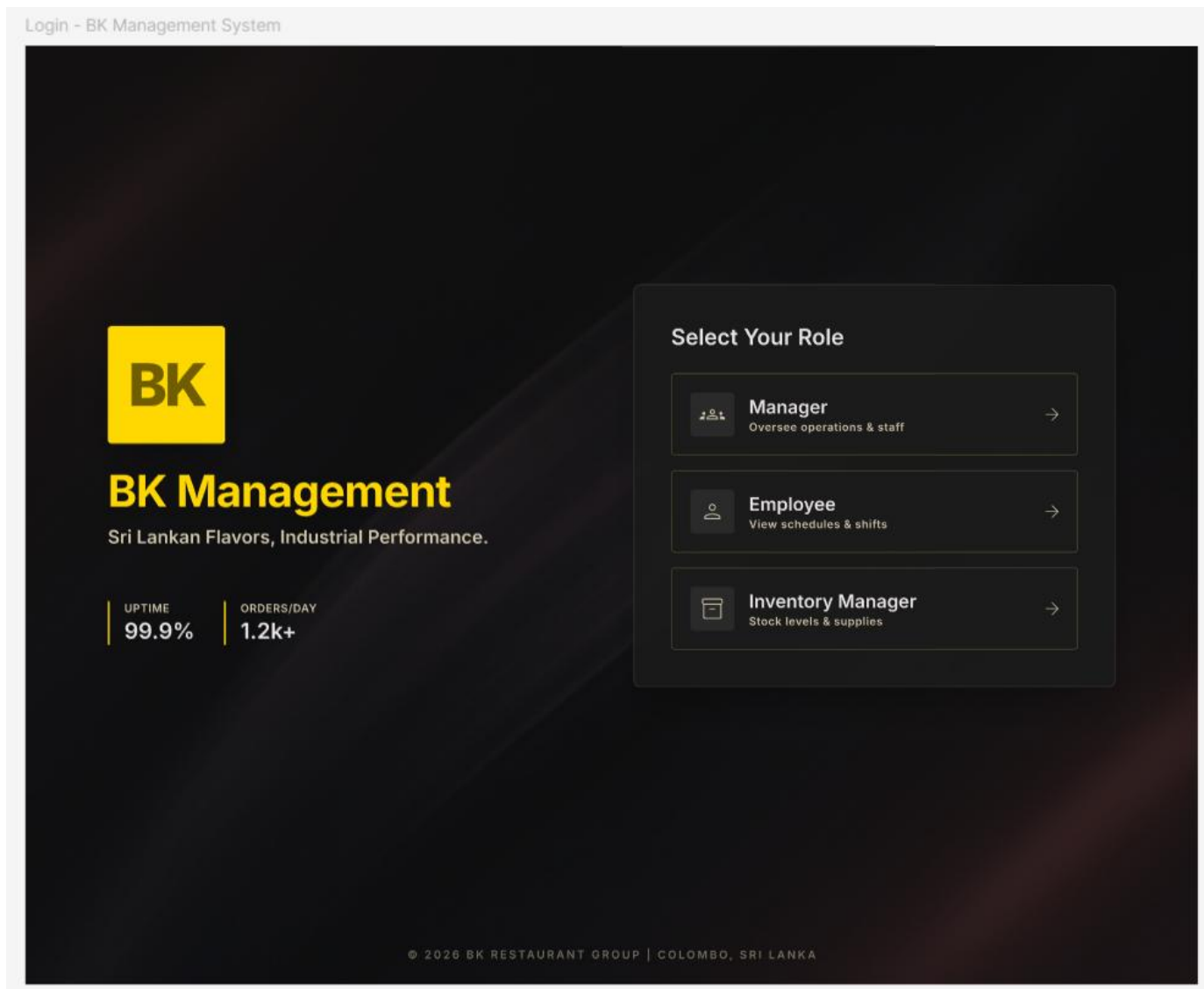


5.1.3.3 Stock Usage Analytics

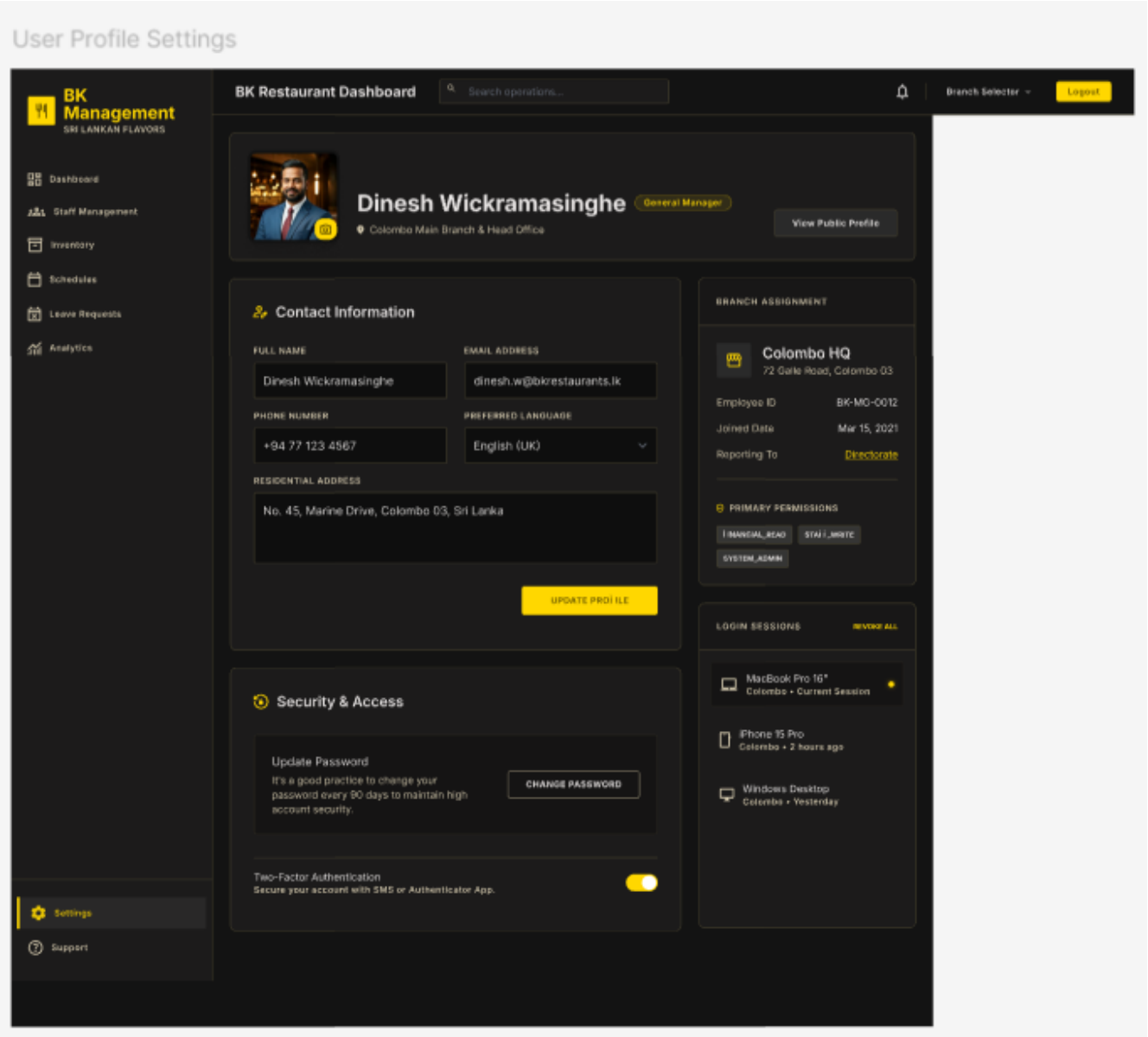


5.1.4 Common Screens

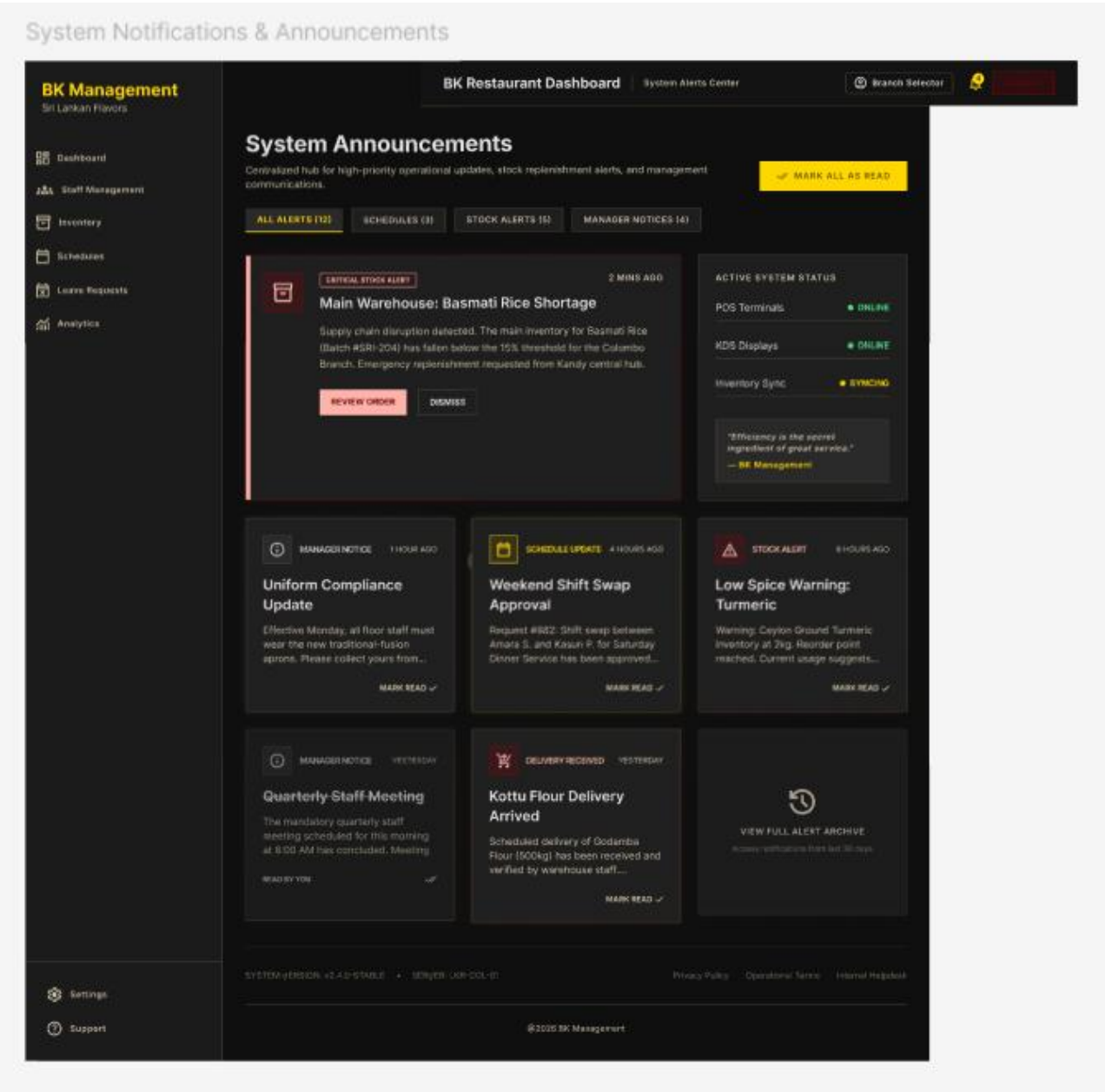
5.1.4.1 Login - BK Management System



5.1.4.2 User Profile Settings



5.1.4.3 System Notifications & Announcements

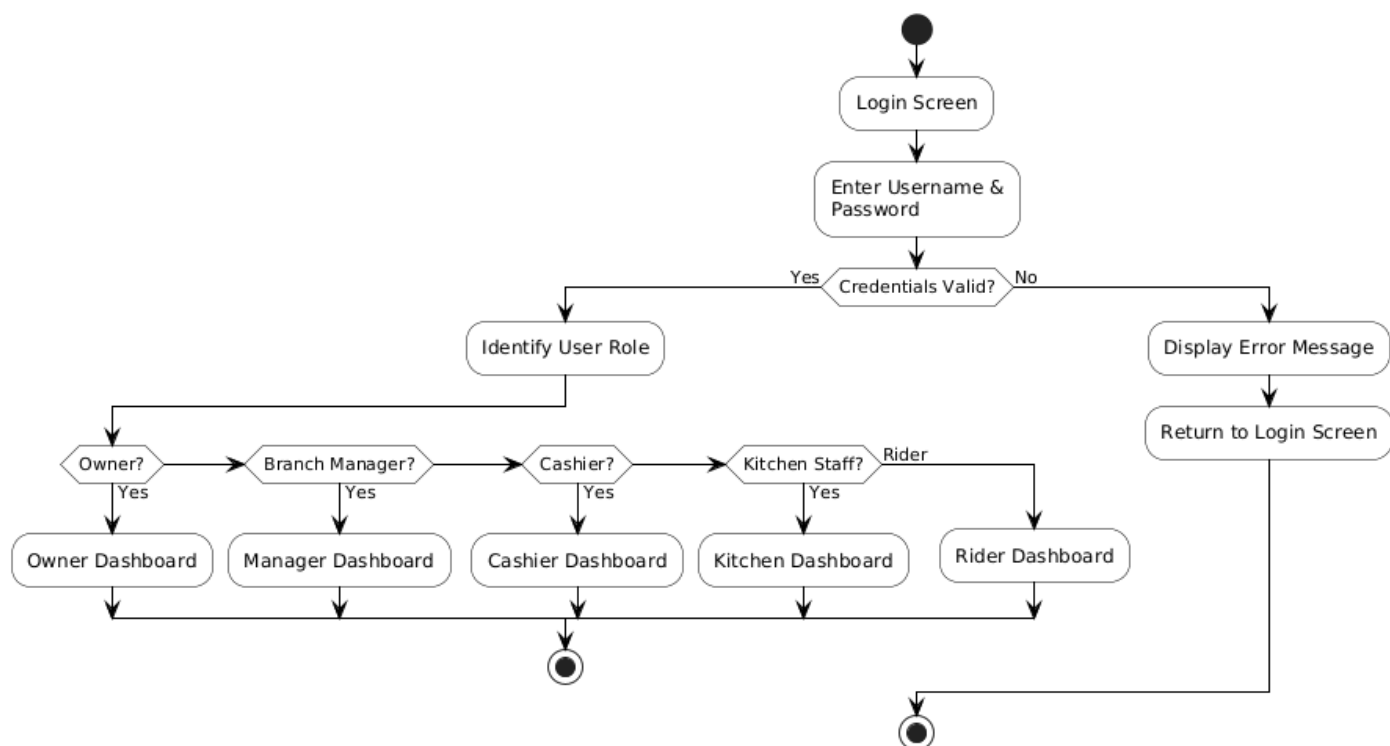


5.2 Navigation Flow Diagram

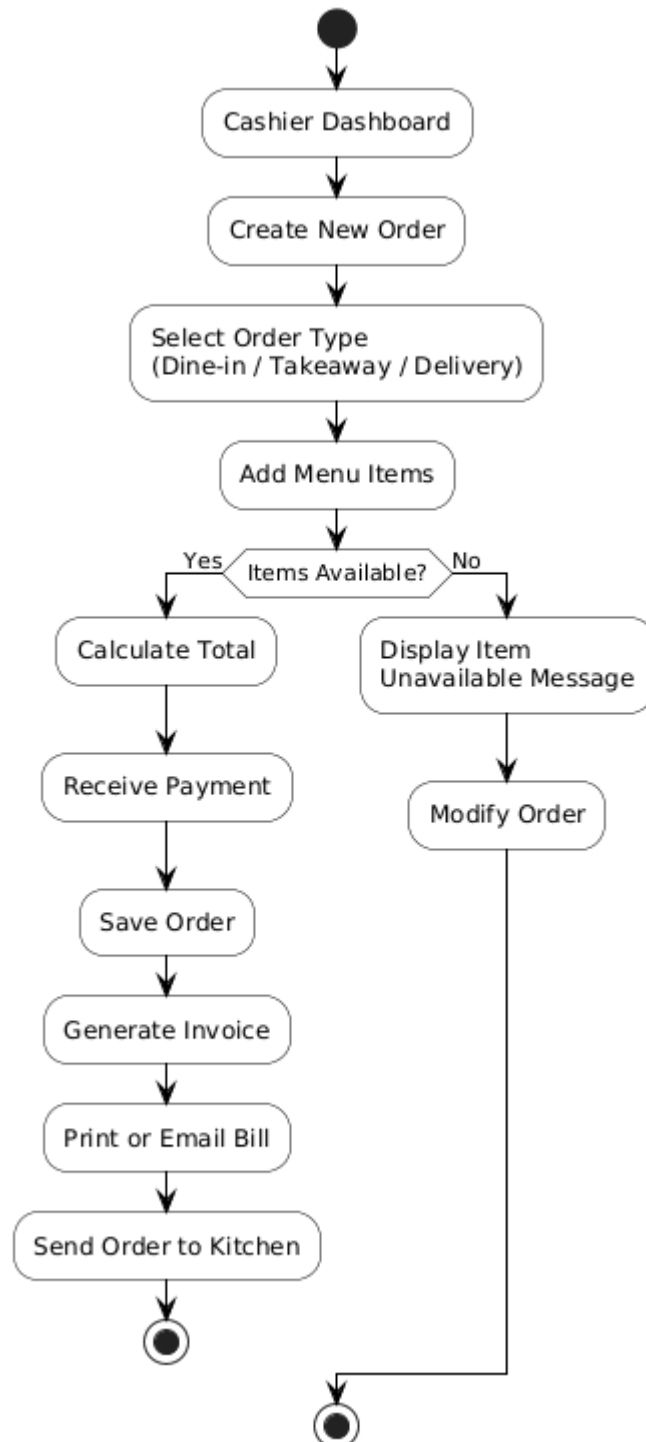
The navigation flow diagram below shows how users move between screens based on their role, from login through to each major feature area.

1. User Login & Role-Based Access Flow
2. Customer Order Creation & Billing Flow
3. Kitchen Order Processing Flow
4. Delivery Rider Flow
5. Inventory Management & Supplier Request Flow
6. Owner Monitoring & Analytics Flow

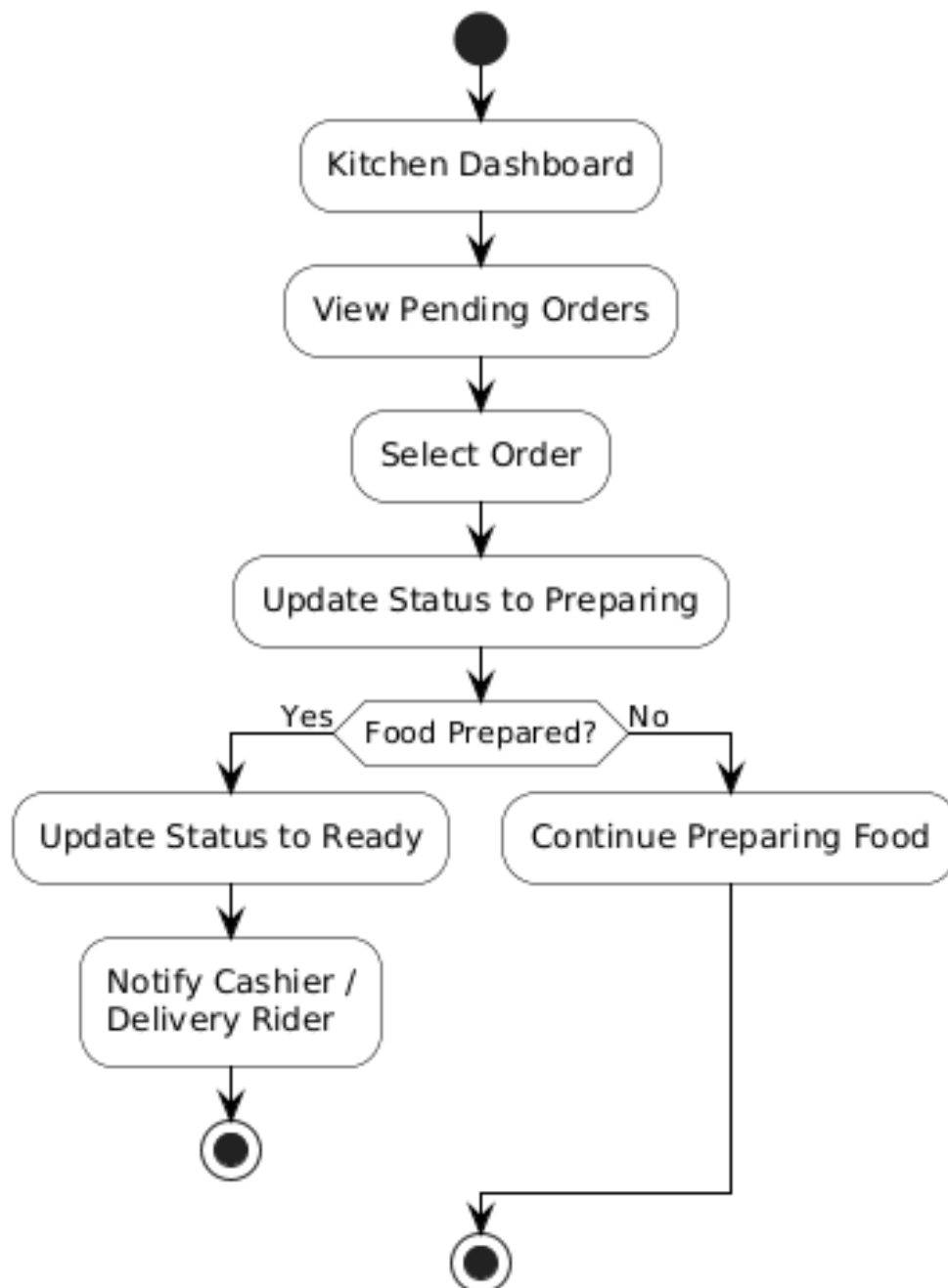
5.2.1 User Login & Role-Based Access Flow



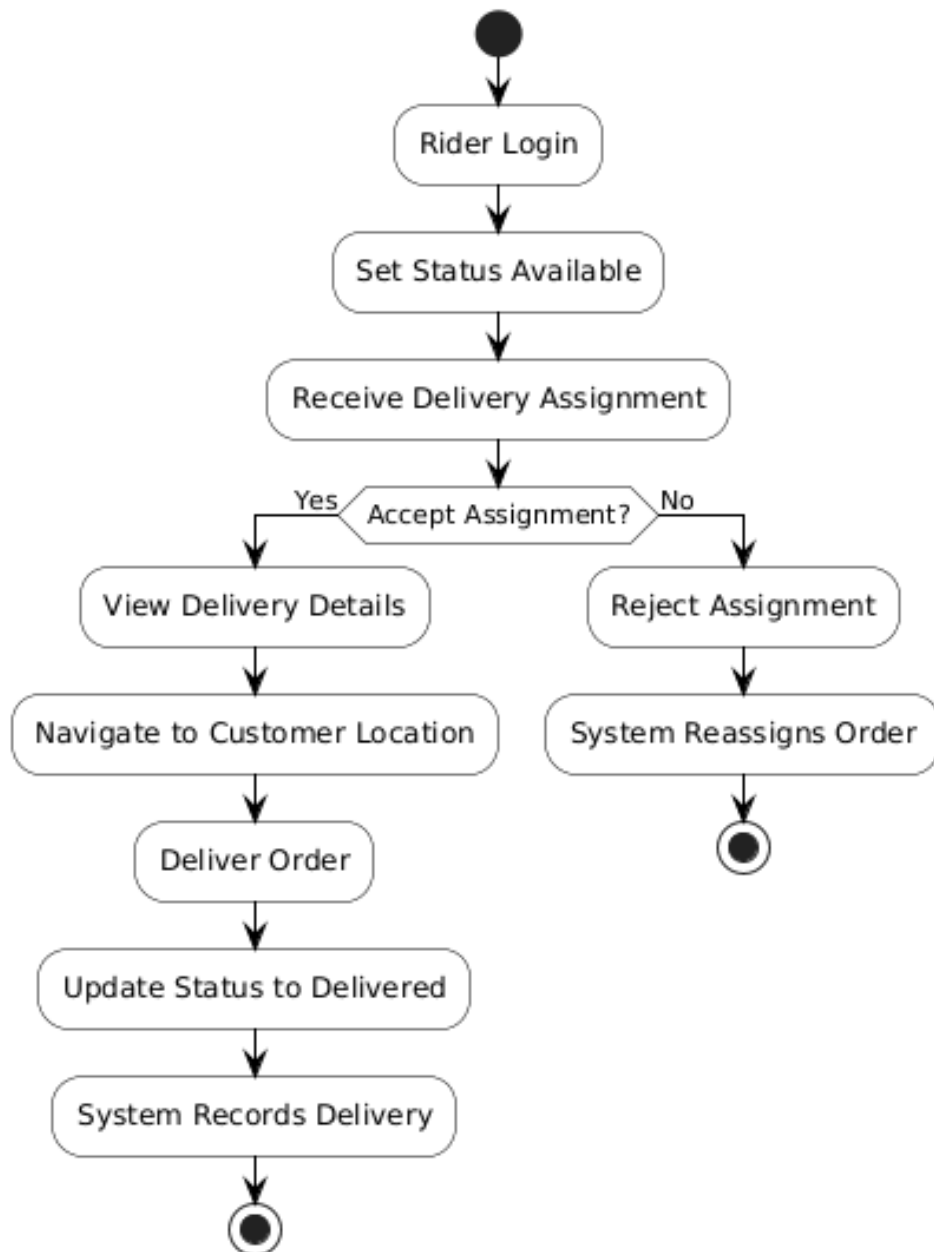
5.2.2 Customer Order Creation & Billing Flow



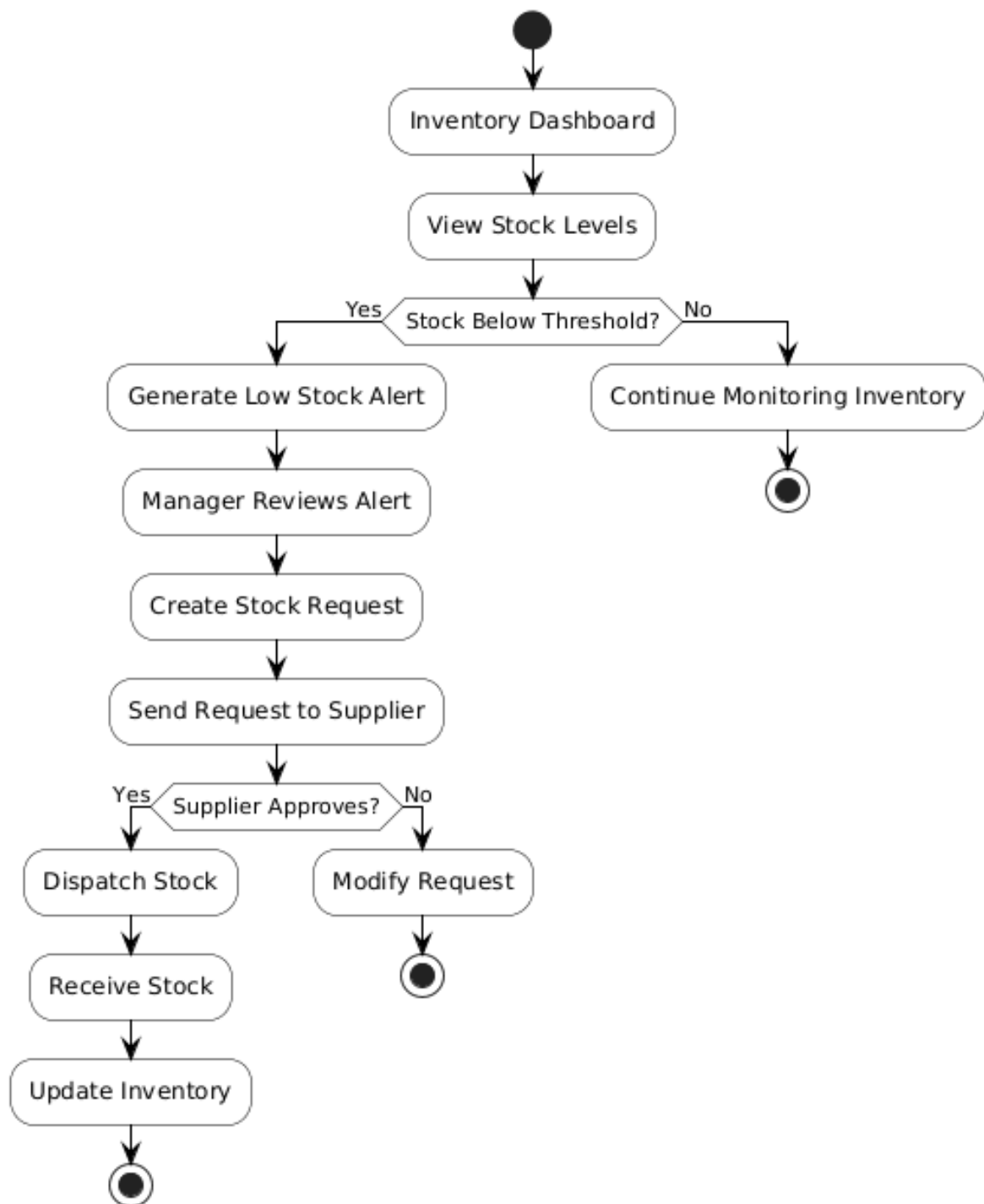
5.2.3 Kitchen Order Processing Flow



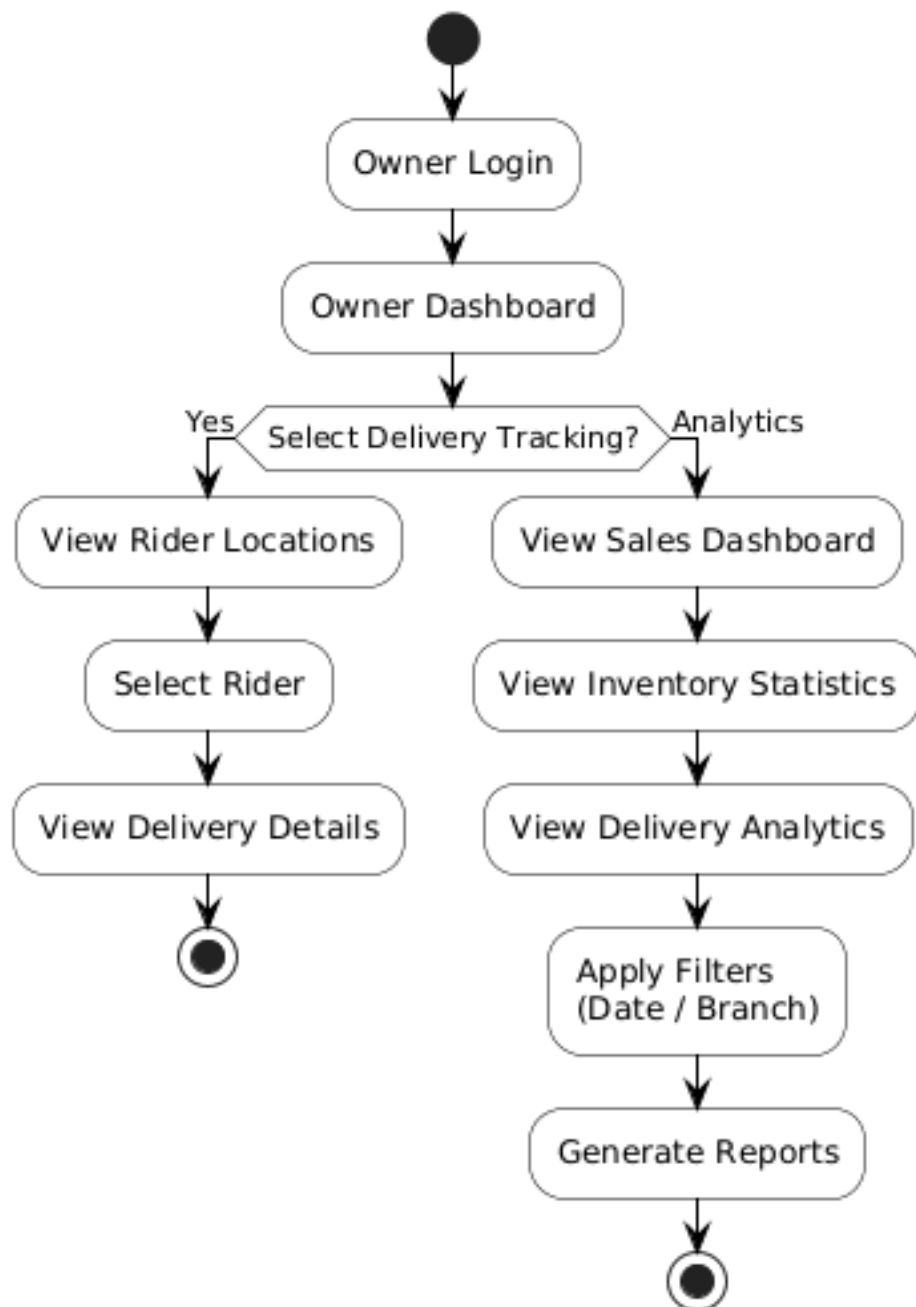
5.2.4 Delivery Rider Flow



5.2.5 Inventory Management & Supplier Request Flow



5.2.6 Owner Monitoring & Analytics Flow



5.3 UX Decisions and Rationale

UX Decision	Rationale
Large, tappable tiles for POS menu items and KDS actions	Reception and kitchen staff have low-to-medium technical proficiency (SRS Section 2.2). Large touch targets reduce input errors and support the 30-minute training target identified in the feasibility study (SRS Section 6.2.3).
Colour-coded status indicators (e.g., green/red for stock levels, order status colours on KDS)	Allows staff to recognise urgent items (low stock, new orders) at a glance without reading detailed text, supporting NFR-08 and NFR-09.
Step-by-step guided flow in the Rider App with integrated navigation	Delivery riders are smartphone-literate but benefit from minimal-decision interfaces; a guided flow with one primary action per screen reduces errors during delivery (NFR-10).
Single-page analytics dashboard with pre-built charts for the Owner	The Owner persona requires high-level visibility without manual data entry; pre-built charts and filters (date range, branch) align with the 'standard web conventions' expectation noted in the SRS feasibility study.
Consistent navigation bar/menu structure across all web screens, scoped to the logged-in user's role	Reinforces role-based access control (FR-17) at the UI level and ensures users are not shown features they cannot use, reducing confusion.
Real-time updates (e.g., new orders on KDS, delivery status on Owner Dashboard) without manual refresh	Matches the real-time processing requirements (NFR-01, NFR-03) and removes the need for staff to repeatedly refresh screens.
Confirmation messages and undo-safe flows for cancellations and stock adjustments	Reduces the risk of accidental data loss for non-technical users and supports data consistency (NFR-12).

6. Interface Design

6.1 External Interfaces (APIs, Hardware, Third-Party Services)

The system integrates with the following external interfaces:

External Interface	Purpose	Integration Notes
Google Maps Platform (Maps SDK, Directions API, Geocoding API)	Provides live GPS tracking for delivery riders, route navigation, and delivery zone validation (FR-07, UC-09).	Accessed via REST/SDK calls from the Delivery Module (backend) and the Rider App and Owner Dashboard (frontend). Requires a Google Maps API key, with usage restricted by domain/app for security.
Thermal Receipt Printer	Prints physical bills/invoices at the POS counter (UC-05).	Connected to POS terminals via USB or network printing; the Orders Module sends a formatted print job (e.g., via a printer driver/SDK or a local print server) when 'Print Invoice' is selected.
Email/SMS Notification Gateway (e.g., SendGrid for email, Twilio for SMS)	Sends digital invoices to customers, notifies suppliers of stock requests, and notifies employees of schedules (FR-05, UC-05, UC-12, UC-13).	Accessed via REST API from a shared Notification Service used by multiple backend modules; API keys stored securely as environment variables (see Section 7.2).
Cloud Hosting Platform (AWS / Railway.app)	Hosts the Node.js/Express API, PostgreSQL database, and Redis cache (SRS Section 2.3, 6.2.2).	The API and database are deployed as managed services; configuration (connection strings, secrets) is provided via environment variables, not hard-coded.
Modern Web Browsers (Chrome, Edge, Firefox, Safari)	Client runtime for the React-based POS, KDS, and Owner Dashboard (NFR-19).	The frontend uses standard, widely-supported web APIs and avoids browser-specific features to ensure cross-browser compatibility.
Android OS / GPS Hardware	Runtime environment and location sensor for the Flutter Rider App (NFR-20).	The app requests location permissions at runtime and uses background location updates while a delivery is in progress.

6.2 Internal Module Interfaces

Internal communication between frontend clients and the backend, and between backend modules, takes place over a versioned REST API (e.g., /api/v1/...) using JSON payloads and JWT-based authentication headers. The table below summarises the primary internal interfaces exposed by each backend module.

Module	Representative Endpoints	Consumed By
Authentication Module	POST /api/v1/auth/login, POST /api/v1/auth/refresh, GET /api/v1/auth/me	Web Client (POS, KDS, Owner Dashboard), Mobile Client (Rider App)
Orders Module	POST /api/v1/orders, GET /api/v1/orders/{id}, PATCH /api/v1/orders/{id}, PATCH /api/v1/orders/{id}/status, POST /api/v1/orders/{id}/finalize	POS UI, KDS UI
Inventory Module	GET /api/v1/inventory, PATCH /api/v1/inventory/{id}, GET /api/v1/inventory/alerts, POST /api/v1/stock-requests, PATCH /api/v1/stock-requests/{id}	Inventory UI (Branch Manager), Orders Module (internal call to check/deduct stock)
Delivery Module	POST /api/v1/deliveries, PATCH /api/v1/deliveries/{id}/status, GET /api/v1/deliveries/active, GET /api/v1/deliveries/{id}/location	Rider App, Owner Dashboard, Orders Module (internal call on order ready)
Employee Module	POST /api/v1/shifts, GET /api/v1/shifts, POST /api/v1/attendance, POST /api/v1/leave-requests, PATCH /api/v1/leave-requests/{id}	Employee Scheduling UI, Attendance UI
Reporting Module	GET /api/v1/analytics/sales, GET /api/v1/analytics/inventory, GET /api/v1/analytics/delivery	Owner Dashboard

Internal module-to-module calls (e.g., the Orders Module calling the Inventory Module to check stock availability, or the Orders Module calling the Delivery Module when an order becomes 'Ready for Delivery') are implemented as direct service-layer function calls within the same Node.js process for simplicity, with the option to extract them as internal HTTP/message-queue calls if the modules are later split into separate services to support NFR-14 and NFR-15.

7. Security Design

7.1 Authentication and Authorisation Strategy

The system uses a centralised Authentication Module to manage identity and access across all clients:

- Users authenticate with an email/username and password (UC-01, FR-18). Passwords are hashed using a strong, salted hashing algorithm (e.g., bcrypt) before storage, addressing NFR-05.
- On successful login, the Authentication Module issues a signed JSON Web Token (JWT) containing the user's ID, role, and branch ID. The token has a limited expiry, with a refresh-token mechanism to obtain new access tokens without re-entering credentials.
- Every protected API endpoint validates the JWT and extracts the user's role and branch ID before processing the request, addressing FR-18 and NFR-04.
- Role-Based Access Control (RBAC) is enforced at the API layer using middleware that checks the user's role against the permissions required for each endpoint (e.g., only Branch Managers and Restaurant Owners can access inventory adjustment endpoints), addressing FR-17 and NFR-06.
- Branch-level data isolation is enforced by filtering queries on branchId for roles other than Restaurant Owner, ensuring Branch Managers and staff can only access data for their own branch.
- All significant actions (order modifications/cancellations, inventory adjustments, stock request approvals, schedule changes) are recorded in an audit log table with user ID, action, timestamp, and affected entity, addressing NFR-07.

7.2 Data Protection (At Rest, In Transit)

- In Transit: all communication between clients (web, mobile) and the API occurs over HTTPS/TLS. The API does not accept plain HTTP connections in production, protecting login credentials, customer data, and order details from interception (NFR-04).
- At Rest: passwords are stored as salted bcrypt hashes, never in plain text. Sensitive configuration values (database credentials, JWT signing keys, third-party API keys) are stored as environment variables/secrets, not committed to source control.
- Database Access Control: the database is not exposed directly to the public internet; only the backend API server can connect to it, using a dedicated database user with the minimum privileges required.
- Backups: scheduled automated backups of the PostgreSQL database are taken (e.g., daily) and stored securely, with a documented restore procedure, addressing NFR-13.
- Personal Data: customer contact details (for delivery orders) and employee personal data (attendance, leave) are only accessible to roles that require them for their job function, consistent with the RBAC model in Section 7.1.

7.3 Input Validation Strategy

- All API endpoints validate incoming request bodies against a defined schema (e.g., using a validation library such as Joi or class-validator) before any business logic executes, rejecting malformed or unexpected fields.

- Numeric fields (quantities, prices, totals) are validated to be non-negative and within sensible bounds; the Orders Module enforces that order totals must be greater than zero (per the Business Rule in UC-02).
- Status transition fields (Order.status, Delivery.status, StockRequest.status) are validated against the allowed state machine transitions defined in Section 4.3, preventing invalid transitions (e.g., directly from 'Pending' to 'Completed').
- All database queries use parameterised queries or an ORM's query builder, never string concatenation, to prevent SQL injection.
- File and text inputs (e.g., adjustment reasons, leave request reasons) are length-limited and sanitised before storage and before being rendered in the UI to prevent stored cross-site scripting (XSS).
- GPS coordinates submitted by the Rider App are validated to be within plausible ranges and are cross-checked against the delivery address before a delivery can be marked 'Delivered' (per the Business Rule in UC-08).

7.4 OWASP Top 10 Considerations

OWASP Top 10 Category	Mitigation in this Design
A01: Broken Access Control	Centralised RBAC middleware (Section 7.1) checks role and branch ownership on every request; resource ownership is verified before allowing updates (e.g., a Branch Manager cannot modify another branch's inventory).
A02: Cryptographic Failures	Passwords hashed with bcrypt; all traffic over HTTPS/TLS; JWT signed with a strong secret/key stored outside source control (Section 7.2).
A03: Injection	Parameterised queries/ORM usage and schema-based input validation on all endpoints (Section 7.3) prevent SQL injection and similar attacks.
A04: Insecure Design	Status transitions for Order and Delivery follow explicit state machines (Section 4.3) so invalid business states cannot be reached via crafted requests.
A05: Security Misconfiguration	Environment-based configuration for secrets and database connections; production deployments disable debug endpoints and verbose error messages.
A06: Vulnerable and Outdated Components	Dependencies (React, Express, Flutter packages) are kept up to date and monitored using standard tooling (e.g., npm audit, Dependabot) as part of maintenance (NFR-17, NFR-18).
A07: Identification and Authentication Failures	JWT-based authentication with expiring tokens and refresh tokens; account lockout/rate-limiting can be applied to the login endpoint to mitigate brute-force attempts.
A08: Software and Data Integrity Failures	Audit logging (Section 7.1) records changes to critical records (orders, inventory, stock requests) to detect unauthorised data changes.

A09: Security Logging and Monitoring Failures	Audit logs and application logs capture authentication attempts, authorisation failures, and key business events, supporting later monitoring and incident investigation.
A10: Server-Side Request Forgery (SSRF)	External calls (Maps API, notification gateway) use fixed, configuration-defined endpoints rather than user-supplied URLs, preventing SSRF via untrusted input.

8. Non-Functional Design Decisions

This section maps each Non-Functional Requirement (NFR) from the SRS (Section 5) to the specific design decision in this SDS that addresses it.

(i) Performance		
NFR ID	Category	Design Decision Addressing the NFR
NFR-01	Response Time	Stateless API layer (Section 3.1) with Redis caching for menu and dashboard data (Section 3.5) keeps POS order update response times within the 2-second target.
NFR-02	Concurrent Access	Horizontally scalable, stateless Express API behind a load balancer, with a single shared PostgreSQL database and connection pooling, supports concurrent access from all branches.
NFR-03	Real-Time Processing	Real-time event notifications (e.g., WebSocket or push-based updates) are used for order status changes (SD-04), delivery status changes (SD-08), and inventory updates (SD-10/SD-11).
(ii) Security		
NFR ID	Category	Design Decision Addressing the NFR
NFR-04	User Authentication	JWT-based authentication over HTTPS for all clients (Section 7.1, 7.2).
NFR-05	Password Encryption	Passwords stored as salted bcrypt hashes (Section 7.1, 7.2).
NFR-06	Role-Based Restriction	RBAC middleware enforced on every API endpoint, scoped by role and branch (Section 7.1).
NFR-07	Audit Logs	Audit log table records key user actions with user ID, timestamp, and affected entity (Section 7.1).
(iii) Usability		
NFR ID	Category	Design Decision Addressing the NFR
NFR-08	Simple UI	Large tappable tiles, colour-coded statuses, and minimal-input forms across POS, KDS, and Inventory screens (Section 5.3).
NFR-09	Clear Navigation	Consistent, role-scoped navigation structure across all web screens, defined in the navigation flow diagram (Section 5.2).
NFR-10	Rider App Usability	Step-by-step guided delivery flow with integrated map navigation and single-action screens in the Rider App (Section 5.1.10, 5.1.11, 5.3).
(iv) Reliability		

NFR ID	Category	Design Decision Addressing the NFR
NFR-11	Offline Sync	POS and Rider App clients can queue actions locally (e.g., order creation, status updates) and synchronise with the API once connectivity is restored, with conflict handling defined at the API layer.
NFR-12	Data Consistency	All order, inventory, and delivery updates are performed as database transactions (e.g., deducting inventory and creating an order atomically) to maintain consistent records across branches.
NFR-13	Backup & Recovery	Scheduled automated database backups with a documented restore procedure (Section 7.2).
(v) Scalability		
NFR ID	Category	Design Decision Addressing the NFR
NFR-14	Branch Expansion	New branches are added as new Branch records with associated users, menu items, and inventory; no architectural change is required (Section 2.2, 3.1).
NFR-15	Load Scalability	Stateless API design allows horizontal scaling of API instances; Redis caching reduces database load as the number of users, orders, and inventory records grows (Section 3.1, 3.5).
(vi) Maintainability		
NFR ID	Category	Design Decision Addressing the NFR
NFR-16	Modular Architecture	Backend organised into independent feature modules (Authentication, Orders, Inventory, Delivery, Employees, Reporting), each with its own routes, controllers, and services (Section 3.1).
NFR-17	Coding Standards	Consistent project structure, linting, and code review practices are applied across modules; dependency versions are tracked and updated (Section 7.4, A06).
NFR-18	Non-Disruptive Updates	Stateless API instances allow rolling deployments (deploying new instances and retiring old ones) without taking the whole system offline.
(vii) Compatibility		
NFR ID	Category	Design Decision Addressing the NFR
NFR-19	Browser Support	React/Tailwind frontend built using standard, cross-browser-compatible web technologies, tested on Chrome, Edge, and Firefox (Section 6.1).
NFR-20	Android Support	Rider App built with Flutter, targeting Android smartphones with GPS capability (Section 3.5, 6.1).
(viii) Availability		

NFR ID	Category	Design Decision Addressing the NFR
NFR-21	Operating Hours	Cloud hosting on AWS/Railway.app with managed uptime, combined with offline-tolerant clients (NFR-11), minimises downtime during operating hours.
NFR-22	Continuous Operation	A single shared backend and database serve all branches continuously; horizontal scaling and rolling deployments (NFR-15, NFR-18) avoid system-wide outages during updates.